# EMV®
# Contactless Card Level 1
# Type Approval

## PICC Digital Test Cases

Version 3.2b.r
September 2025

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com.
EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# Legal Notice

This document summarizes EMVCo's present plans for evaluation services and related policies and is subject to change by EMVCo at any time. This document does not create any binding obligations upon EMVCo or any third party regarding the subject matter of this document, which obligations will exist, if at all, only to the extent set forth in separate written agreements executed by EMVCo or such third parties. In the absence of such a written agreement, no product provider, test laboratory or any other third party should rely on this document, and EMVCo shall not be liable for any such reliance.

No product provider, test laboratory or other third party may refer to a product, service or facility as EMVCo approved, in form or in substance, nor otherwise state or imply that EMVCo (or any agent of EMVCo) has in whole or part approved a product provider, test laboratory or other third party or its products, services, or facilities, except to the extent and subject to the terms, conditions and restrictions expressly set forth in a written agreement with EMVCo, or in an approval letter, compliance certificate or similar document issued by EMVCo. All other references to EMVCo approval are strictly prohibited by EMVCo.

Under no circumstances should EMVCo approvals, when granted, be construed to imply any endorsement or warranty regarding the security, functionality, quality, or performance of any particular product or service, and no party shall state or imply anything to the contrary. EMVCo specifically disclaims any and all representations and warranties with respect to products that have received evaluations or approvals, and to the evaluation process generally, including, without limitation, any implied warranties of merchantability, fitness for purpose or non-infringement. All warranties, rights and remedies relating to products and services that have undergone evaluation by EMVCo are provided solely by the parties selling or otherwise providing such products or services, and not by EMVCo, and EMVCo will have no liability whatsoever in connection with such products and services.

This document is provided "AS IS" without warranties of any kind, and EMVCo neither assumes nor accepts any liability for any errors or omissions contained in this document. EMVCO DISCLAIMS ALL REPRESENTATIONS AND WARRANTIES, EXPRESS OR IMPLIED, INCLUDING WITHOUT LIMITATION IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AS TO THIS DOCUMENT.

EMVCo makes no representations or warranties with respect to intellectual property rights of any third parties in or in relation to this document. EMVCo undertakes no responsibility to determine whether any implementation of this document may violate, infringe, or otherwise exercise the patent, copyright, trademark, trade secret, know-how, or other intellectual property rights of third parties, and thus any person who implements any part of this document should consult an intellectual property attorney before any such implementation.

Without limiting the foregoing, this document may provide for the use of public key encryption and other technology, which may be the subject matter of patents in several countries. Any party seeking to implement this document is solely responsible for determining whether its activities require a license to any such technology, including for patents on public key encryption technology. EMVCo shall not be liable under any theory for any party's infringement of any intellectual property rights in connection with this document.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# Revision Log – Version 3.2b.r

The following changes have been made to the document since the publication of Version 3.2b. Some of the numbering and cross references in this version have been updated to reflect changes introduced by the published bulletins. The numbering of existing requirements did not change, unless explicitly stated otherwise.

**Changes based on Specification Updates:**

**Other changes (test coverage, test clarification, editorial correction):**

- Editorial updates

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# Contents

**1. About this Document ..... 1**
1.1. Introduction ..... 1
1.2. Scope ..... 1
1.3. Reference Documents ..... 1
1.3.1. Specification Documents ..... 1
1.3.2. Other Reference Documents ..... 2
1.4. Acronyms and Abbreviations ..... 3

**2. Generic Information about the Tests ..... 6**
2.1. ICS (Implementation Conformance Statement) and Application Selection ..... 6
2.2. Default Environmental Test Conditions ..... 7
2.3. Default Tool Requirements ..... 7
2.4. Default Timings ..... 8
2.5. Definition for Timing Measurement ..... 9
2.6. Scenario Notations during Half Duplex Protocol ..... 10
2.7. Types of Frames in the Scenario ..... 11
2.8. Notational Conventions ..... 11
2.9. Tests References ..... 12
2.10. Basic Installation ..... 12
2.11. Pre- and Post-Processing ..... 14
2.12. Transaction Completion ..... 15
2.13. Loopback Command Description ..... 16
2.14. Parameters Values ..... 17

**3. Type A Executable Tests ..... 20**
3.1. Basic Type A Exchange and Timings Measurement (UID size = 1) [CA001] ..... 20
3.2. Basic Type A Exchange and Timings Measurement (UID size = 2) [CA002] ..... 24
3.3. Basic Type A Exchange and Timings Measurement (UID size = 3) [CA003] ..... 28
3.4. Basic Type A Exchange with the minimum and longer Frame Delay Times PCD→PICC (UID size = 1) [CA010.XY] ..... 32
3.5. Basic Type A Exchange with the minimum and longer Frame Delay Times PCD→PICC (UID size = 2) [CA011.XY] ..... 36
3.6. Basic Type A Exchange with the minimum and longer Frame Delay Times PCD→PICC (UID size = 3) [CA012.XY] ..... 41
3.7. Type A Correct Installation with respect of the anticollision state machine (UID size = 1) [CA116] ..... 46
3.8. Type A Correct Installation with respect of the anticollision state machine (UID size = 2) [CA117] ..... 48
3.9. Type A Correct Installation with respect of the anticollision state machine (UID size = 3) [CA118] ..... 50
3.10. Type A Handling of RATS (UID size = 1) [CA120.x] ..... 52
3.11. Type A Handling of RATS (UID size = 2) [CA121.x] ..... 54
3.12. Type A Handling of RATS (UID size = 3) [CA122.x] ..... 56
3.13. Type A Consecutive Installations (UID size = 1, fixed UID) [CA126.x] ..... 58

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

3.14. Type A Consecutive Installations (UID size = 2) [CA127.x] ..... 60
3.15. Type A Consecutive Installations (UID size = 3) [CA128.x] ..... 62
3.16. Type A Handling of the REQA Command (UID size = 1) [CA130.x] ..... 65
3.17. Type A Handling of the REQA Command (UID size = 2) [CA131.x] ..... 71
3.18. Type A Handling of the REQA Command (UID size = 3) [CA132.x] ..... 77
3.19. Type A Installation with error in the IDLE state [CA204.x] ..... 84
3.20. Type A Installation with error in the READY state [CA206.xy] ..... 86
3.21. Type A Installation with error in the READY' state (UID size = 2 or 3) [CA207.xy] ..... 89
3.22. Type A Installation with error in the READY'' state (UID size = 3) [CA208.xy] ..... 92
3.23. Type A Installation with error in the ACTIVE state (UID size = 1) [CA216.xy] ..... 95
3.24. Type A Installation with error in the ACTIVE state (UID size = 2) [CA217.xy] ..... 97
3.25. Type A Installation with error in the ACTIVE state (UID size = 3) [CA218.xy] ..... 100
3.26. Type A Installation with error in the HALT state following ACTIVE state (UID size = 1) [CA226] ..... 103
3.27. Type A Installation with error in the HALT state following ACTIVE state (UID size = 2) [CA227] ..... 105
3.28. Type A Installation with error in the HALT state following ACTIVE state (UID size = 3) [CA228] ..... 108
3.29. Type A Installation with error in the HALT state following PROTOCOL State (UID size = 1) [CA230] ..... 111
3.30. Type A Installation with error in the HALT state following PROTOCOL State (UID size = 2) [CA231] ..... 113
3.31. Type A Installation with error in the HALT state following PROTOCOL State (UID size = 3) [CA232] ..... 116
3.32. Type A Installation with polling and with PICC Reset [CA233.xy] ..... 119

**4. Type B Executable Tests ..... 121**
4.1. Basic Type B Exchange and Timings Measurement [CB001] ..... 121
4.2. Basic Type B Exchange with the minimum and longer Frame Delay Times PCD→PICC [CB010.xy] ..... 125
4.3. Basic Type B Exchange with the minimum supported SFGT PCD→PICC [CB015.x] ..... 129
4.4. Basic Type B Exchange with the minimum and the maximum values of $EGT_{PCD}$ [CB021.x] ..... 131
4.5. Basic Type B Exchange with minimum and maximum durations of (SoS) and (EoS) [CB025.x] ..... 133
4.6. Type B Correct Installation with respect of the anticollision state machine [CB116] ..... 135
4.7. Type B Consecutive Installations (fixed PUPI) [CB126.x] ..... 137
4.8. Type B Handling of the REQB Command [CB130.x] ..... 139
4.9. Type B Correct Installation with C-APDU sent in the ATTRIB Command Higher Layer INF field [CB150] ..... 141
4.10. Type B Correct Installation with RFU values or values to be ignored [CB155.x] ..... 143
4.11. Type B Installation with error in the IDLE state [CB204.x] ..... 145
4.12. Type B Installation with error in the READY state [CB206.xy] ..... 148

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

4.13. Type B Installation with error in the HALT state following the READY state [CB226.x] ..... 151
4.14. Type B Installation with error in the HALT state following the ACTIVE State [CB227.x] ..... 154
4.15. Type B Installation with WUPA command in the HALT state [CB228.x] ..... 157
4.16. Type B Installation with polling and with PICC Reset [CB233.xy] ..... 160

**5. Block Protocol Executable Tests ..... 162**
5.1. Support of the EMV CL Polling [CC001.x] ..... 162
5.2. Reception of an empty I-Block or a R(NAK) block [CC101.x] ..... 168
5.3. Support of presence check procedure [CC102] ..... 170
5.4. Reception of chained I-Blocks from the PCD [CC110.x] ..... 172
5.5. Emission of APDUs with different FSDI values [CC120.x] ..... 182
5.6. Error notification on an I-Block not indicating chaining [CC203.x] ..... 184
5.7. Error after reception of a non-chained I-Block [CC206.xy] ..... 187
5.8. Management of b2 in the PCB of S-Blocks [CC207] ..... 193
5.9. Error notification and error after reception of a chained I-Block [CC210.xy] ..... 195
5.10. Block Protocol with PICC Reset [CC233.xy] ..... 207
5.11. Installation commands during Block Protocol [CC241.x] ..... 209
5.12. Error at the beginning of the Block Protocol [CC245.x] ..... 212

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# Scenarios

Scenario 1: Basic Type A Exchange and Timings Measurement (UID size = 1) ..... 23
Scenario 2: Basic Type A Exchange and Timings Measurement (UID size = 2) ..... 27
Scenario 3: Basic Type A Exchange and Timings Measurement (UID size = 3) ..... 31
Scenario 4: Basic Type A Exchange with the minimum and longer Frame Delay Time PCD→PICC (UID size = 1, xy=00 to xy=01) ..... 34
Scenario 5: Basic Type A Exchange with the minimum Frame Delay Time PCD→PICC (UID size = 1, xy=10) ..... 35
Scenario 6: Basic Type A Exchange with the minimum and longer Frame Delay Time PCD→PICC (UID size = 2, xy=00 to xy=01) ..... 39
Scenario 7: Basic Type A Exchange with the minimum Frame Delay Time PCD→PICC (UID size = 2, xy=10) ..... 40
Scenario 8: Basic Type A Exchange with the minimum and longer Frame Delay Time PCD→PICC (UID size = 3, xy=00 to xy=01) ..... 43
Scenario 9: Basic Type A Exchange with the minimum Frame Delay Time PCD→PICC (UID size = 3, xy=10) ..... 45
Scenario 10: Type A Correct Installation with respect of the anticollision state machine (UID size = 1) ..... 47
Scenario 11: Type A Correct Installation with respect of the anticollision state machine (UID size = 2) ..... 49
Scenario 12: Type A Correct Installation with respect of the anticollision state machine (UID size = 3) ..... 51
Scenario 13: Type A Handling of RATS (UID size x = 1, x=0 to 2) ..... 53
Scenario 14: Type A Type A Handling of RATS (UID size = 2, x=0 to 2) ..... 55
Scenario 15: Type A Type A Handling of RATS (UID size = 3, x=0 to 2) ..... 57
Scenario 16: Type A Consecutive Installations (UID size = 1, fixed UID, x=0 to 1) ..... 59
Scenario 17: Type A Consecutive Installations (UID size = 2, x=0 to 1) ..... 61
Scenario 18: Type A Consecutive Installations (UID size = 3, x=0 to 1) ..... 64
Scenario 19: Type A Handling of the REQA Command (UID size = 1, x=0) ..... 66
Scenario 20: Type A Handling of the REQA Command (UID size = 1, x=1) ..... 66
Scenario 21: Type A Handling of the REQA Command (UID size = 1, x=2) ..... 67
Scenario 22: Type A Handling of the REQA Command (UID size = 1, x=3) ..... 68
Scenario 23: Type A Handling of the REQA Command (UID size = 1, x=4) ..... 69
Scenario 24: Type A Handling of the REQA Command (UID size = 1, x=5) ..... 70
Scenario 25: Type A Handling of the REQA Command (UID size = 2, x=0) ..... 72
Scenario 26: Type A Handling of the REQA Command (UID size = 2, x=1) ..... 72
Scenario 27: Type A Handling of the REQA Command (UID size = 2, x=2) ..... 73
Scenario 28: Type A Handling of the REQA Command (UID size = 2, x=3) ..... 74
Scenario 29: Type A Handling of the REQA Command (UID size = 2, x=4) ..... 75
Scenario 30: Type A Handling of the REQA Command (UID size = 2, x=5) ..... 76
Scenario 31: Type A Handling of the REQA Command (UID size = 3, x=0) ..... 78
Scenario 32: Type A Handling of the REQA Command (UID size = 3, x=1) ..... 79
Scenario 33: Type A Handling of the REQA Command (UID size = 3, x=2) ..... 80
Scenario 34: Type A Handling of the REQA Command (UID size = 3, x=3) ..... 81
Scenario 35: Type A Handling of the REQA Command (UID size = 3, x=4) ..... 82
Scenario 36: Type A Handling of the REQA Command (UID size = 3, x=5) ..... 83
Scenario 37: Type A Installation with error in the IDLE state (x=0 and 1) ..... 85
Scenario 38: Type A Installation with error in the READY state (xy=00 to 16) ..... 88
Scenario 39: Type A Installation with error in the READY' state (UID size = 2 or 3 / xy=00 to 16) ..... 91
Scenario 40: Type A Installation with error in the READY'' state (UID size = 3 / xy=00 to 16) ..... 94

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

Scenario 41: Type A Installation with error in the ACTIVE state (UID size = 1 / xy=00 to 10) ..... 96
Scenario 42: Type A Installation with error in the ACTIVE state (UID size = 2 / xy=00 to 10) ..... 99
Scenario 43: Type A Installation with error in the ACTIVE state (UID size = 3 / xy=00 to 10) ..... 102
Scenario 44: Type A Installation with error in the HALT state following ACTIVE state (UID size = 1) ..... 104
Scenario 45: Type A Installation with error in the HALT state following ACTIVE State (UID size = 2) ..... 107
Scenario 46: Type A Installation with error in the HALT state following ACTIVE State (UID size = 3) ..... 110
Scenario 47: Type A Installation with error in the HALT state following PROTOCOL state (UID size = 1) ..... 112
Scenario 48: Type A Installation with error in the HALT state following PROTOCOL State (UID size = 2) ..... 115
Scenario 49: Type A Installation with error in the HALT state following PROTOCOL State (UID size = 3) ..... 118
Scenario 50: Type A Installation with polling and with PICC Reset (xy=00, 01, 10 and 11) 120
Scenario 51: Basic Type B Exchange and Timings Measurement ..... 124
Scenario 52: Basic Type B Exchange with the minimum and longer Frame Delay Time PCD→PICC (xy=00 to 02) ..... 127
Scenario 53: Basic Type B Exchange with the minimum and longer Frame Delay Time PCD→PICC (xy=10) ..... 128
Scenario 54: Basic Type B Exchange with the minimum supported SFGT PCD→PICC (x=0 and 1) ..... 130
Scenario 55: Basic Type B Exchange with the minimum and the maximum value of $EGT_{PCD}$ (x=0 to 1) ..... 132
Scenario 56: Basic Type B Exchange with the minimum and the maximum durations of the (SoS) and (EoS) sequences (x=0 to 1) ..... 134
Scenario 57: Type B Correct Installation with respect of the anticollision state machine ... 136
Scenario 58: Type B Consecutive Installations (fixed PUPI, x=0 to 1) ..... 138
Scenario 59: Type B Handling of the REQB Command (x=0) ..... 140
Scenario 60: Type B Handling of the REQB Command (x=1) ..... 140
Scenario 61: Type B Correct Installation with C-APDU sent in the ATTRIB Command Higher Layer INF field ..... 142
Scenario 62: Type B Correct Installation with RFU values (x=0 to 2 and 7 to 8). ..... 144
Scenario 63: Type B Installation with error in the IDLE state (x=0 to 3) ..... 147
Scenario 64: Type B Installation with error in the READY state (x=00 to 05, 07 to 11 and 14 to 18) ..... 150
Scenario 65: Type B Installation with error in the HALT state (x=0 and 1) ..... 153
Scenario 66: Type B Installation with error in the HALT state (x=0 and 1) ..... 156
Scenario 67: Type B Installation with WUPA command in the HALT state (PUPI is a fixed number / x=0) ..... 158
Scenario 68: Type B Installation with WUPA command in the HALT state (PUPI is not a fixed number (random PUPI) / x=1) ..... 159
Scenario 69: Type B Installation with polling and with PICC Reset (xy=00, 01, 10, 11, 20, 21, 30 and 31) ..... 161
Scenario 70: Support of the EMV CL Polling (x=0) ..... 164
Scenario 71: Support of the EMV CL Polling (x=1) ..... 165
Scenario 72: Support of the EMV CL Polling (x=2) ..... 166
Scenario 73: Support of the EMV CL Polling (x=3) ..... 167
Scenario 74: Reception of an empty I-Block or a R(NAK) block (x=0) ..... 169
Scenario 75: Reception of an empty I-Block or a R(NAK) (x=1) ..... 169

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

Scenario 76: Support of presence check procedure ..... 171
Scenario 77: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 2 / x=0) ..... 174
Scenario 78: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 3 / x=1) ..... 175
Scenario 79: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 4 / x=2) ..... 176
Scenario 80: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 5 / x=3) ..... 178
Scenario 81: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 6 / x=4) ..... 179
Scenario 82: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 7 / x=5) ..... 180
Scenario 83: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 8 / x=6) ..... 181
Scenario 84: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 9 / x=7) ..... 181
Scenario 85: Emission of APDUs with different FSDI values (x=0 to 7) ..... 183
Scenario 86: Error notification on an I-Block not indicating chaining (x=0 and 1) ..... 186
Scenario 87: Error after an I-Block not indicating chaining (xy=00 to 6 and 8 to 15) ..... 192
Scenario 88: Management of b2 in the PCB of S-Blocks ..... 194
Scenario 89: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 2 / xy=00 or 01) ..... 197
Scenario 90: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 3 / xy=10 or 11) ..... 198
Scenario 91: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 4 / xy=20 or 21) ..... 200
Scenario 92: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 5 / xy=30 or 31) ..... 201
Scenario 93: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 6 / xy=40 or 41) ..... 203
Scenario 94: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 7 / xy=50 or 51) ..... 204
Scenario 95: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 8 / xy=60 or 61) ..... 206
Scenario 96: Block Protocol with PICC Reset (PICC Type = A / x=0 / y=0 to 1) ..... 208
Scenario 97: Block Protocol with PICC Reset (PICC Type = B / x=1 / y=0 to 1) ..... 208
Scenario 98: Installation commands during Block Protocol (PICC Type = A / x=0) ..... 210
Scenario 99: Installation commands during Block Protocol (PICC Type = B / x=1) ..... 211
Scenario 100: Error at the beginning of the Block Protocol (x=0, 1, 3 and 4) ..... 213

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# Tables

Table 1: Specification Documents ..... 2
Table 2: Other Reference Documents ..... 2
Table 3: List of Abbreviations ..... 5
Table 4: Basic installation PICC Type A – UID size = 1 ..... 13
Table 5: Basic installation PICC Type A – UID size = 2 ..... 13
Table 6: Basic installation PICC Type A – UID size = 3 ..... 14
Table 7: Basic installation – PICC Type B ..... 14
Table 8: Modification of scenario in case of chaining from CUT (1/2) ..... 15
Table 9: Modification of scenario in case of chaining from CUT (2/2) ..... 15
Table 10: Scenario to be performed in case of 'Continue C/R – APDU exchanges until transaction is completed' ..... 16
Table 11: Loopback Command Coding ..... 16
Table 12: Response to Loopback Command ..... 17
Table 13: Parameter values Common Type A and Type B ..... 18
Table 14: Parameter values Specific Type A ..... 18
Table 15: Parameter values Specific Type B ..... 19

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 1. About this Document

## 1.1. Introduction

The present document describes the detailed organization and requirements of the executable tests the vendors has to comply with, to allow the digital part of the Contactless Card Level 1 Type Approval Testing.

The intended audience of this document includes test tool vendors, recognised test laboratories and auditors.

Describing the executable tests and the associated procedures is necessary to ensure reproducibility of the test results, even across different test laboratories.

## 1.2. Scope

For each individual executable test in this document, the following information is available:

- The test number,
- The objective of the test,
- The related reference specification section(s) of the Test Case,
- The procedure of the test,
- The acceptance criteria(s),
- The pre- and post-processing to be performed on a specific PICC (Conditional),
- The scenario of the test presented as a flow chart.

## 1.3. Reference Documents

EMV documents are available on the EMVCo web site:

http://www.emvco.com/approvals.aspx and
http://www.emvco.com/specifications.aspx

### 1.3.1. Specification Documents

| Document | Version<br>Issue date |
|---|---|
| EMV® Level 1 Specifications for Payment Systems — EMV Contactless Interface Specification | 3.2, July 2022 |
| EMV® SB No. 282 PPS0 and PPS1 RFU Value Handling | First Edition<br>March 2023 |
| EMV® SB No. 283 PCD Carrier Phase Drift | First Edition<br>March 2023 |
| EMV® SB No. 303 PCD Adding an Optional Suspend and a Mandatory Suspend in the Polling Loop | Second Edition<br>May 2024 |
| EMV® SB No. 300 PICC Presence Check Procedure | First Edition<br>June 2024 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Table 1: Specification Documents**

### 1.3.2. Other Reference Documents

| Document | Version<br>Issue date |
|---|---|
| ISO/IEC 14443-2 – Contactless ICCs – Radio frequency power and signal interface" + Amendment 1 “Bit rates of $f_c/64$, $f_c/32$ and $f_c/16$” – 2005. | 1st edition, 2001 |
| "ISO/IEC 14443-3 – Contactless ICCs – Initialization and anticollision"<br>+ Amendment 1 “Bit rates of $f_c/64$, $f_c/32$ and $f_c/16$” – 2005.<br>+ Amendment 3 “Handling of reserved fields and values” – 2006. | 1st edition, 2001 |
| "ISO/IEC 14443-4 – Contactless ICCs – Transmission protocol"<br>+ Amendment 1 “Handling of reserved fields and values” – 2006. | 1st edition, 2001 |
| "ISO/IEC 10373-6 – Identification cards – Test methods – Proximity cards"<br>+ FPD Amendments 1.2 “Additional PICC test methods” – 2001.<br>+ FPD Amendments 3 “Protocol test methods for proximity coupling devices” – 2001. | 1st edition, 2001 |
| "ISO/IEC 9646-1 – Open Systems Interconnection – Conformance Systems methodology and framework – Part 1: General concepts" | 2nd edition, 1994 |
| "ISO/IEC 9646-2 – Open Systems Interconnection – Conformance Systems methodology and framework – Part 2: Abstract test suite specification" | 2nd edition, 1994 |
| "ISO/IEC 9646-3 – Open Systems Interconnection – Conformance Systems methodology and framework – Part 3: The Tree and Tabular Combined Notation (TTCN)" | 2nd edition, 1998 |
| "ISO/IEC 9646-4 – Open Systems Interconnection – Conformance Systems methodology and framework – Part 4: Test realization" | 2nd edition, 1994 |
| “ISO/IEC 7816-4 – Identification cards - Integrated circuit cards - Part 4: Organization, security and commands for interchange” | 2nd edition, 2005 |
| NFC Forum, NFC Digital Protocol, Technical Specification | 1.0, 2010-11-17 |
| NFC Forum, NFC Analog Specification, Technical Specification | 1.0, 2012-07-11 |

**Table 2: Other Reference Documents**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 1.4. Acronyms and Abbreviations

The following abbreviations and notations are used in this document:

| Abbreviation | Description |
|---|---|
| AC | AntiCollision |
| ACK | Positive ACKnowledgement |
| ADC | Application Data Coding, Type B |
| AFI | Application Family Identifier, Type B |
| ASK | Amplitude Shift Keying |
| ATQA | Answer To reQuest, Type A |
| ATQB | Answer To reQuest, Type B |
| ATS | Answer To Select, Type A |
| ATTRIB | Selection command, Type B |
| BCC | Check byte, Type A |
| b.n | Block Number |
| BPSK | Binary Phase Shift Keying |
| C-APDU | Command Application Protocol Data Unit |
| CID | Card IDentifier |
| CLn | Cascade Level n, Type A |
| CT | Cascade Tag, Type A |
| CRC_A | Cyclic Redundancy Check error detection code for Type A |
| CRC_B | Cyclic Redundancy Check error detection code for Type B |
| CUT | Card Under Test. When the Instance Under Test as defined in ISO 9646 is a EMVCo Proximity IC Card |
| EDC | Error Detection Code |
| EGT | Extra Guard Time, Type B |
| EOF | End Of Frame |
| EoS | End of Sequence |
| etu | Elementary time unit |
| EMD | Electro Magnetic Disturbance |
| FDT | Frame Delay Time |
| $f_c$ | Carrier frequency |
| FO | Frame Option, Type B |
| $f_s$ | Subcarrier frequency |
| FSC | Frame Size for proximity Card |
| FSCI | Frame Size for proximity Card Integer |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Abbreviation | Description |
|---|---|
| FSD | Frame Size for proximity coupling Device |
| FSDI | Frame Size for proximity coupling Device Integer |
| FWI | Frame Waiting time Integer |
| FWT | Frame Waiting Time |
| HLTA | Halt Command, Type A |
| HLTB | Halt Command, Type B |
| ICS | Implementation Conformance Statement |
| IEC | International Electronical Commission |
| INF | INFormation field |
| ISO | International Organization for Standardization |
| LSB | Least Significant Bit |
| LT | Lower Tester |
| MBL | Maximum Buffer Length |
| MBLI | Maximum Buffer Length Integer |
| MSB | Most Significant Bit |
| NAD | Node ADdress |
| NAK | Negative Acknowledgment |
| NRZ-L | Non-Return to Zero, (L for Level) |
| PCB | Protocol Control Byte |
| PCD | Proximity Coupling Device (reader) |
| PICC | Proximity IC Card |
| PPSE | Proximity Payment System Environment |
| PUPI | Pseudo-Unique PICC Identifier, Type B |
| RATS | Request for Answer To Select |
| R-APDU | Response Application Protocol Data Unit |
| REQA | REQuest command, Type A |
| REQB | REQuest command, Type B |
| RF | Radio Frequency |
| RFU | Reserved for Future Use |
| SAK | Select AcKnowledge, Type A |
| SEL | SELect code, Type A |
| SOF | Start Of Frame |
| SoS | Start of Sequence |
| $T_{nn}$ | Minimum time period during the PICC is not allowed to produce any detectable disturbance before sending its response |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Abbreviation | Description |
|---|---|
| UID | Unique IDentifier, Type A |
| uidn | Byte number n of Unique Identifier |
| WTX | Waiting Time eXtension |
| WTXM | Waiting Time eXtension Multiplier |
| WUPA | Wake UP command, Type A |
| WUPB | Wake UP command, Type B |

**Table 3: List of Abbreviations**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 2. Generic Information about the Tests

## 2.1. ICS (Implementation Conformance Statement) and Application Selection

In the ICS (Implementation Conformance Statement) form document the PICC vendors have to indicate the parameters used by the PICC they want to go through Type Approval. This way, the tests scenarios which must be performed on the Card Under Test can be selected and configured by the laboratory. The parameters indicated in the ICS (Implementation Conformance Statement) form document shall be used.

The application residing in the PICC shall respond to its AID identified in ICS following selection of PPSE. The application shall be able to provide successful response to the test application commands, C-APDUs, which allows next command to be executed (if any) as identified in ICS. ICS shall have an example of trace and log with enough C and R-APDU exchange pairs to allow the payment transaction to be completed. The laboratory should be able to enter the C-APDUs and R-APDUs to the test tool (if necessary) for the application to execute the test. However the test tool may have already setup for a full transaction that would eliminate the tool preparation process for the application residing in the Card Under Test.

In the tests that ‘Continue C/R – APDU exchanges until transaction is completed’ is stated the test tool shall compare the data send in response from R-APDUs with the data and status words presented in ICS except the dynamic data (data changing each transaction).

Important: the laboratory shall ensure that the parameters indicated by the vendors in the ICS (Implementation Conformance Statement) form for contactless PICC digital testing are consistent with the EMV Contactless specification.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 2.2. Default Environmental Test Conditions

The following environmental conditions shall be used for all the tests described in the present document. They will be referred to as the 'Standard Test Conditions':

- The Card Under Test shall be placed as close as possible to the landing plane of the test tool so that the transaction is less perturbed.
- The external perturbations shall be suppressed: no metal objects or other perturbing elements in a volume of 30 centimeters around the test system and no other antennas (contactless terminals, cell phones,…) in a volume of 1 meter around the test system.
- Test tool environment shall be able to progress:
  - Application transaction flow according to test cases.
  - EMD algorithm and deaf time against noise generated by the PICC unless the test procedures state otherwise. This is to alleviate any disturbance during test session and to improve interoperability between test tool and the PICCs with EMD.

- Normal temperature: 23°C (±3°C)
- Ambient relative humidity: 50% ± 10%

## 2.3. Default Tool Requirements

Unless otherwise specified in the test description (i.e. for exception processing tests), the blocks sent by the LT shall never contain a CID or a NAD field as it is never used by an EMV Contactless terminal.

Unless otherwise specified in the PICC parameters section of the test the relevant test shall be applicable for all PICCs. If PICC parameters section of the test indicates special condition only PICCs with that condition shall be tested against the subject test. For example if the PICC parameters section indicates 'Type A/ UID size=1' the test is applicable all Type A PICCs with UID size 1 including dynamic UID PICCs. Another example is that if the parameters section indicates 'Type A' the test is applicable to all type A PICCs with fixed or dynamic UIDs.

Unless otherwise specified in the test description (i.e. to perform a PICC Reset), the LT shall never stop sending the carrier during a test transaction.

Unless otherwise specified in the test description, the LT shall apply the default EMV Contactless timings and the default EMV Contactless parameters values (as defined in the present section 2).

Unless otherwise specified in the test description, for PICC Type = A, the LT shall use the Type A frame format and a bit rate of 106 kb/s.

Unless otherwise specified in the test description, for PICC Type = B, the LT shall use the Type B frame format and a bit rate of 106 kb/s.

Unless otherwise specified in the test description, the LT shall initiate the test by generating the RF field for at least $t_P$ (to put the CUT in the IDLE state).

Although it is not specified in the test description, the LT shall respect the maximum frame size FSC of the PICC and use chaining if necessary when sending commands in I-Blocks.

Unless otherwise specified in the test description, the LT shall wait for at least 100 ms before considering that the CUT did not respond to a frame (i.e there is a time-out error).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

When, for Type Approval, a Test Tool is used as a LT, the following requirements apply:

- Any executed subcase shall generate traces, logs and a test report.
- The generation of traces, logs and test reports shall indicate whether the acceptance criterias have been met. All acceptance criterias and related values shall be visible in the traces, logs or test reports (e.g. for a timing measurement, the timing measured by the Test Tool shall be present, so shall be the tolerance used by the Test Tool and the expected value or expected range of values for this timing).
- Any applied timing shall be visible in the traces, logs or test reports.
- The Test Tool shall be able to manage WTX exchanges for at least 300ms when expecting an I-Block or R-Block from the CUT.
- Unless indicated otherwise, the Test Tool shall send all its frames indicating chaining with a size equal to FSC.

- All the frames sent by the CUT shall be checked. It means that the Test Tool shall check that:

    - Regarding the Type A frames: the presence of Start of Frame (SoF), parity bits and the two CRC bytes.

    - Regarding the type B frames: the presence of Start of Sequence (SoS), End of Sequence (EoS), a start bit (Logic “0”) followed by 8 data bits and a stop bit (Logic “1”) and the two CRC bytes.

- The Test Tool shall implement the following options: “Check APDU”, “Do not check APDU” and “Check SW only”. “Check APDU” means that all the data bytes included in the I-Blocks sent by the CUT shall be checked. “Do not check APDU” means that only the bytes related to the EMV Contactless Level 1 included in the frames sent by the CUT shall be checked (i.e. PCB and CRC bytes). “Check SW only” means that only the “Status Word” bytes in the I-Blocks sent by the CUT shall be checked.

-

In addition to the above requirements, it is recommended that the test tools comply with the following:

- The test tool should be capable of automatically detecting the digital parameters used by the CUT (e.g. technology supported, UID size, support of extended ATQB, etc).

## 2.4. Default Timings

For a time parameters given below, when a test description does not indicates a specific value which have to be used to perform the test, then the default time value shall be used by the LT to send sequences (i.e. the LT shall send sequences to the CUT with delays equal to the default values).

The concerned parameters and their default values are as follow.

**Common to Type A and Type B:**

After RF field is ON (after a RESET or at the beginning of a test case), the test tool shall wait at least 100ms before sending the 1st command.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**In Type A:**

SFGT = 1179648/ $f_c$ between the end of the sequence sent by the CUT and the beginning of the sequence sent by the LT containing first I-Block of the half-duplex block protocol.

$FDT_{A,PCD}$ = 8184/ $f_c$ (> $FDT_{A,PCD,MIN}$) between the end of a sequence sent by the CUT and the beginning of the next sequence sent by the LT.

During the whole test, unless otherwise specified in the test description, the LT applies a delay of unmodulated field for at least tP before WUPA command sent following WUPB or REQB or PICC reset sent by the LT.

During the whole test, the LT applies a delay of unmodulated field for at least $t_P$ between the end of the HLTA sequence sent by the LT and the beginning of the next sequence sent by the LT.

**In Type B:**

$FDT_{B,PCD}$ = 8184/ $f_c$ (> $FDT_{B,PCD,MIN}$) between the end of a sequence sent by the CUT and the beginning of the next sequence sent by the LT.

$EGT_{PCD}$ = 384/ $f_c$ (< $EGT_{PCD,MAX}$) between two consecutive characters within any sequence.

Start of Sequence (SoS) = 1344/ $f_c$ of logical state $(0)_b$ followed by 320/ $f_c$ of logical state $(‘1’)_b$.

End of Sequence (EoS) = 1344/ $f_c$ of logical state $(0)_b$ followed by a logical state transition.

During the whole test, unless otherwise specified in the test description, the LT applies a delay of unmodulated field for at least $t_P$ before WUPB command sent following WUPA or REQA or PICC reset sent by the LT.

## 2.5. Definition for Timing Measurement

When an executable test asks for the measurement (or the application) of a timing between two consecutive sequences sent in the same direction or in opposite directions, the delay shall be measured (or applied) between the end of the first sequence and the beginning of the second sequence:

- Beginning of a sequence sent by a Type A PICC: the start (i.e. the first modulation transmitted) of the Start of Frame (SoF) of the PICC sequence.
- Beginning of a sequence sent by a Type A PCD: the start of the lower level within the Start of Frame (SoF) of the PCD sequence.
- Beginning of a sequence sent by a Type B PICC: the start of the Start of Sequence (SoS) of PICC sequence.
- Beginning of a sequence sent by Type B PCD: the start of the Start of Sequence (SoS) of the PCD sequence.
- End of a sequence sent by a Type A PICC: the last modulation transmitted in the PICC sequence.
- End of a sequence sent by a Type A PCD: the rising edge of the last lower level of the PCD sequence (within the last data bit sent by the PCD if this last data bit is a Logic “1” or within the End of Frame if the last data bit sent by the PCD is a Logic “0”).
- End of a sequence sent by a Type B PICC: the start of the End of Sequence (EoS) of the PICC sequence.
- End of a sequence sent by a Type B PCD: the end of the End of Sequence (EoS) of the PCD sequence.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 2.6. Scenario Notations during Half Duplex Protocol

| Notation | Description |
|---|---|
| $I(0)_x$ [‘HH … HH’] | Not chained I-Block or last I-Block of a chain with block number x and containing the hexadecimal data bytes ‘HH … HH’ (i.e. transmitted APDUs) |
| $I(1)_x$ [‘HH … HH’] | Chained I-Block (except the last I-Block of a chain) with block number x and containing the hexadecimal data bytes ‘HH … HH’ (i.e. transmitted APDUs) |
| $R(ACK)_x$ | R-Block indicating a positive acknowledgment with block number x |
| $R(NAK)_x$ | R-Block indicating a negative acknowledgment with block number x |
| S(…) | S-Block |

Remark: The Test Tool shall never send blocks with a NAD or a CID unless indicated otherwise.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 2.7. Types of Frames in the Scenario

In this document, when describing the transactions within the test scenarios, the following rules apply:

In Type A tests, if nothing is indicated, the bytes are transported within a Type A standard frame with the 2 bytes of CRC_A (i.e. a valid CRC_A is automatically added after the bytes by the Test Tool).

In Type A tests, if *"(short frame)"* is indicated, the bytes are transported within a Type A short frame.

In Type A tests, if *"(no CRC_A)"* is indicated, the bytes are transported within a Type A standard frame with no CRC_A bytes.

In Type A tests, if *"(Type B frame)"* is indicated, the bytes are transported within a Type B frame with 2 bytes of CRC_B (i.e. a valid CRC_B is automatically added after the bytes by the Test Tool).

In Type B tests, if nothing is indicated, the bytes are transported within Type B frames with 2 bytes of CRC_B (i.e. a valid CRC_B is automatically added after the bytes by the Test Tool).

In Type B tests, if *"(Type A short frame)"* is indicated, the bytes are transported within a Type A short frame (WUPA Command sent during collision detection).

In Type A and Type B tests, if *"(NFC-F Frame)"* is indicated, the bytes are transported within an NFC-F Frame at 212 kbps as defined in NFC Forum Referenced Documents.

In Type A and Type B tests, if *"(Type A Frame)"* is indicated, the bytes are transported within a Type A standard frame with the 2 bytes of CRC_A (i.e. a valid CRC_A is automatically added after the bytes by the Test Tool).

In Type A and Type B tests, if *"(Type B Frame)"* is indicated, the bytes are transported within Type B frames with 2 bytes of CRC_B (i.e. a valid CRC_B is automatically added after the bytes by the Test Tool).

## 2.8. Notational Conventions

The following notations apply:

**'00' to 'FF' or "00…00" to "FF…FF":** hexadecimal values (sometimes the hexadecimal values are indicated between parenthesis and followed by a lower case "h").

**$(0)_b$ or $(\text{"1001"})_b$:** binary notation. Values expressed in binary form are followed by a lower case "b".

**u:** any value (e.g. byte A = $(\text{"1u11 0101"})_b$ means that b7 of byte A can take any value).

**UID Size 1, 2 or 3:** refers to the single, double or triple size Type A UID as defined in the EMV specification.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 2.9. Tests References

The tests described in the following sections are referenced this way:

**Test codification:**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **C** | **-** | **-** | **-** | **-** | **.** | **-** | **-** |
| **-** | **a** | **b** | **c** | | **.** | **d** | |

**a:** Frame Type:

- A = Type A tests
- B = Type B tests
- C = Common tests to Type A and Type B

**b:** Test Type:

- 0 = Basic tests
- 1 = Correct tests
- 2 = Error tests

**c:** Test Number (within the Test Type):

- From 00 to 99

**d:** Optional Subcase Reference xy:

- From 0 to 99 (decimal)

e.g. CA223.2 represents the Subcase 2 of the Test number 23 within the tests concerning the Type A transactions with error.

Remark: Common tests to Type A and Type B must be performed in Type A and in Type B on dual Type PICCs.

## 2.10. Basic Installation

When the test description indicates that the LT initiates the test by performing a basic installation of the CUT, then the following basic installation shall be applied by the test implementation.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

For a Type A PICC with UID Size 1:

| | | | | |
|---|---|---|---|---|
| i | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| i+1 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| i+2 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| i+3 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| i+4 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| i+5 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| i+6 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS (FSD = 256) |
| i+7 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |

**Table 4: Basic installation PICC Type A – UID size = 1**

For a Type A PICC with UID Size 2:

| | | | | |
|---|---|---|---|---|
| i | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| i+1 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| i+2 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| i+3 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| i+4 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| i+5 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| i+6 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| i+7 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| i+8 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| i+9 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| i+10 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS (FSD = 256) |
| i+11 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |

**Table 5: Basic installation PICC Type A – UID size = 2**

For a Type A PICC with UID Size 3:

| | | | | |
|---|---|---|---|---|
| i | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| i+1 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| i+2 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| i+3 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| | | | | |
|---|---|---|---|---|
| i+4 | **CUT** ◂ | ’**93 70**’ + ***UID CL1*** + ***BCC*** | ◂ **LT** | SELECT CL1 |
| i+5 | **CUT** ▸ | (“uu1u u1uu”)$_b$ | ▸ **LT** | SAK |
| i+6 | **CUT** ◂ | ’**95 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL2 |
| i+7 | **CUT** ▸ | ***UID CL2*** + ***BCC*** *(no CRC_A)* | ▸ **LT** | UID CL2 + BCC |
| i+8 | **CUT** ◂ | ’**95 70**’ + ***UID CL2*** + ***BCC*** | ◂ **LT** | SELECT CL2 |
| i+9 | **CUT** ▸ | (“uu1u u1uu”)$_b$ | ▸ **LT** | SAK |
| i+10 | **CUT** ◂ | ’**97 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL3 |
| i+11 | **CUT** ▸ | ***UID CL3*** + ***BCC*** *(no CRC_A)* | ▸ **LT** | UID CL3 + BCC |
| i+12 | **CUT** ◂ | ’**97 70**’ + ***UID CL3*** + ***BCC*** | ◂ **LT** | SELECT CL3 |
| i+13 | **CUT** ▸ | (“uu1u u0uu”)$_b$ | ▸ **LT** | SAK |
| i+14 | **CUT** ◂ | ‘**E0 80**’ | ◂ **LT** | RATS (FSD = 256) |
| i+15 | **CUT** ▸ | ***ATS*** | ▸ **LT** | ATS |

**Table 6: Basic installation PICC Type A – UID size = 3**

For a Type B PICC:

| | | | | |
|---|---|---|---|---|
| i | **CUT** ◂ | ’**05 00 08**’ | ◂ **LT** | WUPB to poll for the CUT |
| i+1 | **CUT** ▸ | ***ATQB*** *(containing PUPI)* | ▸ **LT** | ATQB |
| i+2 | **CUT** ◂ | ’**1D**’ + ***PUPI*** + ’**00 08 01 00**’ | ◂ **LT** | ATTRIB |
| i+3 | **CUT** ▸ | ‘**00**’ | ▸ **LT** | ATTRIB Response |

**Table 7: Basic installation – PICC Type B**

## 2.11.Pre- and Post-Processing

Unless stated otherwise in the test description, when the CUT sends an S(WTX) Request or uses PICC chaining, the following processing is applied during any test described in the present document:

On reception of an S(WTX) Request (i.e. block with PCB=’F2’) from the CUT, the test script shall be modified so that the LT checks that the S(WTX) Block sent by the CUT has a PCB (i.e. first byte of the Block) equal to (‘F2’)$_h$ (i.e. b2 of S-Block PCB is set to (1)$_b$) and that the value of WTXM indicated in the S(WTX) Request respects 1 ≤ WTXM ≤ 59. Then, if the value of WTXM is correct, the LT shall reply, with respect of FWT, with the corresponding S(WTX) Response (i.e. send back the received block) and this S(WTX) exchange shall be ignored (the consecutive step numbers shall be adapted – *no impact on the block numbers*). The LT shall ensure that the delay between the end of the S(WTX) Response sequence sent by the LT and the beginning of the next sequence sent by the CUT is at most $FWT_{TEMP}$ =

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

((256 x 16/ $f_c$) x $2^{FWI}$ x WTXM) where WTXM is the FWT Extension Multiplier indicated in the S(WTX) Request.

On reception of a I-Block indicating chaining (i.e. block with PCB='12' or '13') in a frame of size S from the CUT, the test script shall be modified so that the LT checks that the INF field of the I-Block is not empty, and so that the steps:

| | | | |
|---|---|---|---|
| i | **CUT ▸** | **I(0)$_n$ [**'R-APDU$_0$' (including a Status Word')**]** **▸ LT** | Block containing R-APDU$_0$ |
| i+1 | **CUT ◂** | **I(M)$_{n+1}$** **◂ LT** | Block containing C-APDU$_0$ |

**Table 8: Modification of scenario in case of chaining from CUT (1/2)**

are replaced by:

| | | | |
|---|---|---|---|
| i | **CUT ▸** | **I(1)$_n$ [**'R-APDU$_1$' **]** in a frame of size S **▸ LT** | Block containing R-APDU$_0$ |
| i+1 | **CUT ◂** | **R(ACK)$_{n+1}$** **◂ LT** | |
| … | *As many I(1)-Blocks / R(ACK) Blocks exchanges as needed* | | … |
| i+j | **CUT ▸** | **I(0)$_{n+j/2}$ [**'R-APDU$_{(1+j/2)}$' (including a Status Word')**]** in a frame of size ≤ S **▸ LT** | last I-Block of the chain containing R-APDU$_{(1+j/2)}$ |
| i+j+1 | **CUT ◂** | **I(M)$_{n+j/2+1}$** **◂ LT** | Block containing C-APDU$_0$ |

**Table 9: Modification of scenario in case of chaining from CUT (2/2)**

where 'R-APDU$_0$' = 'R-APDU$_1$' + 'R-APDU$_2$'+ … + 'R-APDU$_{(1+j/2)}$' and where I - blocks above are send with b.n. ={n, n+1, n+2 , n+3 , …, n+j/2+1} in mod 2.

And the consecutive step numbers and block numbers shall be adapted.

## 2.12. Transaction Completion

Unless stated otherwise in the test description, where the test procedures identify as 'Continue C/R – APDU exchanges until transaction is completed' the I-block exchanges shall be included in the subject steps until the relevant payment transaction is completed as indicated below.

Parameters:

- 'k' parameter indicates step number for **the last step before** the 'Continue C/R – APDU exchanges until transaction is completed' where response sent by CUT to Select AID command
- 'j/2-2' is the number of C/R – APDU pairs to complete transaction

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

The ‘Continue C/R – APDU exchanges until transaction is completed’ process applied as below:

| | | |
|---|---|---|
| k+1 | **CUT ◂ I(0)$_0$ [‘C-APDU‘$_1$] ◂ LT** | |
| k+2 | **CUT ▸ I(0)$_0$[‘R-APDU‘$_1$] ▸ LT** | |
| … | ‘j/2-2’ number of I(0) Block (C/R – APDUs) exchange pairs | … |
| k+j-1 | **CUT ◂ I(0)$_{j/2}$ [‘C-APDU‘$_{j/2}$] ◂ LT** | Last transaction C-APDU |
| k+j | **CUT ▸ I(0)$_{j/2}$ [‘R-APDU‘$_{j/2}$] ▸ LT** | Last transaction R-APDU |

**Table 10: Scenario to be performed in case of ‘Continue C/R – APDU exchanges until transaction is completed’**

where I - blocks above are send with b.n. ={0, 1, 3 , …, j/2} in mod 2.

## 2.13.Loopback Command Description

The Loopback Command is used to simulate a chaining of I-Blocks sent by the LT.

The test tool shall support Loopback Command as coded in Table 11: Loopback Command Coding.

The support of the Loopback Command by CUT is optional. A successful response to a Loopback Command is shown in Table 12: Response to Loopback Command. If supported, CUT shall fill in the Data field of the response and status words, ‘9000’. All status conditions are defined in ISO 7816.

| Field | Content | Length |
|---|---|---|
| CLA | 80 | 1 |
| INS | EE | 1 |
| P1 | 00 | 1 |
| P2 | 00 | 1 |
| Lc | FA | 1 |
| Data | 00 01 02 03 … F7 F8 F9 | Up to 256 |
| Le | 00 | 1 |

**Table 11: Loopback Command Coding**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Field | Content | Length |
|---|---|---|
| Data | The Data field returned by the PICC is the data sent in the loopback command | Up to 256 |
| SW1-SW2 | $(‘9000’)_h$ | 2 |

**Table 12: Response to Loopback Command**

## 2.14. Parameters Values

This section defines the parameter values to be applied or observed during the PICC digital tests unless identified in the test descriptions otherwise.

The timing values in the table below are as given in the EMV Contactless specification and do not take the test margins induced by the tool inaccuracy into account.

When a timing value is applied by the test tool (e.g. $FDT_{A,PCD,MIN}$), the test implementation should be as close as possible to the timing given in the table and the following rules shall be respected by the test tool:

- In the case where a minimum timing value shall be supported by the tested PICC, the applied timing value shall never be lower than the specification value given in the table
- In the case where a maximum timing value shall be supported by the tested PICC, the applied timing value shall never be higher than the specification value given in the table

When a timing value is measured during a test, the tolerances of the test tool shall be used. The following is the maximum tolerance p accepted for a test tool:

- If **T** is defined in ‘$1/f_c$’ and **T** $\leq$ 12800 x $1/f_c$ then **p** = 12.8 x $1/f_c$
- If **T** is defined in ‘$1/f_c$’ and 12800 < **T** $\leq$ 128000 x $1/f_c$ then **p** = 64 x $1/f_c$
- If **T** is defined in ‘$1/f_c$’ and 128000 < **T** $\leq$ 1280000 x $1/f_c$ then **p** = 128 x $1/f_c$
- If **T** is defined in ‘$1/f_c$’ and **T** > 1280000 x $1/f_c$ then **p** = 0.001 * **T**
- If **T** is defined in ‘ms’ then **p** = 0.001 * **T** (e.g. **T** = 10 ms => **p** = 10 µs)

Notation: T is the timing which shall be measured within a Test Case and p is the time precision needed for this Test Cases.

For all the tests described in the present document, the following parameters values shall apply:

Parameters common to Type A and Type B:

| Parameter name | Parameter value | Comments |
|---|---|---|
| Mute_Time | 100000 µs | Delay following time-out after which a PICC not sending any sequence is considered as mute (specific to the tests), e.g. FWT+ ΔFWT+Mute_Time after the end of a sequence sent by the test tool |
| $t_P$ | 5000 µs | |
| $t_{RESET}$ | 5000 µs | |
| $t_{SUSPEND}$ | 30000 µs | |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Parameter name | Parameter value | Comments |
|---|---|---|
| $t_{RETRANSMISSION}$ | 10.0 ms | |
| FWT | $(256 \times 16/ f_c) \times 2^{FWI}$ | Detailed in the test description |
| $FWT_{DEFAULT}$ | $65536/ f_c$ | Corresponding to the basic installation: FWI = 4 |
| FWT x WTXM | $(WTXM \times (256 \times 16/ f_c) \times 2^{FWI}$ | Detailed in the test description |
| $FWT_{MAX}$ | $524288/ f_c$ | |
| ΔFWT | $49152/ f_c$ | |
| $FWT_{DESELECT}$ | $512 \times 128/ f_c$ | |
| FSC | n bytes | In function of FSCI or Max_Frame_Size (see the EMV specification for a detailed definition) |
| FSD | 256 bytes | |
| SFGT | $(256 \times 16/ f_c) \times 2^{SFGI}$ | Detailed in the test description |
| SFGI | 0 to 8 | |

**Table 13: Parameter values Common Type A and Type B**

Parameters specific to Type A:

| Parameter name | Parameter value | Comments |
|---|---|---|
| $FDT_{A,PICC,MIN}$ | $(1152 + 20)/ f_c$ if the last bit transmitted by the Test Tool is $(0)_b$<br>$(1152 + 84)/ f_c$ if the last bit transmitted by the Test Tool is $(1)_b$ | |
| $FDT_{A,PICC,ANTICOLLISION}$ | $(1152 + 20)/ f_c$ if the last bit transmitted by the Test Tool is $(0)_b$<br>$(1152 + 84)/ f_c$ if the last bit transmitted by the Test Tool is $(1)_b$ | |
| $FWT_{ACTIVATION}$ | $65536/ f_c$ | |
| $t_H$ | $12800/ f_c$ | |
| FSCI | 2 to 8 | |
| $FDT_{A,PCD,MIN}$ | $1172/ f_c$ | |

**Table 14: Parameter values Specific Type A**

Parameters specific to Type B:

| Parameter name | Parameter value | Comments |
|---|---|---|
| $FDT_{B,PICC,MIN}$ | $2304/ f_c$ | $= TR0_{MIN} + TR1_{MIN}$<br>$= 1024 / f_c + 1280/ f_c$ |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Parameter name | Parameter value | Comments |
|---|---|---|
| $TR0_{MIN}$ | 1024/ $f_c$ | |
| $TR0_{MAX,ATQB}$ | 4096/ $f_c$ | |
| $TR1_{MIN}$ | 1280/ $f_c$ | |
| $TR1_{MAX}$ | 3200/ $f_c$ | |
| $EGT_{PICC,MAX}$ | 256/ $f_c$ | |
| $EGT_{PCD,MIN}$ | 0 µs | |
| $EGT_{PCD,MAX}$ | 768/ $f_c$ | |
| $t_{FSOFF,MAX}$ | 256/ $f_c$ | |
| Max_Frame_Size | 2 to 8 | |
| $FDT_{B,PCD,MIN}$ | 1792/ $f_c$ | |

**Table 15: Parameter values Specific Type B**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3. Type A Executable Tests

## 3.1. Basic Type A Exchange and Timings Measurement (UID size = 1) [CA001]

**Test Codification:**

CA001

**ICS Parameters:**

PICC Type = A / UID size = 1

**Test Objective:**

To ensure that the Type A PICC with a single size UID respects the frame format, the timings and the order of the commands during a basic exchange (installation and block protocol).

**Reference Requirements:**

4.7.1.1, 4.7.1.2, 4.7.1.3, 4.8.1.2, 4.8.1.5, 4.8.1.9, 4.8.1.11, 4.8.1.13, 5.2.1.1, 5.2.1.2, 5.3.2.3, 5.5.2.2, 5.7.1.1, 5.7.2.1, 5.7.2.3, 5.7.2.4, 5.7.2.7, 5.7.2.9, 5.7.2.10, 5.7.2.12, 5.7.2.15, 7.1.1.5, 10.3.4.8

**Procedure:**

Run the following scenario.

When a sequence sent by the LT is followed by a sequence sent by the CUT, the LT measures the Frame Delay Time between the end of the sequence sent by the LT and the beginning of the sequence replied by the CUT.

Remark: The Frame Delay Time used by the CUT to reply to any command shall be measured, and, if the CUT uses chaining or sends WTX Requests when replying to any command, then the measurement is performed for each sequence sent by the CUT until the next command is sent by the LT.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

Each frame sent by the CUT is a Type A standard frame composed of a Start of Frame i.e. SoF = Logic “1”, followed by the bytes coding the CUT response (and eventually the CRC). For each byte, the 8 bits are transmitted LSB first and are followed by an odd parity bit.

Each Type A frame sent by the CUT is contained in a sequence ending with an End of Sequence i.e. EoS = the carrier is not modulated with the subcarrier for one bit duration.

The ATQA and UID CL1 + BCC frames sent by the CUT do not contain any CRC_A bytes. Other Commands and Block frames sent by the CUT contain 2 CRC_A bytes (last two bytes of the frames) calculated as defined in ISO/IEC 13239 (with a register initialized to ‘6363’).

On reception of the WUPA Command before the RESET, the CUT sends always the same ATQA. On reception of the WUPA after the RESET, all the ATQA have the same value. An

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

ATQA shall always be sent with the most significant byte (Byte 1) and the least significant byte (Byte 2) having the value:
Byte 1 = (“0001 0000”)$_b$ or
(“0000 1000”)$_b$ or
(“0000 0100”)$_b$ or
(“0000 0010”)$_b$ or
(“0000 0001”)$_b$
Byte 2 = (“0000 uuuu”)$_b$ where the bits indicated as ‘u’ may take any value.

On reception of the ANTICOLLISION CL1 Command, the CUT sends a UID CL1 + BCC with the most significant byte different from (‘88’)$_h$ and with the byte BCC (least significant byte) equal to the result of an exclusive-or over the 4 previous bytes UID CL1 (4 most significant bytes).

On reception of the SELECT CL1 Command, the CUT sends a SAK byte having the value:
SAK = (“uu1u u0uu”)$_b$ where the bits indicated as ‘u’ may take any value.

On reception of the RATS Command, the CUT sends an ATS having the value:

| **ATS** | | | | | |
|---|---|---|---|---|---|
| Byte 1 | Byte 2 | Byte 3 | Byte 4 | Byte 5 | Byte 6 – Byte 6+k |
| TL | T0 | TA(1) | TB(1) | TC(1) | Historical Bytes |

Where:

- TL = (number of Historical Bytes + 5) and (‘05’)$_h$ ≤ TL ≤ (‘14’)$_h$
- T0 = (“0111 aaaa”)$_b$ with (“aaaa”)$_b$ = FSCI where (‘2’)$_h$ ≤ FSCI ≤ (‘C’)$_h$
- TA(1) = (‘00’)$_h$ or (‘80’)$_h$
- TB(1) = (“XY”)$_h$ with
  - X = FWI where X ≤ (‘7’)$_h$
  - Y = SFGI where Y ≤ (‘8’)$_h$
- TC(1) = (“0000 00uu”)$_b$ where the bits indicated as ‘u’ may take any value
- Historical Bytes = up to 15 bytes (k ≤ 14) which may take any value

On reception of an I-Block with a block number equal to 0, the CUT replies with an I-Block having a PCB field (i.e. first byte of the Block) equal to (‘02’)$_h$ *or (‘12’)$_h$*.
On reception of an I-Block with a block number equal to 1, the CUT replies with an I-Block having a PCB field (i.e. first byte of the Block) equal to (‘03’)$_h$ *or (‘13’)$_h$*.

The delay $FDT_{A,PICC}$ between the end of a sequence sent by the LT and the beginning of the next sequence sent by the CUT is aligned to the following time grid:

- $FDT_{A,PICC}$ = (n x 128 + 20)/ $f_c$ if the last bit transmitted by the LT is $(0)_b$
- $FDT_{A,PICC}$ = (n x 128 + 84)/ $f_c$ if the last bit transmitted by the LT is $(1)_b$

with the integer n ≥ 9 (see below).

During installation after WUPA, ANTICOLLISION CL1 and SELECT CL1 and after WUPA Commands following the PICC Reset, the delay between the end of the sequence sent by the LT and the beginning of the next sequence sent by the CUT is equal to $FDT_{A,PICC,ANTICOLLISION}$ with a tolerance of 0 to 0.4µs.

After the RATS Command, the delay between the end of the sequence sent by the LT and the beginning of the ATS sequence sent by the CUT is at least $FDT_{A,PICC,MIN}$ and is at most $FWT_{ACTIVATION}$.

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that does not contain an S(WTX) Response and the beginning of the next sequence sent by the CUT is at least $FDT_{PICC,MIN}$ and is at most FWT = (256 x 16/$f_c$) x $2^{FWI}$.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that contains an S(WTX) Response and the beginning of the next sequence sent by the CUT is at least $FDT_{PICC,MIN}$ and is at most $FWT_{TEMP}$ = ((256 x 16/$f_c$) x $2^{FWI}$) x WTXM where WTXM is the FWT Extension Multiplier indicated in the S(WTX) Request and (WTXM x (256 x 16/ $f_c$)x $2^{FWI}$) ≤ $FWT_{MAX}$.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◄ | '**52**' *(short frame)* | ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | ATQA |
| 3 | **CUT** ◄ | '**50 00**' | ◄ **LT** | HLTA |
| 4 | **CUT** ◄ | '**05 00 08**' *(Type B frame)* sent after $t_P$ | ◄ **LT** | WUPB |
| 5 | **CUT** ◄ | '**52**' *(short frame)* sent after $t_P$ | ◄ **LT** | WUPA to wake up the CUT |
| 6 | **CUT** ► | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | ATQA |
| 7 | **CUT** ◄ | '**93 20**' *(no CRC_A)* | ◄ **LT** | ANTICOLLISION CL1 |
| 8 | **CUT** ► | ***UID CL1 + BCC*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | UID CL1 + BCC |
| 9 | **CUT** ◄ | '**93 70**' **+** ***UID CL1 + BCC*** | ◄ **LT** | SELECT CL1 |
| 10 | **CUT** ► | ("uu1u u0uu")$_b$ sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | SAK |
| 11 | **CUT** ◄ | '**E0 80**' | ◄ **LT** | RATS (FSD = 256) |
| 12 | **CUT** ► | ***ATS*** sent between $FDT_{A,PICC,MIN}$ and $FWT_{ACTIVATION}$ | ► **LT** | ATS |
| 13 | **CUT** ◄ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◄ **LT** | Select PPSE / b.n. = 0 |
| 14 | **CUT** ► | **I(0)$_0$ ['6F' +** n data bytes + '**90 00**'**]** sent between $FDT_{A,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | ► **LT** | b.n. = 0 |
| 15 | **CUT** ◄ | **I(0)$_1$ ['00 A4 04 00'** + Lc + AID + **'00']** | ◄ **LT** | Select AID / b.n. = 1 |
| 16 | **CUT** ► | **I(0)$_1$ ['6F' +** n data bytes + '**90 00**'**]** sent between $FDT_{A,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | ► **LT** | b.n. = 1 |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of R-APDUs are sent between $FDT_{A,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | | | |
| 17+j | **CUT** ◄ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◄ **LT** | PICC Reset |
| 18+j | **CUT** ◄ | '**52**' *(short frame)* sent after $t_P$ | ◄ **LT** | WUPA to poll for the CUT |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 19+j | **CUT** ▸ ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ ▸ **LT** | ATQA |
| 20+j | **CUT** ◂ ’**50 00**’ ◂ **LT** | HLTA |
| 21+j | **CUT** ◂ ‘**52**’ *(short frame)* sent after $t_P$ ◂ **LT** | WUPA to wake up the CUT |
| 22+j | **CUT** ▸ ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ ▸ **LT** | ATQA |
| 23+j | **CUT** ◂ ’**50 00**’ ◂ **LT** | HLTA |
| 24+j | **CUT** ◂ ‘**52**’ *(short frame)* sent after $t_P$ ◂ **LT** | WUPA to wake up the CUT |
| 25+j | **CUT** ▸ ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ ▸ **LT** | ATQA |
| 26+j | **CUT** ◂ ’**50 00**’ ◂ **LT** | HLTA |
| 27+j | **CUT** ◂ ‘**52**’ *(short frame)* sent after $t_P$ ◂ **LT** | WUPA to wake up the CUT |
| 28+j | **CUT** ▸ ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ ▸ **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 1: Basic Type A Exchange and Timings Measurement (UID size = 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.2. Basic Type A Exchange and Timings Measurement (UID size = 2) [CA002]

**Test Codification:**

CA002

**ICS Parameters:**

PICC Type = A / UID size = 2

**Test Objective:**

To ensure that the Type A PICC with a double size UID respects the frame format, the timings and the order of the commands during a basic exchange (installation and block protocol).

**Reference Requirements:**

4.7.1.1, 4.7.1.2, 4.7.1.3, 4.8.1.2, 4.8.1.5, 4.8.1.9, 4.8.1.11, 4.8.1.13, 5.3.2.3, 5.5.2.2, 5.7.1.1, 5.7.2.1, 5.7.2.3, 5.7.2.4, 5.7.2.7, 5.7.2.9, 5.7.2.10, 5.7.2.12, 5.7.2.15, 7.1.1.5, 10.3.4.8

**Procedure:**

Run the following scenario.

When a sequence sent by the LT is followed by a sequence sent by the CUT, the LT measures the Frame Delay Time between the end of the sequence sent by the LT and the beginning of the frame replied by the CUT.

Remark: The Frame Delay Time used by the CUT to reply to the any command shall also be measured. And, if the CUT uses chaining or sends WTX Requests when replying to the any command, then the measurement is performed for each sequence sent by the CUT until the next command is sent by the LT.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

Each frame sent by the CUT is a Type A standard frame composed of a Start of Frame i.e. SoF = Logic “1”, followed by the bytes coding the CUT response (and eventually the CRC). For each byte, the 8 bits are transmitted LSB first and are followed by an odd parity bit.

Each Type A frame sent by the CUT is contained in a sequence ending with an End of Sequence i.e. EoS = the carrier is not modulated with the subcarrier for one bit duration.

The ATQA, UID CL1 + BCC and UID CL2 + BCC frames sent by the CUT do not contain any CRC_A bytes. Other Commands and Block frames sent by the CUT contain 2 CRC_A bytes (last two bytes of the frames) calculated as defined in ISO/IEC 13239 (with a register initialized to ‘6363’).

On reception of the WUPA Command before the RESET, the CUT sends always the same ATQA. On reception of the WUPA after the RESET, all the ATQA have the same value. An ATQA shall always be sent with the most significant byte (Byte 1) and the least significant byte (Byte 2) having the value:
Byte 1 = (“0101 0000”)$_b$ or
(“0100 1000”)$_b$ or
(“0100 0100”)$_b$ or
(“0100 0010”)$_b$ or
(“0100 0001”)$_b$
Byte 2 = (“0000 uuuu”)$_b$ where the bits indicated as ‘u’ may take any value.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

On reception of the ANTICOLLISION CL1 Command, the CUT sends a UID CL1 + BCC with the most significant byte equal to $(‘88’)_h$.

On reception of the ANTICOLLISION CL2 Command, the CUT sends a UID CL2 + BCC with the most significant byte different from $(‘88’)_h$.

The byte BCC (least significant byte) of UID CLn + BCC (n=1 or 2) is equal to the result of an exclusive-or over the 4 previous bytes UID CLn (4 most significant bytes).

On reception of the SELECT CL1 Command, the CUT sends a SAK byte having the value: SAK = $(“uu1u\ u1uu”)_b$ where the bits indicated as ‘u’ may take any value.

Remark: For backward compatibility the bit 6 of SAK sent as response to the SELECT CL1 Command remains $(“1”)_b$.

On reception of the SELECT CL2 Command, the CUT sends a SAK byte having the value: SAK = $(“uu1u\ u0uu”)_b$ where the bits indicated as ‘u’ may take any value.

On reception of the RATS Command, the CUT sends an ATS having the value:

| **ATS** | | | | | |
|---|---|---|---|---|---|
| Byte 1 | Byte 2 | Byte 3 | Byte 4 | Byte 5 | Byte 6 – Byte 6+k |
| TL | T0 | TA(1) | TB(1) | TC(1) | Historical Bytes |

Where:

- TL = (number of Historical Bytes + 5) and $(‘05’)_h \leq TL \leq (‘14’)_h$
- T0 = $(“0111\ aaaa”)_b$ with $(“aaaa”)_b$ = FSCI where $(‘2’)_h \leq FSCI \leq (‘C’)_h$
- TA(1) = $(‘00’)_h$ or $(‘80’)_h$
- TB(1) = $(“XY”)_h$ with
  - X = FWI where $X \leq (‘7’)_h$
  - Y = SFGI where $Y \leq (‘8’)_h$
- TC(1) = $(“0000\ 00uu”)_b$ where the bits indicated as ‘u’ may take any value
- Historical Bytes = up to 15 bytes (k ≤ 14) which may take any value

On reception of an I-Block with a block number equal to 0, the CUT replies with an I-Block having a PCB field (i.e. first byte of the Block) equal to $(‘02’)_h$ or $(‘12’)_h$.

On reception of an I-Block with a block number equal to 1, the CUT replies with an I-Block having a PCB field (i.e. first byte of the Block) equal to $(‘03’)_h$ or $(‘13’)_h$.

The delay $FDT_{A,PICC}$ between the end of a sequence sent by the LT and the beginning of the next sequence sent by the CUT is aligned to the following time grid:

- $FDT_{A,PICC} = (n \times 128 + 20)/ f_c$ if the last bit transmitted by the LT is $(0)_b$
- $FDT_{A,PICC} = (n \times 128 + 84)/ f_c$ if the last bit transmitted by the LT is $(1)_b$

with the integer n ≥ 9 (see below).

During installation after WUPA, ANTICOLLISION CLn (n=1 to 2) and SELECT CLn (n=1 to 2) and after WUPA Commands following the PICC Reset, the delay between the end of the sequence sent by the LT and the beginning of the next sequence sent by the CUT is equal to $FDT_{A,PICC,ANTICOLLISION}$ with a tolerance of 0 to 0.4µs.

After the RATS Command, the delay between the end of the sequence sent by the LT and the beginning of the next sequence sent by the CUT is at least $FDT_{A,PICC,MIN}$ and is at most $FWT_{ACTIVATION}$.

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that does not contain an S(WTX) Response and the beginning of the next sequence sent by the CUT is at least $FDT_{PICC,MIN}$ and is at most FWT = $(256 \times 16/f_c) \times 2^{FWI}$.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that contains an S(WTX) Response and the beginning of the next sequence sent by the CUT is at least $FDT_{PICC,MIN}$ and is at most $FWT_{TEMP}$ = ((256 x 16/$f_c$) x $2^{FWI}$) x WTXM where WTXM is the FWT Extension Multiplier indicated in the S(WTX) Request and (WTXM x (256 x 16/ $f_c$)x $2^{FWI}$) ≤ $FWT_{MAX}$.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**50 00**' | ◀ **LT** | HLTA |
| 4 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent after $t_P$ | ◀ **LT** | WUPB |
| 5 | **CUT** ◀ | '**52**' *(short frame)* sent after $t_P$ | ◀ **LT** | WUPA to wake up the CUT |
| 6 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▶ **LT** | ATQA |
| 7 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 8 | **CUT** ▶ | ***UID CL1 + BCC*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▶ **LT** | UID CL1 + BCC |
| 9 | **CUT** ◀ | '**93 70**' **+** ***UID CL1 + BCC*** | ◀ **LT** | SELECT CL1 |
| 10 | **CUT** ▶ | ("uu1u u1uu")$_b$ sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 12 | **CUT** ▶ | ***UID CL2 + BCC*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▶ **LT** | UID CL2 + BCC |
| 13 | **CUT** ◀ | '**95 70**' **+** ***UID CL2 + BCC*** | ◀ **LT** | SELECT CL2 |
| 14 | **CUT** ▶ | ("uu1u u0uu")$_b$ sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▶ **LT** | SAK |
| 15 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS (FSD = 256) |
| 16 | **CUT** ▶ | ***ATS*** sent between $FDT_{A,PICC,MIN}$ and $FWT_{ACTIVATION}$ | ▶ **LT** | ATS |
| 17 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00']** | ◀ **LT** | Select PPSE / b.n. = 0 |
| 18 | **CUT** ▶ | **I(0)$_0$ ['6F'** + n data bytes + '**90 00']** sent between $FDT_{A,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | ▶ **LT** | b.n. = 0 |
| 19 | **CUT** ◀ | **I(0)$_1$ ['00 A4 04 00'** + Lc + AID + '**00']** | ◀ **LT** | Select AID / b.n. = 1 |
| 20 | **CUT** ▶ | **I(0)$_1$ ['6F'** + n data bytes + '**90 00']** sent between $FDT_{A,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | ▶ **LT** | b.n. = 1 |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of R-APDUs are sent between FDTA,PICC,MIN and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | | | |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 21+j | **CUT** ◀ **The LT performs a PICC Reset** (i.e. stops sending the Carrier during a time $t_{RESET}$) ◀ **LT** | PICC Reset |
| 22+j | **CUT** ◀ '**52**' *(short frame)* sent after $t_P$ ◀ **LT** | WUPA to poll for the CUT |
| 23+j | **CUT** ▶ ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ ▶ **LT** | ATQA |
| 24+j | **CUT** ◀ '**50 00**' ◀ **LT** | HLTA |
| 25+j | **CUT** ◀ '**52**' *(short frame)* sent after $t_P$ ◀ **LT** | WUPA to wake up the CUT |
| 26+j | **CUT** ▶ ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ ▶ **LT** | ATQA |
| 27+j | **CUT** ◀ '**50 00**' ◀ **LT** | HLTA |
| 28+j | **CUT** ◀ '**52**' *(short frame)* sent after $t_P$ ◀ **LT** | WUPA to wake up the CUT |
| 29+j | **CUT** ▶ ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ ▶ **LT** | ATQA |
| 30+j | **CUT** ◀ '**50 00**' ◀ **LT** | HLTA |
| 31+j | **CUT** ◀ '**52**' *(short frame)* sent after $t_P$ ◀ **LT** | WUPA to wake up the CUT |
| 32+j | **CUT** ▶ ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ ▶ **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 2: Basic Type A Exchange and Timings Measurement (UID size = 2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.3. Basic Type A Exchange and Timings Measurement (UID size = 3) [CA003]

**Test Codification:**

CA003

**ICS Parameters:**

PICC Type = A / UID size = 3

**Test Objective:**

To ensure that the Type A PICC with a triple size UID respects the frame format, the timings and the order of the commands during a basic exchange (installation and block protocol).

**Reference Requirements:**

4.7.1.1, 4.7.1.2, 4.7.1.3, 4.8.1.2, 4.8.1.5, 4.8.1.9, 4.8.1.11, 4.8.1.13, 5.3.2.3, 5.5.2.2, 5.7.1.1, 5.7.2.1, 5.7.2.3, 5.7.2.4, 5.7.2.7, 5.7.2.9, 5.7.2.10, 5.7.2.12, 5.7.2.15, 7.1.1.5, 10.3.4.8

**Procedure:**

Run the following scenario.

When a sequence sent by the LT is followed by a sequence sent by the CUT, the LT measures the Frame Delay Time between the end of the sequence sent by the LT and the beginning of the sequence replied by the CUT.

Remark: The Frame Delay Time used by the CUT to reply to any command shall also be measured. And, if the CUT uses chaining or sends WTX Requests when replying to any command, then the measurement is performed for each sequence sent by the CUT until the next command is sent by the LT.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

Each frame sent by the CUT is a Type A standard frame composed of a Start of Frame i.e. SoF = Logic “1”, followed by the bytes coding the CUT response (and eventually the CRC). For each byte, the 8 bits are transmitted LSB first and are followed by an odd parity bit.

Each Type A frame sent by the CUT is contained in a sequence ending with an End of Sequence i.e. EoS = the carrier is not modulated with the subcarrier for one bit duration.

The ATQA, UID CL1 + BCC, UID CL2 + BCC and UID CL3 + BCC frames sent by the CUT do not contain any CRC_A bytes. Other Commands and Blocks frames sent by the CUT contain 2 CRC_A bytes (last two bytes of the frames) calculated as defined in ISO/IEC 13239 (with a register initialized to ‘6363’).

On reception of the WUPA Command before the RESET, the CUT sends always the same ATQA. On reception of the WUPA after the RESET, all the ATQA have the same value. An ATQA shall always be sent with the most significant byte (Byte 1) and the least significant byte (Byte 2) having the value:
Byte 1 = (“1001 0000”)$_b$ or
(“1000 1000”)$_b$ or
(“1000 0100”)$_b$ or
(“1000 0010”)$_b$ or
(“1000 0001”)$_b$
Byte 2 = (“0000 uuuu”)$_b$ where the bits indicated as ‘u’ may take any value.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

On reception of the ANTICOLLISION CL1 Command, the CUT sends a UID CL1 + BCC with the most significant byte equal to $(‘88’)_h$.

On reception of the ANTICOLLISION CL2 Command, the CUT sends a UID CL2 + BCC with the most significant byte equal to $(‘88’)_h$.

The byte BCC (least significant byte) of UID CLn + BCC (n=1, 2 or 3) is equal to the result of an exclusive-or over the 4 previous bytes UID CLn (4 most significant bytes).

On reception of the SELECT CL1 and SELECT CL2 Commands, the CUT sends a SAK byte having the value: SAK = $(“uu1u\ u1uu”)_b$ where the bits indicated as ‘u’ may take any value.

Remark: For backward compatibility the bit 6 of SAK sent as response to the SELECT CL1 and SELECT CL2 commands remains $(“1”)_b$.

On reception of the SELECT CL3 Command, the CUT sends a SAK byte having the value: SAK = $(“uu1u\ u0uu”)_b$ where the bits indicated as ‘u’ may take any value.

On reception of the RATS Command, the CUT sends an ATS having the value:

| **ATS** | | | | | |
|---|---|---|---|---|---|
| Byte 1 | Byte 2 | Byte 3 | Byte 4 | Byte 5 | Byte 6 – Byte 6+k |
| TL | T0 | TA(1) | TB(1) | TC(1) | Historical Bytes |

Where:

- TL = (number of Historical Bytes + 5) and $(‘05’)_h \leq TL \leq (‘14’)_h$
- T0 = $(“0111\ aaaa”)_b$ with $(“aaaa”)_b$ = FSCI where $(‘2’)_h \leq FSCI \leq (‘C’)_h$
- TA(1) = $(‘00’)_h$ or $(‘80’)_h$
- TB(1) = $(“XY”)_h$ with
  - X = FWI where $X \leq (‘7’)_h$
  - Y = SFGI where $Y \leq (‘8\ ’)_h$
- TC(1) = $(“0000\ 00uu”)_b$ where the bits indicated as ‘u’ may take any value
- Historical Bytes = up to 15 bytes (k ≤ 14) which may take any value

On reception of an I-Block with a block number equal to 0, the CUT replies with an I-Block having a PCB field (i.e. first byte of the Block) equal to $(‘02’)_h$ or $(‘12’)_h$.

On reception of an I-Block with a block number equal to 1, the CUT replies with an I-Block having a PCB field (i.e. first byte of the Block) equal to $(‘03’)_h$ or $(‘13’)_h$.

Remark: If a block is received with a PCB field equal to $(‘12’)_h$ or $(‘13’)_h$, it means that the CUT is using chaining.

The delay $FDT_{A,PICC}$ between the end of a sequence sent by the LT and the beginning of the next sequence sent by the CUT is aligned to the following time grid:

- $FDT_{A,PICC} = (n \times 128 + 20)/ f_c)$ if the last bit transmitted by the LT is $(0)_b$
- $FDT_{A,PICC} = (n \times 128 + 84)/ f_c)$ if the last bit transmitted by the LT is $(1)_b$

with the integer n ≥ 9 (see below).

During installation after WUPA, ANTICOLLISION CLn (n=1 to 3) and SELECT CLn (n=1 to 3) and after WUPA Commands following the PICC Reset, the delay between the end of the sequence sent by the LT and the beginning of the next sequence sent by the CUT is equal to $FDT_{A,PICC,ANTICOLLISION}$ with a tolerance of 0 to 0.4µs.

After the RATS Command, the delay between the end of the sequence sent by the LT and the beginning of the next sequence sent by the CUT is at least $FDT_{A,PICC,MIN}$ and is at most $FWT_{ACTIVATION}$.

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that does not contain an S(WTX) Response and the beginning

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

of the next sequence sent by the CUT is at least $FDT_{PICC,MIN}$ and is at most FWT = (256 x 16/$f_c$) x $2^{FWI}$.

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that contains an S(WTX) Response and the beginning of the next sequence sent by the CUT is at least $FDT_{PICC,MIN}$ and is at most $FWT_{TEMP}$ = ((256 x 16/$f_c$) x $2^{FWI}$) x WTXM where WTXM is the FWT Extension Multiplier indicated in the S(WTX) Request and (WTXM x (256 x 16/ $f_c$)x $2^{FWI}$) ≤ $FWT_{MAX}$.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◄ | '**52**' *(short frame)* | ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | ATQA |
| 3 | **CUT** ◄ | '**50 00**' | ◄ **LT** | HLTA |
| 4 | **CUT** ◄ | **'05 00 08'** *(Type B frame)* sent after $t_P$ | ◄ **LT** | WUPB |
| 5 | **CUT** ◄ | '**52**' *(short frame)* sent after $t_P$ | ◄ **LT** | WUPA to wake up the CUT |
| 6 | **CUT** ► | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | ATQA |
| 7 | **CUT** ◄ | '**93 20**' *(no CRC_A)* | ◄ **LT** | ANTICOLLISION CL1 |
| 8 | **CUT** ► | ***UID CL1 + BCC*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | UID CL1 + BCC |
| 9 | **CUT** ◄ | '**93 70**' + ***UID CL1 + BCC*** | ◄ **LT** | SELECT CL1 |
| 10 | **CUT** ► | ("uu1u u1uu")$_b$ sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | SAK |
| 11 | **CUT** ◄ | '**95 20**' *(no CRC_A)* | ◄ **LT** | ANTICOLLISION CL2 |
| 12 | **CUT** ► | ***UID CL2 + BCC*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | UID CL2 + BCC |
| 13 | **CUT** ◄ | '**95 70**' + ***UID CL2 + BCC*** | ◄ **LT** | SELECT CL2 |
| 14 | **CUT** ► | ("uu1u u1uu")$_b$ sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | SAK |
| 15 | **CUT** ◄ | '**97 20**' *(no CRC_A)* | ◄ **LT** | ANTICOLLISION CL3 |
| 16 | **CUT** ► | ***UID CL3 + BCC*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | UID CL3 + BCC |
| 17 | **CUT** ◄ | '**97 70**' + ***UID CL3 + BCC*** | ◄ **LT** | SELECT CL3 |
| 18 | **CUT** ► | ("uu1u u0uu")$_b$ sent at $FDT_{A,PICC,ANTICOLLISION}$ | ► **LT** | SAK |
| 19 | **CUT** ◄ | '**E0 80**' | ◄ **LT** | RATS (FSD = 256) |
| 20 | **CUT** ► | ***ATS*** sent between $FDT_{A,PICC,MIN}$ and $FWT_{ACTIVATION}$ | ► **LT** | ATS |
| 21 | **CUT** ◄ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◄ **LT** | Select PPSE / b.n. = 0 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 22 | **CUT** ▸ | **I(0)$_0$ ['6F' +** n data bytes + '**90 00**'**]** sent between $FDT_{A,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | ▸ **LT** | b.n. = 0 |
| 23 | **CUT** ◂ | **I(0)$_1$ ['00 A4 04 00'** + Lc + AID + '**00**'**]** | ◂ **LT** | Select AID / b.n. = 1 |
| 24 | **CUT** ▸ | **I(0)$_1$ ['6F' +** n data bytes + '**90 00**'**]** sent between $FDT_{A,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | ▸ **LT** | b.n. = 1 |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of R-APDUs are sent between $FDT_{A,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | | | |
| 25+j | **CUT** ◂ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◂ **LT** | PICC Reset |
| 26+j | **CUT** ◂ | '**52**' *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to poll for the CUT |
| 27+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▸ **LT** | ATQA |
| 28+j | **CUT** ◂ | '**50 00**' | ◂ **LT** | HLTA |
| 29+j | **CUT** ◂ | '**52**' *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to wake up the CUT |
| 30+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▸ **LT** | ATQA |
| 31+j | **CUT** ◂ | '**50 00**' | ◂ **LT** | HLTA |
| 32+j | **CUT** ◂ | '**52**' *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to wake up the CUT |
| 33+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▸ **LT** | ATQA |
| 34+j | **CUT** ◂ | '**50 00**' | ◂ **LT** | HLTA |
| 35+j | **CUT** ◂ | '**52**' *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to wake up the CUT |
| 36+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* sent at $FDT_{A,PICC,ANTICOLLISION}$ | ▸ **LT** | ATQA |
| End of test: Reset PICC | | | | |

**Scenario 3: Basic Type A Exchange and Timings Measurement (UID size = 3)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.4. Basic Type A Exchange with the minimum and longer Frame Delay Times PCD→PICC (UID size = 1) [CA010.XY]

**Test Codification:**

CA010.xy

**ICS Parameters:**

PICC Type = A / UID size = 1

**Test Objective:**

To ensure that the Type A PICC with a single size UID accepts sequences received with the minimum and longer Frame Delay Time during installation and block protocol.

**Reference Requirements:**

4.8.2.2, 4.8.2.4, 4.8.2.6, 7.1.1.5, 7.2.8.3, 10.1.2.1, 10.3.6.1, 10.3.6.2

**Procedure:**

Run the following scenarios.

In the case of 2 consecutive sequences sent in opposite directions, the LT sends all its responses and blocks (but the first block of the block protocol) using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT:

- For xy=00 and xy=10: $FDT_{PCD} = FDT_{A,PCD,MIN}$
- For xy=01: $FDT_{PCD}$ = 100 ms

On reception of the ATS sent by the CUT, the LT sends the first I-Block of the half-duplex block protocol using the delay SFGT between the end of the sequence sent by the CUT and the beginning of the sequence sent by the LT:

- For xy=00 and xy=10:
  - SFGT = $FDT_{A,PCD,MIN}$ if SFGI = 0
  - SFGT = $(256 \times 16/ f_c) \times 2^{SFGI}$ if SFGI > 0
- For xy=01:
  - SFGT = 100 ms

Remark: For x=0, the EMV CL v.2.0 installation procedure is performed. For x=1, a “backward compatibility” installation procedure is performed.

For xy=10:
When a sequence that contains a S(DESELECT) sent by the LT is followed by a sequence that contains a S(DESELECT) sent by the CUT, the LT measures the delay between the end of the sequence sent by the LT and the beginning of the sequence replied by the CUT.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT accepts and replies to the sequences sent by the LT with the minimum Frame Delay Time).

For xy=10:
The delay between the sequence that contains the S(DESELECT) sent by the LT and the sequence that contains the S(DESELECT) replied by the CUT is at least $FDT_{A,PICC,MIN}$ and at most $FWT_{DESELECT}$.
On reception of an S(DESELECT) Block, the CUT replies with an S(DESELECT) Block having a PCB field (i.e. first byte of the Block) equal to $(\text{'C2'})_h$ (i.e. $b_2$ of S-Block PCB is set to $(1)_b$).

**Pre-Processing with chaining:**

If the test script needs to be processed because the CUT sends an I-Block indicating chaining during the transaction, then the LT shall send the R(ACK) Block using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT.

**Pre-Processing with S(WTX):**

If the test script needs to be processed because the CUT sends a S(WTX) Request during the transaction, then the LT shall send the S(WTX) Response using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**50 00**' sent using the Frame Delay Time $FDT_{PCD}$ ◀ **LT** | HLTA |
| 4 | **CUT** ◀ '**05 00 08**' *(Type B frame)* sent after $t_P$ ◀ **LT** | WUPB |
| 5 | **CUT** ◀ '**52**' *(short frame)* sent after $t_P$ ◀ **LT** | WUPA to wake up the CUT |
| 6 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 7 | **CUT** ◀ '**93 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ ◀ **LT** | ANTICOLLISION CL1 |
| 8 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 9 | **CUT** ◀ '**93 70**' **+ *UID CL1 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ ◀ **LT** | SELECT CL1 |
| 10 | **CUT** ▶ $(\text{"uu1u u0uu"})_b$ ▶ **LT** | SAK |
| 11 | **CUT** ◀ **'E0 80'** sent using the Frame Delay Time $FDT_{PCD}$ ◀ **LT** | RATS (FSD = 256) |
| 12 | **CUT** ▶ ***ATS*** ▶ **LT** | ATS |
| 13 | **CUT** ◀ **$I(0)_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'**] sent using the delay SFGT ◀ **LT** | Select PPSE / b.n. = 0 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 14 | **CUT** ▸ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▸ **LT** | b.n. = 0 |
| 15 | **CUT** ◂ | **I(0)$_1$ ['00 A4 04 00'** + Lc + AID + **'00']** sent using the Frame Delay Time FDT$_{PCD}$ | ◂ **LT** | Select AID / b.n. = 1 |
| 16 | **CUT** ▸ | **I(0)$_1$ ['6F' +** n data bytes + **'90 00']** | ▸ **LT** | b.n. = 1 |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent using the Frame Delay Time FDT$_{PCD}$ | | | |
| 17+j | **CUT** ◂ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◂ **LT** | PICC Reset |
| 18+j | **CUT** ◂ | '**52**' *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to poll for the CUT |
| 19+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 20+j | **CUT** ◂ | '**50 00**' sent using the Frame Delay Time FDT$_{PCD}$ | ◂ **LT** | HLTA |
| 21+j | **CUT** ◂ | '**52**' *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to poll for the CUT |
| 22+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| End of test: Reset PICC | | | | |

## Scenario 4: Basic Type A Exchange with the minimum and longer Frame Delay Time PCD→PICC (UID size = 1, xy=00 to xy=01)

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◂ | '**52**' *(short frame)* | ◂ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 3 | **CUT** ◂ | '**93 20**' *(no CRC_A)* sent using the Frame Delay Time FDT$_{PCD}$ | ◂ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▸ | ***UID CL1 + BCC*** *(no CRC_A)* | ▸ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◂ | '**93 70**' **+** ***UID CL1 + BCC*** sent using the Frame Delay Time FDT$_{PCD}$ | ◂ **LT** | SELECT CL1 |
| 6 | **CUT** ▸ | ("uu1u u0uu")$_b$ | ▸ **LT** | SAK |
| 7 | **CUT** ◂ | '**50 00**' sent using the Frame Delay Time FDT$_{PCD}$ | ◂ **LT** | HLTA |
| 8 | **CUT** ◂ | '**26**' *(short frame)* sent after $t_P$ | ◂ **LT** | REQA |
| 9 | **CUT** ◂ | '**05 00 08**' *(Type B frame)* sent after $t_P$ | ◂ **LT** | WUPB |
| 10 | **CUT** ◂ | '**52**' *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to wake up the CUT |
| 11 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 12 | **CUT** ◀ | **'93 70' + *UID CL1* + *BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | SELECT CL1 |
| 13 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 14 | **CUT** ◀ | **'E0 80'** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | RATS (FSD = 256) |
| 15 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |
| 16 | **CUT** ◀ | **$I(0)_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'**] sent using the delay SFGT | ◀ **LT** | Select PPSE / b.n. = 0 |
| 17 | **CUT** ▶ | **$I(0)_0$ ['6F' +** n data bytes + '**90 00'**] | ▶ **LT** | b.n. = 0 |
| 18 | **CUT** ◀ | **$I(0)_1$ ['00 A4 04 00'** + Lc + AID + '**00'**] sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | Select AID / b.n. = 1 |
| 19 | **CUT** ▶ | **$I(0)_1$ ['6F' +** n data bytes + '**90 00'**] | ▶ **LT** | b.n. = 1 |
| **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent using the Frame Delay Time $FDT_{PCD}$ | | | | |
| 20+j | **CUT** ◀ | **S(DESELECT) Request** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | DESELECT Request |
| 21+j | **CUT** ▶ | **S(DESELECT) Response** sent between $FDT_{A,PICC,MIN}$ and $FWT_{DESELECT}$ | ◀ **LT** | DESELECT Response |
| 22+j | **CUT** ◀ | '**52**' *(short frame)* sent after $t_P$ | ◀ **LT** | WUPA to poll for the CUT |
| 23+j | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 24+j | **CUT** ◀ | '**50 00**' sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | HLTA |
| 25+j | **CUT** ◀ | '**52**' *(short frame)* sent after $t_P$ | ◀ **LT** | WUPA to poll for the CUT |
| 26+j | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| End of test: Reset PICC | | | | |

## Scenario 5: Basic Type A Exchange with the minimum Frame Delay Time PCD→PICC (UID size = 1, xy=10)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.5. Basic Type A Exchange with the minimum and longer Frame Delay Times PCD→PICC (UID size = 2) [CA011.XY]

**Test Codification:**

CA011.xy

**ICS Parameters:**

PICC Type = A / UID size = 2

**Test Objective:**

To ensure that the Type A PICC with a double size UID accepts sequences received with the minimum and longer Frame Delay Time during installation and block protocol.

**Reference Requirements:**

4.8.2.2, 4.8.2.2, 7.1.1.5, 10.1.2.1, 10.3.6.1, 10.3.6.2

**Procedure:**

Run the following scenarios.

In the case of 2 consecutive sequences sent in opposite directions, the LT sends all its responses and blocks (but the first block of the block protocol) using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT:

- For xy=00 and xy=10: $FDT_{PCD} = FDT_{A,PCD,MIN}$
- For xy=01: $FDT_{PCD}$ = 100 ms

On reception of the ATS sent by the CUT, the LT sends the first I-Block of the half-duplex block protocol using the delay SFGT between the end of the sequence sent by the CUT and the beginning of the sequence sent by the LT:
For xy=00 and xy=10:

- SFGT = $FDT_{A,PCD,MIN}$ if SFGI = 0
- SFGT = $(256 \times 16/ f_c) \times 2^{SFGI}$ if SFGI > 0

For xy=01: SFGT = 100 ms

Remark: For x=0, the EMV CL v.2.0 installation procedure is performed. For x=1, a “backward compatibility” installation procedure is performed.

For xy=10:
When a sequence that contains a S(DESELECT) sent by the LT is followed by a sequence that contains a S(DESELECT) sent by the CUT, the LT measures the delay between the end of the sequence sent by the LT and the beginning of the sequence replied by the CUT.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT accepts and replies to the sequences sent by the LT with the minimum Frame Delay Time).

For xy=10:
The delay between the sequence that contains the S(DESELECT) sent by the LT and the sequence that contains the S(DESELECT) replied by the CUT is at least $FDT_{A,PICC,MIN}$ and at most $FWT_{DESELECT}$.
On reception of an S(DESELECT) Block, the CUT replies with an S(DESELECT) Block having a PCB field (i.e. first byte of the Block) equal to $(\text{'C2'})_h$ (i.e. $b_2$ of S-Block PCB is set to $(1)_b$).

**Pre-Processing with chaining:**

If the test script needs to be processed because the CUT sends an I-Block indicating chaining during the transaction, then the LT shall send the R(ACK) Block using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT.

**Pre-Processing with S(WTX):**

If the test script needs to be processed because the CUT sends a S(WTX) Request during the transaction, then the LT shall send the S(WTX) Response using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◄ | '**52**' *(short frame)* | ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► | ***ATQA*** *(no CRC_A)* | ► **LT** | ATQA |
| 3 | **CUT** ◄ | '**50 00**' sent using the Frame Delay Time $FDT_{PCD}$ | ◄ **LT** | HLTA |
| 4 | **CUT** ◄ | '**05 00 08**' *(Type B frame)* sent after $t_P$ | ◄ **LT** | WUPB |
| 5 | **CUT** ◄ | '**52**' *(short frame)* sent after $t_P$ | ◄ **LT** | WUPA to wake up the CUT |
| 6 | **CUT** ► | ***ATQA*** *(no CRC_A)* | ► **LT** | ATQA |
| 7 | **CUT** ◄ | '**93 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◄ **LT** | ANTICOLLISION CL1 |
| 8 | **CUT** ► | ***UID CL1 + BCC*** *(no CRC_A)* | ► **LT** | UID CL1 + BCC |
| 9 | **CUT** ◄ | '**93 70**' **+ *UID CL1 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◄ **LT** | SELECT CL1 |
| 10 | **CUT** ► | ("uu1u u1uu")$_b$ | ► **LT** | SAK |
| 11 | **CUT** ◄ | '**95 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◄ **LT** | ANTICOLLISION CL2 |
| 12 | **CUT** ► | ***UID CL2 + BCC*** *(no CRC_A)* | ► **LT** | UID CL2 + BCC |
| 13 | **CUT** ◄ | '**95 70**' **+ *UID CL2 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◄ **LT** | SELECT CL2 |
| 14 | **CUT** ► | ("uu1u u0uu")$_b$ | ► **LT** | SAK |
| 15 | **CUT** ◄ | **'E0 80'** sent using the Frame Delay Time $FDT_{PCD}$ | ◄ **LT** | RATS (FSD = 256) |
| 16 | **CUT** ► | ***ATS*** | ► **LT** | ATS |
| 17 | **CUT** ◄ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** sent using the delay SFGT | ◄ **LT** | Select PPSE / b.n. = 0 |
| 18 | **CUT** ► | **I(0)$_0$ ['6F'** **+** n data bytes + **'90 00']** | ► **LT** | b.n. = 0 |
| 19 | **CUT** ◄ | **I(0)$_1$ ['00 A4 04 00'** + Lc + AID + **'00']** sent using the Frame Delay Time $FDT_{PCD}$ | ◄ **LT** | Select AID / b.n. = 1 |
| 20 | **CUT** ► | **I(0)$_1$ ['6F'** **+** n data bytes + **'90 00']** | ► **LT** | b.n. = 1 |
| **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent using the Frame Delay Time $FDT_{PCD}$ | | | | |
| 21+j | **CUT** ◄ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◄ **LT** | PICC Reset |
| 22+j | **CUT** ◄ | '**52**' *(short frame)* sent after $t_P$ | ◄ **LT** | WUPA to poll for the CUT |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 23+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 24+j | **CUT** ◂ | '**50 00**' sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | HLTA |
| 25+j | **CUT** ◂ | '**52**' *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to poll for the CUT |
| 26+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| End of test: Reset PICC | | | | |

## Scenario 6: Basic Type A Exchange with the minimum and longer Frame Delay Time PCD→PICC (UID size = 2, xy=00 to xy=01)

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◂ | '**52**' *(short frame)* | ◂ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 3 | **CUT** ◂ | '**93 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▸ | ***UID CL1 + BCC*** *(no CRC_A)* | ▸ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◂ | '**93 70**' **+ *UID CL1 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | SELECT CL1 |
| 6 | **CUT** ▸ | ("uu1u u1uu")$_b$ | ▸ **LT** | SAK |
| 7 | **CUT** ◂ | '**95 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▸ | ***UID CL2 + BCC*** *(no CRC_A)* | ▸ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◂ | '**95 70**' **+ *UID CL2 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | SELECT CL2 |
| 10 | **CUT** ▸ | ("uu1u u0uu")$_b$ | ▸ **LT** | SAK |
| 11 | **CUT** ◂ | '**50 00**' sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | HLTA |
| 12 | **CUT** ◂ | '**26**' *(short frame)* sent after $t_P$ | ◂ **LT** | REQA |
| 13 | **CUT** ◂ | '**05 00 08**' *(Type B frame)* sent after $t_P$ | ◂ **LT** | WUPB |
| 14 | **CUT** ◂ | '**52**' *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to wake up the CUT |
| 15 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 16 | **CUT** ◂ | '**93 70**' **+ *UID CL1 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | SELECT CL1 |
| 17 | **CUT** ▸ | ("uu1u u1uu")$_b$ | ▸ **LT** | SAK |
| 18 | **CUT** ◂ | '**95 70**' **+ *UID CL2 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | SELECT CL2 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 19 | **CUT** ▸ | (“uu1u u0uu”)$_b$ | ▸ **LT** | SAK |
| 20 | **CUT** ◂ | **‘E0 80’** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | RATS (FSD = 256) |
| 21 | **CUT** ▸ | ***ATS*** | ▸ **LT** | ATS |
| 22 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** sent using the delay SFGT | ◂ **LT** | Select PPSE / b.n. = 0 |
| 23 | **CUT** ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | b.n. = 0 |
| 24 | **CUT** ◂ | **I(0)$_1$ [’00 A4 04 00’** + Lc + AID + **‘00’]** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | Select AID / b.n. = 1 |
| 25 | **CUT** ▸ | **I(0)$_1$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | b.n. = 1 |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent using the Frame Delay Time $FDT_{PCD}$ | | | |
| 26+j | **CUT** ◂ | **S(DESELECT) Request** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | DESELECT Request |
| 27+j | **CUT** ▸ | **S(DESELECT) Response** sent between $FDT_{A,PICC,MIN}$ and $FWT_{DESELECT}$ | ◂ **LT** | DESELECT Response |
| 28+j | **CUT** ◂ | ‘**52**’ *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to poll for the CUT |
| 29+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 30+j | **CUT** ◂ | ’**50 00**’ sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | HLTA |
| 31+j | **CUT** ◂ | ‘**52**’ *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to poll for the CUT |
| 32+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| End of test: Reset PICC | | | | |

## Scenario 7: Basic Type A Exchange with the minimum Frame Delay Time PCD→PICC (UID size = 2, xy=10)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.6. Basic Type A Exchange with the minimum and longer Frame Delay Times PCD→PICC (UID size = 3) [CA012.XY]

**Test Codification:**

CA012.xy

**ICS Parameters:**

PICC Type = A / UID size = 3

**Test Objective:**

To ensure that the Type A PICC with a triple size UID accepts sequences received with the minimum and longer Frame Delay Time during installation and block protocol.

**Reference Requirements:**

4.8.2.2, 4.8.2.2, 7.1.1.5, 10.1.2.1, 10.3.6.1, 10.3.6.2

**Procedure:**

Run the following scenarios.

In the case of 2 consecutive sequences sent in opposite directions, the LT sends all its responses and blocks (but the first block of the block protocol) using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT:

- For xy=00 and xy=10: $FDT_{PCD} = FDT_{A,PCD,MIN}$
- For xy=01: $FDT_{PCD}$ = 100 ms

On reception of the ATS sent by the CUT, the LT sends the first I-Block of the half-duplex block protocol using the delay SFGT between the end of the sequence sent by the CUT and the beginning of the sequence sent by the LT:
For xy=00 and xy=10:

- SFGT = $FDTA_{,PCD,MIN}$ if SFGI = 0
- SFGT = $(256 \times 16/ f_c) \times 2^{SFGI}$ if SFGI > 0

For xy=01: SFGT = 100 ms

Remark: For x=0, the EMV CL v.2.0 installation procedure is performed. For x=1, a “backward compatibility” installation procedure is performed.

For xy=10:
When a sequence that contains a S(DESELECT) sent by the LT is followed by a sequence that contains a S(DESELECT) sent by the CUT, the LT measures the delay between the end of the sequence sent by the LT and the beginning of the sequence replied by the CUT.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT accepts and replies to the sequences sent by the LT with the minimum Frame Delay Time).

For xy=10:
The delay between the sequence that contains the S(DESELECT) sent by the LT and the sequence that contains the S(DESELECT) replied by the CUT is at least $FDT_{A,PICC,MIN}$ and at most $FWT_{DESELECT}$.
On reception of an S(DESELECT) Block, the CUT replies with an S(DESELECT) Block having a PCB field (i.e. first byte of the Block) equal to $(\text{'C2'})_h$ (i.e. $b_2$ of S-Block PCB is set to $(1)_b$).

**Pre-Processing with chaining:**

If the test script needs to be processed because the CUT sends an I-Block indicating chaining during the transaction, then the LT shall send the R(ACK) Block using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT.

**Pre-Processing with S(WTX):**

If the test script needs to be processed because the CUT sends a S(WTX) Request during the transaction, then the LT shall send the S(WTX) Response using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**50 00**' sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | HLTA |
| 4 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent after $t_P$ | ◀ **LT** | WUPB |
| 5 | **CUT** ◀ | '**52**' *(short frame)* sent after $t_P$ | ◀ **LT** | WUPA to wake up the CUT |
| 6 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 7 | **CUT** ◀ | '**93 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | ANTICOLLISION CL1 |
| 8 | **CUT** ▶ | ***UID CL1 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 9 | **CUT** ◀ | '**93 70**' **+ *UID CL1 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | SELECT CL1 |
| 10 | **CUT** ▶ | $(\text{"uu1u u1uu"})_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | '**95 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | ANTICOLLISION CL2 |
| 12 | **CUT** ▶ | ***UID CL2 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 13 | **CUT** ◀ | '**95 70**' **+ *UID CL2 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | SELECT CL2 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 14 | **CUT** ▸ | (“uu1u u1uu”)$_b$ | ▸ **LT** | SAK |
| 15 | **CUT** ◂ | ’**97 20**’ *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | ANTICOLLISION CL3 |
| 16 | **CUT** ▸ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL3 + BCC |
| 17 | **CUT** ◂ | ’**97 70**’ **+ *UID CL3* + *BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | SELECT CL3 |
| 18 | **CUT** ▸ | (“uu1u u0uu”)$_b$ | ▸ **LT** | SAK |
| 19 | **CUT** ◂ | **‘E0 80’** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | RATS (FSD = 256) |
| 20 | **CUT** ▸ | ***ATS*** | ▸ **LT** | ATS |
| 21 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** sent using the delay SFGT | ◂ **LT** | Select PPSE / b.n. = 0 |
| 22 | **CUT** ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | b.n. = 0 |
| 23 | **CUT** ◂ | **I(0)$_1$ [’00 A4 04 00’** + Lc + AID + **‘00’]** sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | Select AID / b.n. = 1 |
| 24 | **CUT** ▸ | **I(0)$_1$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | b.n. = 1 |
| **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent using the Frame Delay Time $FDT_{PCD}$ | | | | |
| 25+j | **CUT** ◂ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◂ **LT** | PICC Reset |
| 26+j | **CUT** ◂ | ‘**52**’ *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to poll for the CUT |
| 27+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 28+j | **CUT** ◂ | ’**50 00**’ sent using the Frame Delay Time $FDT_{PCD}$ | ◂ **LT** | HLTA |
| 29+j | **CUT** ◂ | ‘**52**’ *(short frame)* sent after $t_P$ | ◂ **LT** | WUPA to poll for the CUT |
| 30+j | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| End of test: Reset PICC | | | | |

## Scenario 8: Basic Type A Exchange with the minimum and longer Frame Delay Time PCD→PICC (UID size = 3, xy=00 to xy=01)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' **+ *UID CL1 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | '**95 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ | ***UID CL2 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ | '**95 70**' **+ *UID CL2 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | '**97 20**' *(no CRC_A)* sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ▶ | ***UID CL3 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◀ | '**97 70**' **+ *UID CL3 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | SELECT CL3 |
| 14 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 15 | **CUT** ◀ | '**50 00**' sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | HLTA |
| 16 | **CUT** ◀ | '**26**' *(short frame)* sent after $t_P$ | ◀ **LT** | REQA |
| 17 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent after $t_P$ | ◀ **LT** | WUPB |
| 18 | **CUT** ◀ | '**52**' *(short frame)* sent after $t_P$ | ◀ **LT** | WUPA to wake up the CUT |
| 19 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 20 | **CUT** ◀ | '**93 70**' **+ *UID CL1 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | SELECT CL1 |
| 21 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 22 | **CUT** ◀ | '**95 70**' **+ *UID CL2 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | SELECT CL2 |
| 23 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 24 | **CUT** ◀ | '**97 70**' **+ *UID CL3 + BCC*** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | SELECT CL3 |
| 25 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 26 | **CUT** ◀ | **'E0 80'** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | RATS (FSD = 256) |
| 27 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |
| 28 | **CUT** ◀ | **$I(0)_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'**] sent using the delay SFGT | ◀ **LT** | Select PPSE / b.n. = 0 |
| 29 | **CUT** ▶ | **$I(0)_0$ ['6F' +** n data bytes + '**90 00'**] | ▶ **LT** | b.n. = 0 |
| 30 | **CUT** ◀ | **$I(0)_1$ ['00 A4 04 00'** + Lc + AID + '**00'**] sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | Select AID / b.n. = 1 |
| 31 | **CUT** ▶ | **$I(0)_1$ ['6F' +** n data bytes + '**90 00'**] | ▶ **LT** | b.n. = 1 |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent using the Frame Delay Time $FDT_{PCD}$ | | | |
| 32+j | **CUT** ◀ | **S(DESELECT) Request** sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | DESELECT Request |
| 33+j | **CUT** ▶ | **S(DESELECT) Response** sent between $FDT_{A,PICC,MIN}$ and $FWT_{DESELECT}$ | ◀ **LT** | DESELECT Response |
| 34+j | **CUT** ◀ | '**52**' *(short frame)* sent after $t_P$ | ◀ **LT** | WUPA to poll for the CUT |
| 35+j | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 36+j | **CUT** ◀ | '**50 00**' sent using the Frame Delay Time $FDT_{PCD}$ | ◀ **LT** | HLTA |
| | **CUT** ◀ | '**52**' *(short frame)* sent after $t_P$ | ◀ **LT** | WUPA to poll for the CUT |
| 38+j | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| End of test: Reset PICC | | | | |

## Scenario 9: Basic Type A Exchange with the minimum Frame Delay Time PCD→PICC (UID size = 3, xy=10)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.7. Type A Correct Installation with respect of the anticollision state machine (UID size = 1) [CA116]

**Test Codification:**

CA116

**ICS Parameters:**

PICC Type = A / UID size = 1

**Test Objective:**

To ensure that the Type A PICC with a single size UID stays in the READY state and sends its UID CL1 when it receives a valid ANTICOLLISION CL1 Command in the READY state.

**Reference Requirements:**

7.2.2.1, 7.2.3.1, 7.3, 7.4, 7.2.6.1, 7.5

**Procedure:**

Run the following scenarios.

On reception of the ATQA, the LT sends the ANTICOLLISION CL1 Command and waits for the UID CL1 three consecutive times (i.e. three consecutive AC CL1 / UID CL1 exchanges).

On reception of the third UID CL1, the LT sends the SELECT CL1 Command, waits for a SAK from the CUT and then sends the RATS Command.

**Acceptance Criteria:**

The CUT behaves as as described in the following scenario (the CUT replies to the SELECT CL1 Command with a SAK equal to (“uu1u u0uu”)$_b$ and to the RATS Command with a correct ATS).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 6 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 7 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 8 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 9 | **CUT** ◄ '**93 70**' **+** ***UID CL1 + BCC*** ◄ **LT** | SELECT CL1 |
| 10 | **CUT** ► ("uu1u u0uu")$_b$ ► **LT** | SAK |
| 11 | **CUT** ◄ '**E0 80**' ◄ **LT** | RATS (FSD = 256) |
| 12 | **CUT** ► **ATS** ► **LT** | ATS |
| End of test: Reset PICC | | |

**Scenario 10: Type A Correct Installation with respect of the anticollision state machine (UID size = 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.8. Type A Correct Installation with respect of the anticollision state machine (UID size = 2) [CA117]

**Test Codification:**

CA117

**ICS Parameters:**

PICC Type = A / UID size = 2

**Test Objective:**

To ensure that the Type A PICC with a double size UID stays in the READY state and sends its UID CL1 when it receives a valid ANTICOLLISION CL1Command in the READY state.

To ensure that the Type A PICC with a double size UID stays in the READY' state and sends its UID CL2 when it receives a valid ANTICOLLISION CL1Command in the READY' state.

**Reference Requirements:**

7.2.2.1, 7.2.3.1, 7.2.4.1, 7.3, 7.4, 7.2.6.1, 7.5

**Procedure:**

Run the following scenario.

On reception of the ATQA, the LT sends the ANTICOLLISION CL1 Command and waits for the UID CL1 three consecutive times (i.e. three consecutive AC CL1 / UID CL1 exchanges).

On reception of the third UID CL1, the LT sends the SELECT CL1 Command, waits for a SAK from the CUT and then sends the ANTICOLLISION CL2 Command and waits for the UID CL2 three consecutive times (i.e. three consecutive AC CL2 / UID CL2 exchanges).

On reception of the third UID CL2, the LT sends the SELECT CL2 Command, waits for a SAK from the CUT and then sends the RATS Command.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT replies to the SELECT CL2 Command with a SAK equal to (“uu1u u0uu”)$_b$ and to the RATS Command with a correct ATS).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 6 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 7 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 8 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 9 | **CUT** ◄ '**93 70**' **+** ***UID CL1 + BCC*** ◄ **LT** | SELECT CL1 |
| 10 | **CUT** ► ("uu1u u1uu")$_b$ ► **LT** | SAK |
| 11 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| 12 | **CUT** ► ***UID CL2 + BCC*** *(no CRC_A)* ► **LT** | UID CL2 + BCC |
| 13 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| 14 | **CUT** ► ***UID CL2 + BCC*** *(no CRC_A)* ► **LT** | UID CL2 + BCC |
| 15 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| 16 | **CUT** ► ***UID CL2 + BCC*** *(no CRC_A)* ► **LT** | UID CL2 + BCC |
| 17 | **CUT** ◄ '**95 70**' **+** ***UID CL2 + BCC*** ◄ **LT** | SELECT CL2 |
| 18 | **CUT** ► ("uu1u u0uu")$_b$ ► **LT** | SAK |
| 19 | **CUT** ◄ '**E0 80**' ◄ **LT** | RATS (FSD = 256) |
| 20 | **CUT** ► ***ATS*** ► **LT** | ATS |
| End of test: Reset PICC | | |

**Scenario 11: Type A Correct Installation with respect of the anticollision state machine (UID size = 2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.9. Type A Correct Installation with respect of the anticollision state machine (UID size = 3) [CA118]

**Test Codification:**

CA118

**ICS Parameters:**

PICC Type = A / UID size = 3

**Test Objective:**

To ensure that the Type A PICC with a triple size UID stays in the READY state and sends its UID CL1 when it receives a valid ANTICOLLISION CL1 Command in the READY state.

To ensure that the Type A PICC with a triple size UID stays in the READY' state and sends its UID CL2 when it receives a valid ANTICOLLISION CL2 Command in the READY' state.

To ensure that the Type A PICC with a triple size UID stays in the READY'' state and sends its UID CL3 when it receives a valid ANTICOLLISION CL3 Command in the READY'' state.

**Reference Requirements:**

7.2.2.1, 7.2.3.1, 7.2.4.1, 7.2.5.1, 7.3, 7.4, 7.2.6.1, 7.5

**Procedure:**

Run the following scenario.

On reception of the ATQA, the LT sends the ANTICOLLISION CL1 Command and waits for the UID CL1 three consecutive times (i.e. three consecutive AC CL1 / UID CL1 exchanges).

On reception of the third UID CL1, the LT sends the SELECT CL1 Command, waits for a SAK from the CUT and then sends the ANTICOLLISION CL2 Command and waits for the UID CL2 three consecutive times (i.e. three consecutive AC CL2 / UID CL2 exchanges).

On reception of the third UID CL2, the LT sends the SELECT CL2 Command, waits for a SAK from the CUT and then sends the ANTICOLLISION CL3 Command and waits for the UID CL3 three consecutive times (i.e. three consecutive AC CL3 / UID CL3 exchanges).

On reception of the third UID CL3, the LT sends the SELECT CL3 Command, waits for a SAK from the CUT and then sends the RATS Command.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT replies to the SELECT CL3 Command with a SAK equal to (“uu1u u0uu”)$_b$ and to the RATS Command with a correct ATS).

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 6 | **CUT** ▸ ***UID CL1 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL1 + BCC |
| 7 | **CUT** ◂ **'93 20'** *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL1 |
| 8 | **CUT** ▸ ***UID CL1 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL1 + BCC |
| 9 | **CUT** ◂ **'93 70' + *UID CL1 + BCC*** ◂ **LT** | SELECT CL1 |
| 10 | **CUT** ▸ ("uu1u u1uu")$_b$ ▸ **LT** | SAK |
| 11 | **CUT** ◂ **'95 20'** *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL2 |
| 12 | **CUT** ▸ ***UID CL2 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL2 + BCC |
| 13 | **CUT** ◂ **'95 20'** *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL2 |
| 14 | **CUT** ▸ ***UID CL2 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL2 + BCC |
| 15 | **CUT** ◂ **'95 20'** *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL2 |
| 16 | **CUT** ▸ ***UID CL2 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL2 + BCC |
| 17 | **CUT** ◂ **'95 70' + *UID CL2 + BCC*** ◂ **LT** | SELECT CL2 |
| 18 | **CUT** ▸ ("uu1u u1uu")$_b$ ▸ **LT** | SAK |
| 19 | **CUT** ◂ **'97 20'** *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL3 |
| 20 | **CUT** ▸ ***UID CL3 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL3 + BCC |
| 21 | **CUT** ◂ **'97 20'** *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL3 |
| 22 | **CUT** ▸ ***UID CL3 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL3 + BCC |
| 23 | **CUT** ◂ **'97 20'** *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL3 |
| 24 | **CUT** ▸ ***UID CL3 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL3 + BCC |
| 25 | **CUT** ◂ **'97 70' + *UID CL3 + BCC*** ◂ **LT** | SELECT CL3 |
| 26 | **CUT** ▸ ("uu1u u0uu")$_b$ ▸ **LT** | SAK |
| 27 | **CUT** ◂ **'E0 80'** ◂ **LT** | RATS (FSD = 256) |
| 28 | **CUT** ▸ ***ATS*** ▸ **LT** | ATS |
| End of test: Reset PICC | | |

**Scenario 12: Type A Correct Installation with respect of the anticollision state machine (UID size = 3)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.10. Type A Handling of RATS (UID size = 1) [CA120.x]

**Test Codification:**

CA120.x

**ICS Parameters:**

PICC Type = A / UID size = 1

**Test Objective:**

To ensure that the Type A PICC handles the RATS Command with various values as required in the specification

**Reference Requirements:**

5.7.1.4

**Procedure:**

Run the following scenario.

Send the RATS Command with the values as indicated below:

- For x=0: CID =0
- For x=1: CID =1
- For x=2: CID = 14

Remark: For x=1 and 2: STOP after step 8 if PICC supports CID as indicated in ICS (i.e $b_2$ of TC(1) byte in ATS is set $(1)_b$).

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT replies to the RATS with its ATS and continues to respond to Select Application command in the following steps).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► ***UID CL1* + *BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ '**93 70**' **+** ***UID CL1* + *BCC*** ◄ **LT** | SELECT CL1 |
| 6 | **CUT** ► ("uu1u u0uu")$_b$ ► **LT** | SAK |
| 7 | **CUT** ◄ **RATS in function of 'x'** ◄ **LT** | RATS |
| 8 | **CUT** ► ***ATS*** ► **LT** | ATS |
| | **For x=1 and 2:** STOP HERE if PICC supports CID as indicated in ICS *($b_2$ of TC(1) byte in ATS is set $(1)_b$)*<br>**For x=0:** *Continue with step 9* | |
| 9 | **CUT** ◄ **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** ◄ **LT** | Select PPSE / b.n. = 0 |
| 10 | **CUT** ► **I(0)$_0$ ['6F'** **+** n data bytes + **'90 00']** ► **LT** | b.n. = 0 |
| End of test: Reset PICC | | |

**Scenario 13: Type A Handling of RATS (UID size x = 1, x=0 to 2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.11. Type A Handling of RATS (UID size = 2) [CA121.x]

**Test Codification:**

CA121.x

**ICS Parameters:**

PICC Type = A / UID size = 2

**Test Objective:**

To ensure that the Type A PICC handles the RATS Command with various values as required in the specification

**Reference Requirements:**

5.7.1.4

**Procedure:**

Run the following scenario.

Send the RATS Command with the values as indicated below:

- For x=0: CID =0
- For x=1: CID =1
- For x=2: CID = 14

Remark: For x=1 and 2: STOP after step 12 if PICC supports CID as indicated in ICS (i.e $b_2$ of TC(1) byte in ATS is set $(1)_b$).

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT replies to the RATS with its ATS and continues to respond to Select Application command in the following steps).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► ***UID CL1* + *BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ '**93 70**' **+** ***UID CL1* + *BCC*** ◄ **LT** | SELECT CL1 |
| 6 | **CUT** ► ("uu1u u1uu")$_b$ ► **LT** | SAK |
| 7 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ► ***UID CL2* + *BCC*** *(no CRC_A)* ► **LT** | UID CL2 + BCC |
| 9 | **CUT** ◄ '**95 70**' **+** ***UID CL2* + *BCC*** ◄ **LT** | SELECT CL2 |
| 10 | **CUT** ► ("uu1u u0uu")$_b$ ► **LT** | SAK |
| 11 | **CUT** ◄ **RATS in function of 'x'** ◄ **LT** | RATS (FSD = 256) |
| 12 | **CUT** ► ***ATS*** ► **LT** | ATS |
| | **For x=1 and 2:** STOP HERE if PICC supports CID as indicated in ICS *($b_2$ of TC(1) byte in ATS is set (1)$_b$)*<br>**For x=0:** Continue with step 13 | |
| 13 | **CUT** ◄ **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** ◄ **LT** | Select PPSE / b.n. = 0 |
| 14 | **CUT** ► **I(0)$_0$ ['6F'** **+** n data bytes + **'90 00']** ► **LT** | b.n. = 0 |
| End of test: Reset PICC | | |

**Scenario 14: Type A Type A Handling of RATS (UID size = 2, x=0 to 2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.12. Type A Handling of RATS (UID size = 3) [CA122.x]

**Test Codification:**

CA122.x

**ICS Parameters:**

PICC Type = A / UID size = 3

**Test Objective:**

To ensure that the Type A PICC handles the RATS Command with various values as required in the specification

**Reference Requirements:**

5.7.1.4

**Procedure:**

Run the following scenario.

Send the RATS Command with the values as indicated below:

- For x=0: CID =0
- For x=1: CID =1
- For x=2: CID = 14

Remark: For x=1 and 2: STOP after step 16 if PICC supports CID as indicated in ICS (i.e $b_2$ of TC(1) byte in ATS is set $(1)_b$).

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT replies to the RATS with its ATS and continues to respond to Select Application command in the following steps).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► ***UID CL1* + *BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ '**93 70**' **+ *UID CL1* + *BCC*** ◄ **LT** | SELECT CL1 |
| 6 | **CUT** ► ("uu1u u1uu")$_b$ ► **LT** | SAK |
| 7 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ► ***UID CL2* + *BCC*** *(no CRC_A)* ► **LT** | UID CL2 + BCC |
| 9 | **CUT** ◄ '**95 70**' **+ *UID CL2* + *BCC*** ◄ **LT** | SELECT CL2 |
| 10 | **CUT** ► ("uu1u u1uu")$_b$ ► **LT** | SAK |
| 11 | **CUT** ◄ '**97 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ► ***UID CL3* + *BCC*** *(no CRC_A)* ► **LT** | UID CL3 + BCC |
| 13 | **CUT** ◄ '**97 70**' **+ *UID CL3* + *BCC*** ◄ **LT** | SELECT CL3 |
| 14 | **CUT** ► ("uu1u u0uu")$_b$ ► **LT** | SAK |
| 15 | **CUT** ◄ **RATS in function of 'x'** ◄ **LT** | RATS (FSD = 256) |
| 16 | **CUT** ► ***ATS*** ► **LT** | ATS |
| | **For x=1 and 2:** STOP HERE if PICC supports CID as indicated in ICS *($b_2$ of TC(1) byte in ATS is set (1)$_b$)*<br>**For x=0:** Continue with step 17 | |
| 17 | **CUT** ◄ **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** ◄ **LT** | Select PPSE / b.n. = 0 |
| 18 | **CUT** ► **I(0)$_0$ ['6F'** **+** n data bytes + **'90 00']** ► **LT** | b.n. = 0 |
| End of test: Reset PICC | | |

**Scenario 15: Type A Type A Handling of RATS (UID size = 3, x=0 to 2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 3.13. Type A Consecutive Installations (UID size = 1, fixed UID) [CA126.x]

**Test Codification:**

CA126.x

**ICS Parameters:**

PICC Type = A / UID size = 1 / fixed UID

**Test Objective:**

To ensure that the UID of the PICC is fixed, when indicated as fixed in the ICS.

**Reference Requirements:**

5.3.2.3

**Procedure:**

Run the following scenario.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

All UID CL1s sent by the CUT have the same value.

The UID sent by the CUT is the one expected as indicated in ICS.

Command frame:

- For x=0: WUPA
- For x=1: REQA

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' **+** ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS (FSD = 256) |
| 8 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |
| 9 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◀ **LT** | Select PPSE / b.n. = 0 |
| 10 | **CUT** ▶ | **I(0)$_0$ ['6F'** **+** n data bytes + **'90 00']** | ▶ **LT** | b.n. = 0 |
| 11 | **CUT** ◀ | **S(DESELECT) Request** | ◀ **LT** | DESELECT Request |
| 12 | **CUT** ▶ | **S(DESELECT) Response** | ▶ **LT** | DESELECT Response |
| 13 | **CUT** ◀ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◀ **LT** | PICC Reset |
| 14 | **CUT** ◀ | **For x=0**: '**52**' *(short frame)*<br>**For x=1**: '**26**' *(short frame)* | ◀ **LT** | WUPA for x=0<br>REQA for x=1 |
| 15 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 16 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 17 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 18 | **CUT** ◀ | '**93 70**' **+** ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 19 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 20 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS (FSD = 256) |
| 21 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |
| 22 | CUT ◀ | I(0)$_0$ ['00 A4 04 00 0E' **+** '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' **+** '00'] | ◀ LT | Select PPSE / b.n. = 0 |
| 23 | CUT ▶ | I(0)$_0$ ['6F' + **n data bytes +** '90 00'] | ▶ LT | b.n. = 0 |
| End of test: Reset PICC | | | | |

**Scenario 16: Type A Consecutive Installations (UID size = 1, fixed UID, x=0 to 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.14. Type A Consecutive Installations (UID size = 2) [CA127.x]

**Test Codification:**

CA127.x

**ICS Parameters:**

PICC Type = A / UID size = 2

**Test Objective:**

To ensure that the UID of the PICC is fixed.

**Reference Requirements:**

5.3.2.3

**Procedure:**

Run the following scenario.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

All UID CL1s sent by the CUT have the same value.

All UID CL2s sent by the CUT have the same value.

The UID sent by the CUT is the one expected as indicated in ICS.

Command frame:

- For x=0: WUPA
- For x=1: REQA

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◂ '**52**' *(short frame)* ◂ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▸ ***ATQA*** *(no CRC_A)* ▸ **LT** | ATQA |
| 3 | **CUT** ◂ '**93 20**' *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▸ ***UID CL1* + *BCC*** *(no CRC_A)* ▸ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◂ '**93 70**' **+** ***UID CL1* + *BCC*** ◂ **LT** | SELECT CL1 |
| 6 | **CUT** ▸ ("uu1u u1uu")$_b$ ▸ **LT** | SAK |
| 7 | **CUT** ◂ '**95 20**' *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▸ ***UID CL2* + *BCC*** *(no CRC_A)* ▸ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◂ '**95 70**' **+** ***UID CL2* + *BCC*** ◂ **LT** | SELECT CL2 |
| 10 | **CUT** ▸ ("uu1u u0uu")$_b$ ▸ **LT** | SAK |
| 11 | **CUT** ◂ '**E0 80**' ◂ **LT** | RATS (FSD = 256) |
| 12 | **CUT** ▸ ***ATS*** ▸ **LT** | ATS |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 13 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◀ **LT** | Select PPSE / b.n. = 0 |
| 14 | **CUT** ▶ | **I(0)$_0$ ['6F'** + n data bytes + **'90 00']** | ▶ **LT** | b.n. = 0 |
| 15 | **CUT** ◀ | **S(DESELECT) Request** | ◀ **LT** | DESELECT Request |
| 16 | **CUT** ▶ | **S(DESELECT) Response** | ▶ **LT** | DESELECT Response |
| 17 | **CUT** ◀ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◀ **LT** | PICC Reset |
| 18 | **CUT** ◀ | **For x=0**: '**52**' *(short frame)*<br>**For x=1**: '**26**' *(short frame)* | ◀ **LT** | WUPA for x=0<br>REQA for x=1 |
| 19 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 20 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 21 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 22 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 23 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 24 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 25 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 26 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 27 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 28 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS (FSD = 256) |
| 29 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |
| 30 | CUT ◀ | I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00'] | ◀ LT | Select PPSE / b.n. = 0 |
| 31 | CUT ▶ | I(0)$_0$ ['6F' + **n data bytes** + '90 00'] | ▶ LT | b.n. = 0 |
| End of test: Reset PICC | | | | |

**Scenario 17: Type A Consecutive Installations (UID size = 2, x=0 to 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.15. Type A Consecutive Installations (UID size = 3) [CA128.x]

**Test Codification:**

CA128.x

**ICS Parameters:**

PICC Type = A / UID size = 3

**Test Objective:**

To ensure that the UID of the PICC is fixed.

**Reference Requirements:**

5.3.2.3

**Procedure:**

Run the following scenario.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

All UID CL1s sent by the CUT have the same value.

All UID CL2s sent by the CUT have the same value.

All UID CL3s sent by the CUT have the same value.

The UID sent by the CUT is the one expected as indicated in ICS.

Command frame:

- For x=0: WUPA
- For x=1: REQA

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ '**93 70**' **+ *UID CL1 + BCC*** ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ ("uu1u u1uu")$_b$ ▶ **LT** | SAK |
| 7 | **CUT** ◀ '**95 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ ***UID CL2 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ '**95 70**' **+ *UID CL2 + BCC*** ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ ("uu1u u1uu")$_b$ ▶ **LT** | SAK |
| 11 | **CUT** ◀ '**97 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL3 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 12 | **CUT** ▸ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◂ | **'97 70' + *UID CL3* + *BCC*** | ◂ **LT** | SELECT CL3 |
| 14 | **CUT** ▸ | ("uu1u u0uu")$_b$ | ▸ **LT** | SAK |
| 15 | **CUT** ◂ | **'E0 80'** | ◂ **LT** | RATS (FSD = 256) |
| 16 | **CUT** ▸ | ***ATS*** | ▸ **LT** | ATS |
| 17 | **CUT** ◂ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◂ **LT** | Select PPSE / b.n. = 0 |
| 18 | **CUT** ▸ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▸ **LT** | b.n. = 0 |
| 19 | **CUT** ◂ | **S(DESELECT) Request** | ◂ **LT** | DESELECT Request |
| 20 | **CUT** ▸ | **S(DESELECT) Response** | ▸ **LT** | DESELECT Response |
| 21 | **CUT** ◂ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◂ **LT** | PICC Reset |
| 22 | **CUT** ◂ | **For x=0**: '**52**' *(short frame)*<br>**For x=1**: '**26**' *(short frame)* | ◂ **LT** | WUPA for x=0<br>REQA for x=1 |
| 23 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 24 | **CUT** ◂ | **'93 20'** *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL1 |
| 25 | **CUT** ▸ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL1 + BCC |
| 26 | **CUT** ◂ | **'93 70' + *UID CL1* + *BCC*** | ◂ **LT** | SELECT CL1 |
| 27 | **CUT** ▸ | ("uu1u u1uu")$_b$ | ▸ **LT** | SAK |
| 28 | **CUT** ◂ | **'95 20'** *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL2 |
| 29 | **CUT** ▸ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL2 + BCC |
| 30 | **CUT** ◂ | **'95 70' + *UID CL2* + *BCC*** | ◂ **LT** | SELECT CL2 |
| 31 | **CUT** ▸ | ("uu1u u1uu")$_b$ | ▸ **LT** | SAK |
| 32 | **CUT** ◂ | **'97 20'** *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL3 |
| 33 | **CUT** ▸ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL3 + BCC |
| 34 | **CUT** ◂ | **'97 70' + *UID CL3* + *BCC*** | ◂ **LT** | SELECT CL3 |
| 35 | **CUT** ▸ | ("uu1u u0uu")$_b$ | ▸ **LT** | SAK |
| 36 | **CUT** ◂ | **'E0 80'** | ◂ **LT** | RATS (FSD = 256) |
| 37 | **CUT** ▸ | ***ATS*** | ▸ **LT** | ATS |
| 38 | CUT ◂ | I(0)$_0$ ['00 A4 04 00 0E' **+** '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' **+** '00'] | ◂ LT | Select PPSE / b.n. = 0 |
| 39 | CUT ▸ | I(0)$_0$ ['6F' + **n data bytes +** '90 00'] | ▸ LT | b.n. = 0 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| End of test: Reset PICC | | |

## Scenario 18: Type A Consecutive Installations (UID size = 3, x=0 to 1)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.16. Type A Handling of the REQA Command (UID size = 1) [CA130.x]

**Test Codification:**

CA130.x

**ICS Parameters:**

PICC Type = A / UID size = 1

**Test Objective:**

To ensure that the Type A PICC does not send an ATQA in response to a REQA Command when it is not in IDLE State.

**Reference Requirements:**

7.2.3.4, 7.2.6.4, 7.2.7.1

**Procedure:**

Run the following scenario.

The LT sends a REQA Command to the PICC:

- x=0: after an ATQA Response (CUT in the READY State)
- x=1: after an ANTICOLLISION CL1 Response (CUT in READY State after an ANTICOLLISION Command)
- x=2: after a SAK Response (CUT in ACTIVE State)
- x=3: after an ATS Response (CUT in PROTOCOL State)
- x=4: after an ATQA Response sent by the PICC after a HLTA Command (CUT in READY* State)
- x=5: after a SAK Response sent by the PICC after a HLTA Command (CUT in ACTIVE* State)

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of the first REQA Command, the CUT stays mute (i.e. does not send any frame during Mute_Time).

For x=0, 1 and 2: the CUT sends an ATQA in response to the REQA Command following the first REQA Command

For x=3: the CUT sends an I-Block in response to the I-Block following the first REQA Command

For x=4 and 5: the CUT sends an ATQA in response to WUPA Command following the first REQA Command

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 4 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| 5 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 19: Type A Handling of the REQA Command (UID size = 1, x=0)**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 6 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| 7 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 20: Type A Handling of the REQA Command (UID size = 1, x=1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ '**93 70**' **+ *UID CL1 + BCC*** ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ ("uu1u u0uu")$_b$ ▶ **LT** | SAK |
| 7 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 8 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| 9 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 21: Type A Handling of the REQA Command (UID size = 1, x=2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' **+** ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | **RATS** | ◀ **LT** | RATS (FSD = 256) |
| 8 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |
| 9 | **CUT** ◀ | '**26**' *(short frame)* | ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 10 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'**] | ◀ **LT** | Select PPSE / b.n. = 0 |
| 11 | **CUT** ▶ | **I(0)$_0$ ['6F'** + n data bytes + '**90 00'**] | ▶ **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

**Scenario 22: Type A Handling of the REQA Command (UID size = 1, x=3)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ '**93 70**' **+ *UID CL1 + BCC*** ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ ("uu1u u0uu")$_b$ ▶ **LT** | SAK |
| 7 | **CUT** ◀ '**50 00**' ◀ **LT** | HLTA |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 8 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 9 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 10 | **CUT** ◀ **'26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 11 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 12 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 23: Type A Handling of the REQA Command (UID size = 1, x=4)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ '**93 70**' **+ *UID CL1 + BCC*** ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ ("uu1u u0uu")$_b$ ▶ **LT** | SAK |
| 7 | **CUT** ◀ '**50 00**' ◀ **LT** | HLTA |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 8 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 9 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 10 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 11 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 12 | **CUT** ◀ '**93 70**' **+ *UID CL1 + BCC*** ◀ **LT** | SELECT CL1 |
| 13 | **CUT** ▶ ("uu1u u0uu")$_b$ ▶ **LT** | SAK |
| 14 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 15 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 16 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 24: Type A Handling of the REQA Command (UID size = 1, x=5)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.17. Type A Handling of the REQA Command (UID size = 2) [CA131.x]

**Test Codification:**

CA131.x

**ICS Parameters:**

PICC Type = A / UID size = 2

**Test Objective:**

To ensure that the Type A PICC does not send an ATQA in response to a REQA Command when it is not in IDLE State.

**Reference Requirements:**

7.2.3.4, 7.2.4.4, 7.2.6.4, 7.2.7.1

**Procedure:**

Run the following scenario.

The LT sends a REQA Command to the PICC:

- x=0: after an ATQA Response (CUT in the READY State)
- x=1: after an ANTICOLLISION CL1 Response (CUT in READY State after an ANTICOLLISION Command)
- x=2: after a SAK Response (CUT in ACTIVE State
- x=3: after an ATS Response (CUT in PROTOCOL State
- x=4: after an ATQA Response sent by the PICC after a HLTA Command (CUT in READY* State)
- x=5: after a SAK Response sent by the PICC after a HLTA Command (CUT in ACTIVE* State)

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of the first REQA Command, the CUT stays mute (i.e. does not send any frame during Mute_Time).

For x=0, 1 and 2: the CUT sends an ATQA in response to the REQA Command following the first REQA Command

For x=3: the CUT sends an I-Block in response to the I-Block following the first REQA Command

For x=4 and 5: the CUT sends an ATQA in response to WUPA Command following the first REQA Command

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**26**' *(short frame)* ◄ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ► **LT** | |
| 4 | **CUT** ◄ '**26**' *(short frame)* ◄ **LT** | REQA to poll for the CUT |
| 5 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 25: Type A Handling of the REQA Command (UID size = 2, x=0)**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ '**93 70**' **+ *UID CL1 + BCC*** ◄ **LT** | SELECT CL1 |
| 6 | **CUT** ► ("uu1u u1uu")$_b$ ► **LT** | SAK |
| 7 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ► ***UID CL2 + BCC*** *(no CRC_A)* ► **LT** | UID CL2 + BCC |
| 9 | **CUT** ◄ '**26**' *(short frame)* ◄ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ► **LT** | |
| 10 | **CUT** ◄ '**26**' *(short frame)* ◄ **LT** | REQA to poll for the CUT |
| 11 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 26: Type A Handling of the REQA Command (UID size = 2, x=1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1* + *BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ '**93 70**' **+** ***UID CL1* + *BCC*** ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ ("uu1u u1uu")$_b$ ▶ **LT** | SAK |
| 7 | **CUT** ◀ '**95 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ ***UID CL2* + *BCC*** *(no CRC_A)* ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ '**95 70**' **+** ***UID CL2* + *BCC*** ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ ("uu1u u0uu")$_b$ ▶ **LT** | SAK |
| 11 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 12 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| 13 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 27: Type A Handling of the REQA Command (UID size = 2, x=2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' **+** ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ | '**95 70**' **+** ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | **RATS** | ◀ **LT** | RATS (FSD = 256) |
| 12 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |
| 13 | **CUT** ◀ | '**26**' *(short frame)* | ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 14 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◀ **LT** | Select PPSE / b.n. = 0 |
| 15 | **CUT** ▶ | **I(0)$_0$ ['6F'** + n data bytes + **'90 00']** | ▶ **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

**Scenario 28: Type A Handling of the REQA Command (UID size = 2, x=3)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ '**93 70**' **+ *UID CL1 + BCC*** ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ ("uu1u u1uu")$_b$ ▶ **LT** | SAK |
| 7 | **CUT** ◀ '**95 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ ***UID CL2 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ '**95 70**' **+ *UID CL2 + BCC*** ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ ("uu1u u0uu")$_b$ ▶ **LT** | SAK |
| 11 | **CUT** ◀ '**50 00**' ◀ **LT** | HLTA |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 12 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 13 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 14 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 15 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 16 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 29: Type A Handling of the REQA Command (UID size = 2, x=4)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | '**50 00**' | ◀ **LT** | HLTA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 12 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 13 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 14 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 15 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 16 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 17 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 18 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 19 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 20 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 21 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 22 | **CUT** ◀ | '**26**' *(short frame)* | ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 23 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 24 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| End of test: Reset PICC | | | | |

**Scenario 30: Type A Handling of the REQA Command (UID size = 2, x=5)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.18. Type A Handling of the REQA Command (UID size = 3) [CA132.x]

**Test Codification:**

CA132.x

**ICS Parameters:**

PICC Type = A / UID size = 3

**Test Objective:**

To ensure that the Type A PICC does not send an ATQA in response to a REQA Command when it is not in IDLE State.

**Reference Requirements:**

7.2.3.4, 7.2.5.3, 7.2.6.4, 7.2.7.1

**Procedure:**

Run the following scenario.

The LT sends a REQA Command to the PICC:

- x=0: after an ATQA Response (CUT in the READY State)
- x=1: after an ANTICOLLISION CL1 Response (CUT in READY State after an ANTICOLLISION Command)
- x=2: after a SAK Response (CUT in ACTIVE Stat,
- x=3: after an ATS Response (CUT in PROTOCOL State
- x=4: after an ATQA Response sent by the PICC after a HLTA Command (CUT in READY* State)
- x=5: after a SAK Response sent by the PICC after a HLTA Command (CUT in ACTIVE* State)

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of the first REQA Command, the CUT stays mute (i.e. does not send any frame during Mute_Time).

For x=0: 1 and 2: the CUT sends an ATQA in response to the REQA Command following the first REQA Command

For x=3: the CUT sends an I-Block in response to the I-Block following the first REQA Command

For x=4 and 5: the CUT sends an ATQA in response to WUPA Command following the first REQA Command

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**26**' *(short frame)* ◄ **LT** | REQA to poll for the CUT |
| | **CUT** II The CUT does not send any frame during Mute_Time ► **LT** | |
| 4 | **CUT** ◄ '**26**' *(short frame)* ◄ **LT** | REQA to poll for the CUT |
| 5 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| End of test: Reset PICC | | |

**Scenario 31: Type A Handling of the REQA Command (UID size = 3, x=0)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ '**93 70**' **+ *UID CL1 + BCC*** ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ ("uu1u u1uu")$_b$ ▶ **LT** | SAK |
| 7 | **CUT** ◀ '**95 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ ***UID CL2 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ '**95 70**' **+ *UID CL2 + BCC*** ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ ("uu1u u1uu")$_b$ ▶ **LT** | SAK |
| 11 | **CUT** ◀ '**97 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ▶ ***UID CL3 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▶ **LT** | |
| 14 | **CUT** ◀ '**26**' *(short frame)* ◀ **LT** | REQA to poll for the CUT |
| 15 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| End of test: Reset PICC | | |

### Scenario 32: Type A Handling of the REQA Command (UID size = 3, x=1)

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ '**52**' *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ '**93 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ '**93 70**' **+ *UID CL1 + BCC*** ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ ("uu1u u1uu")$_b$ ▶ **LT** | SAK |
| 7 | **CUT** ◀ '**95 20**' *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ ***UID CL2 + BCC*** *(no CRC_A)* ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ '**95 70**' **+ *UID CL2 + BCC*** ◀ **LT** | SELECT CL2 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 10 | **CUT** ▸ | (“uu1u u1uu”)$_b$ | ▸ **LT** | SAK |
| 11 | **CUT** ◂ | ’**97 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ▸ | ***UID CL3 + BCC*** *(no CRC_A)* | ▸ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◂ | ’**97 70**’ **+ *UID CL3 + BCC*** | ◂ **LT** | SELECT CL3 |
| 14 | **CUT** ▸ | (“uu1u u0uu”)$_b$ | ▸ **LT** | SAK |
| 15 | **CUT** ◂ | **‘26**’ *(short frame)* | ◂ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 16 | **CUT** ◂ | **‘26**’ *(short frame)* | ◂ **LT** | REQA to poll for the CUT |
| 17 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| End of test: Reset PICC | | | | |

**Scenario 33: Type A Handling of the REQA Command (UID size = 3, x=2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | '**97 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ▶ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◀ | '**97 70**' + ***UID CL3* + *BCC*** | ◀ **LT** | SELECT CL3 |
| 14 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 15 | **CUT** ◀ | **RATS** | ◀ **LT** | RATS (FSD = 256) |
| 16 | **CUT** ▶ | ***ATS*** | ▶ **LT** | ATS |
| 17 | **CUT** ◀ | '**26**' *(short frame)* | ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 18 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'**] | ◀ **LT** | Select PPSE / b.n. = 0 |
| 19 | **CUT** ▶ | **I(0)$_0$ ['6F'** + n data bytes + '**90 00'**] | ▶ **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

## Scenario 34: Type A Handling of the REQA Command (UID size = 3, x=3)

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| 6 | **CUT** ▸ (“uu1u u1uu”)b ▸ **LT** | SAK |
| 7 | **CUT** ◂ ’**95 20**’ *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▸ ***UID CL2 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◂ ’**95 70**’ **+ *UID CL2 + BCC*** ◂ **LT** | SELECT CL2 |
| 10 | **CUT** ▸ (“uu1u u1uu”)b ▸ **LT** | SAK |
| 11 | **CUT** ◂ ’**97 20**’ *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ▸ ***UID CL3 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◂ ’**97 70**’ **+ *UID CL3 + BCC*** ◂ **LT** | SELECT CL3 |
| 14 | **CUT** ▸ (“uu1u u0uu”)b ▸ **LT** | SAK |
| 15 | **CUT** ◂ ’**50 00**’ ◂ **LT** | HLTA |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▸ **LT** | |
| 16 | **CUT** ◂ ‘**52**’ *(short frame)* ◂ **LT** | WUPA to poll for the CUT |
| 17 | **CUT** ▸ ***ATQA*** *(no CRC_A)* ▸ **LT** | ATQA |
| 18 | **CUT** ◂ **‘26**’ *(short frame)* ◂ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ▸ **LT** | |
| 19 | **CUT** ◂ ‘**52**’ *(short frame)* ◂ **LT** | WUPA to poll for the CUT |
| 20 | **CUT** ▸ ***ATQA*** *(no CRC_A)* ▸ **LT** | ATQA |
| End of test: Reset PICC | | |

### Scenario 35: Type A Handling of the REQA Command (UID size = 3, x=4)

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◂ ‘**52**’ *(short frame)* ◂ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▸ ***ATQA*** *(no CRC_A)* ▸ **LT** | ATQA |
| 3 | **CUT** ◂ ’**93 20**’ *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▸ ***UID CL1 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◂ ’**93 70**’ **+ *UID CL1 + BCC*** ◂ **LT** | SELECT CL1 |
| 6 | **CUT** ▸ (“uu1u u1uu”)b ▸ **LT** | SAK |
| 7 | **CUT** ◂ ’**95 20**’ *(no CRC_A)* ◂ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▸ ***UID CL2 + BCC*** *(no CRC_A)* ▸ **LT** | UID CL2 + BCC |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 9 | **CUT** ◀ | '**95 70**' + ***UID CL2*** + ***BCC*** | ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | '**97 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ▶ | ***UID CL3*** + ***BCC*** *(no CRC_A)* | ▶ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◀ | '**97 70**' + ***UID CL3*** + ***BCC*** | ◀ **LT** | SELECT CL3 |
| 14 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 15 | **CUT** ◀ | '**50 00**' | ◀ **LT** | HLTA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 16 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 17 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 18 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 19 | **CUT** ▶ | ***UID CL1*** + ***BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 20 | **CUT** ◀ | '**93 70**' + ***UID CL1*** + ***BCC*** | ◀ **LT** | SELECT CL1 |
| 21 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 22 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 23 | **CUT** ▶ | ***UID CL2*** + ***BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 24 | **CUT** ◀ | '**95 70**' + ***UID CL2*** + ***BCC*** | ◀ **LT** | SELECT CL2 |
| 25 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 26 | **CUT** ◀ | '**97 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL3 |
| 27 | **CUT** ▶ | ***UID CL3*** + ***BCC*** *(no CRC_A)* | ▶ **LT** | UID CL3 + BCC |
| 28 | **CUT** ◀ | '**97 70**' + ***UID CL3*** + ***BCC*** | ◀ **LT** | SELECT CL3 |
| 29 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 30 | **CUT** ◀ | '**26**' *(short frame)* | ◀ **LT** | REQA to poll for the CUT |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 31 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 32 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| End of test: Reset PICC | | | | |

**Scenario 36: Type A Handling of the REQA Command (UID size = 3, x=5)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.19. Type A Installation with error in the IDLE state [CA204.x]

**Test Codification:**

CA204.x

**ICS Parameters:**

PICC Type = A

**Test Objective:**

To ensure that the Type A PICC behaves correctly when it receives an erroneous frame in the IDLE state (i.e. during the polling) and replies to the WUPA and REQA Commands.

**Reference Requirements:**

5.1.1.2, 7.1.1.2, 7.1.1.3, 7.2.2.3

**Procedure:**

Run the following scenario.

Instead of the polling commands, the LT sends consecutive erroneous frames.

After each erroneous frame, the LT stays in reception mode during Mute_Time.

After having sent all the erroneous frames, the LT sends a correct WUPA and REQA Command frame:

- For x=0: WUPA
- For x=1: REQA

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command frame (x=0) or REQA Command frame (x=1), the CUT replies with a correct ATQA.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| | **CUT** II The CUT does not send any frame during Mute_Time ► **LT** | |
| 2 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| | **CUT** II The CUT does not send any frame during Mute_Time ► **LT** | |
| 3 | **CUT** ◄ '**97 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL3 |
| | **CUT** II The CUT does not send any frame during Mute_Time ► **LT** | |
| 4 | **CUT** ◄ '**50 00**' ◄ **LT** | HLTA |
| | **CUT** II The CUT does not send any frame during Mute_Time ► **LT** | |
| 5 | **CUT** ◄ '**E0 80**' ◄ **LT** | RATS (FSD = 256) |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ | **LT** | |
| 6 | **CUT** ◂ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◂ | **LT** | Select PPSE / b.n. = 0 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ | **LT** | |
| 7 | **CUT** ◂ | **R(NAK)$_0$** | ◂ | **LT** | Error Notification |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ | **LT** | |
| 8 | **CUT** ◂ | **For x=0: '52'** *(short frame)*<br>**For x=1: '26'** *(short frame)* | ◂ | **LT** | WUPA/REQA to poll for the CUT |
| 9 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ | **LT** | ATQA |
| 10 | **CUT** ◂ | **'93 20'** *(no CRC_A)* | ◂ | **LT** | ANTICOLLISION CL1 |
| 11 | **CUT** ▸ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▸ | **LT** | UID CL1 + BCC |
| 12 | **CUT** ◂ | **'93 70'** + ***UID CL1* + *BCC*** | ◂ | **LT** | SELECT CL1 |
| 13 | **CUT** ▸ | ("uu1u uuuu")$_b$ | ▸ | **LT** | SAK |
| 14 | **CUT** ◂ | **For x=0: '52'** *(short frame)*<br>**For x=1: '26'** *(short frame)* | ◂ | **LT** | WUPA/REQA to poll for the CUT |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ | **LT** | |
| 15 | **CUT** ◂ | **'93 70'** + ***UID CL1* + *BCC*** | ◂ | **LT** | SELECT CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ | **LT** | |
| 16 | **CUT** ◂ | For UID size = 1: **'95 70'** + ***UID CL1* + *BCC***<br>For UID size ≠ 1: **'95 70'** + ***UID CL2* + *BCC*** | ◂ | **LT** | SELECT CL2 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ | **LT** | |
| 17 | **CUT** ◂ | For UID size ≠ 3: **'97 70'** + ***UID CL1* + *BCC***<br>For UID size = 3: **'97 70'** + ***UID CL3* + *BCC*** | ◂ | **LT** | SELECT CL3 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ | **LT** | |
| 18 | **CUT** ◂ | **For x=0: '52'** *(short frame)*<br>**For x=1: '26'** *(short frame)* | ◂ | **LT** | WUPA/REQA to poll for the CUT |
| 19 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ | **LT** | ATQA |
| End of test: Reset PICC | | | | | |

**Scenario 37: Type A Installation with error in the IDLE state (x=0 and 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.20. Type A Installation with error in the READY state [CA206.xy]

**Test Codification:**

CA206.xy

**ICS Parameters:**

PICC Type = A

**Test Objective:**

To ensure that the Type A PICC behaves correctly when it receives an erroneous frame in the READY state (i.e. after the ATQA).

**Reference Requirements:**

5.1.1.2, 5.3.2.4, 7.1.1.2, 7.1.1.3, 7.2.3.4

**Procedure:**

Run the following scenario.

On reception of UID CL1, the LT sends an erroneous frame.

After having sent the erroneous frame, the LT stays in reception mode during Mute_Time before sending a correct ANTICOLLISION CL1 Command frame.

After having sent the ANTICOLLISION CL1 Command frame, the LT stays in reception mode during Mute_Time before sending a correct WUPA Command frame.

The following errors are generated by the LT:

- xy=00: SELECT CL1 Command frame with a wrong least significant bit in the first byte (CRC_A1) of CRC_A
- xy=01: SELECT CL1 Command frame with a wrong least significant bit in the second byte (CRC_A2) of CRC_A
- xy=02: SELECT CL1 Command frame with a wrong most significant bit in the second byte (CRC_A2) of CRC_A
- xy=03: SELECT CL1 Command frame with Parity error on the first byte ('93')
- xy=04: SELECT CL1 Command frame with Parity error on the first byte of the UID CL1
- xy=05: SELECT CL1 Command frame with Parity error on the third byte of the UID CL1
- xy=06: SELECT CL1 Command frame with Parity error on the first byte of the CRC bytes
- xy=07: SELECT CL1 Command frame with Parity error on the second byte of the CRC bytes
- xy=08: HLTA Command frame
- xy=09: WUPA Command frame
- xy=10: RATS Command frame
- xy=11: ANTICOLLISION CL2 Command frame
- xy=12: SELECT CL2 Command frame (for UID size = 1, UID CL2 = UID CL1)
- xy=13: I-Block frame
- xy=14: SELECT CL1 Command frame with wrong value of BCC
- xy=15: ANTICOLLISION CL1 Command frame with Parity error on the first byte ('93')
- xy=16: ANTICOLLISION CL1 Command frame with Parity error on the second byte ('20')

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the correct ANTICOLLISION CL1 Command frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command frame, the CUT replies with a correct ATQA.

All UID CL1s sent by CUT have the same value.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | **Erroneous frame in function of 'xy'**:<br>**For xy=00 to 07**: '**93 70**' + ***UID CL1* + *BCC* with a transmission error**<br>**For xy=08**: '**50 00**'<br>**For xy=09**: '**52**' *(short frame)*<br>**For xy=10**: '**E0 80**'<br>**For xy=11**: '**95 20**' *(no CRC_A)*<br>**For xy=12**: '**95 70**' + ***UID CL2* + *BCC*** *(for UID size = 1, UID CL2 = UID CL1)*<br>**For xy=13**: **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'**] *(SELECT PPSE)*<br>**For xy=14**: **'93 70'** + ***UID CL1*** + **wrong BCC**<br>**For xy=15 to 16**: '**93 20**' *(no CRC_A)* **with a parity error** | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 6 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 7 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA |
| 8 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 9 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 10 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| End of test: Reset PICC | | | | |

**Scenario 38: Type A Installation with error in the READY state (xy=00 to 16)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.21. Type A Installation with error in the READY' state (UID size = 2 or 3) [CA207.xy]

**Test Codification:**

CA207.xy

**ICS Parameters:**

PICC Type = A/ UID size = 2 or 3

**Test Objective:**

To ensure that the Type A PICC behaves correctly when it receives an erroneous frame in the READY' state (i.e. in response to the first SAK for UID size = 2 or 3).

**Reference Requirements:**

5.1.1.2, 7.1.1.2, 7.1.1.3, 7.2.4.4

**Procedure:**

Run the following scenario.

On reception of UID CL2 sent by the CUT, the LT sends an erroneous frame.

After having sent the erroneous frame, the LT stays in reception mode during Mute_Time before sending a correct ANTICOLLISION CL2 Command frame.

After having sent the ANTICOLLISION CL2 Command frame, the LT stays in reception mode during Mute_Time before sending a correct WUPA Command frame.

The following errors are generated by the LT:

- xy=00: SELECT CL2 Command frame with a wrong least significant bit in the first byte (CRC_A1) of CRC_A
- xy=01: SELECT CL2 Command frame with a wrong least significant bit in the second byte (CRC_A2) of CRC_A
- xy=02: SELECT CL2 Command frame with a wrong most significant bit in the second byte (CRC_A2) of CRC_A
- xy=03: SELECT CL2 Command frame with Parity error on the first byte ('95')
- xy=04: SELECT CL2 Command frame with Parity error on the first byte of the UID CL2
- xy=05: SELECT CL2 Command frame with Parity error on the third byte of the UID CL2
- xy=06: SELECT CL2 Command frame with Parity error on the first byte of the CRC bytes
- xy=07: SELECT CL2 Command frame with Parity error on the second byte of the CRC bytes
- xy=08: HLTA Command frame
- xy=09: WUPA Command frame
- xy=10: RATS Command frame
- xy=11: ANTICOLLISION CL3 Command frame
- xy=12: SELECT CL3 Command frame (for UID size = 2, UID CL3 = UID CL2)
- xy=13: I-Block frame
- xy=14: SELECT CL2 Command frame with wrong value of BCC
- xy=15: ANTICOLLISION CL2 Command frame with Parity error on the first byte ('95')
- xy=16: ANTICOLLISION CL2 Command frame with Parity error on the second byte ('20')

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the correct ANTICOLLISION CL2 Command frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command, the CUT replies with a correct ATQA.

All UID CL1s sent by CUT have the same value.

All UID CL2s sent by CUT have the same value.

**Scenario:**

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' **+** ***UID CL1* + *BCC*** | ◀ **LT** | | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | | SAK |
| 7 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | | UID CL2 + BCC |
| 9 | **CUT** ◀ | **Erroneous frame in function of 'xy'**:<br>**For xy=00 to 07**: '**95 70**' **+** ***UID CL2* + *BCC*** **with a transmission error**<br>**For xy=08**: '**50 00**'<br>**For xy=09**: '**52**' *(short frame)*<br>**For xy=10**: '**E0 80**'<br>**For xy=11**: '**97 20**' *(no CRC_A)*<br>**For xy=12**: '**97 70**' **+** ***UID CL3* + *BCC*** *(for UID size = 2, UID CL3 = UID CL2)*<br>**For xy=13**: **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'] *(SELECT PPSE)***<br>**For xy=14**: **'95 70'** + ***UID CL2*** + **wrong BCC**<br>**For xy=15 to 16**: '**95 20**' *(no CRC_A)* **with a parity error** | ◀ **LT** | | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 10 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | | ANTICOLLISION CL2 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 11 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | | WUPA |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 12 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 13 | **CUT** ◂ | ’**93 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL1 |
| 14 | **CUT** ▸ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL1 + BCC |
| 15 | **CUT** ◂ | ’**93 70**’ **+ *UID CL1* + *BCC*** | ◂ **LT** | SELECT CL1 |
| 16 | **CUT** ▸ | (“uu1u u1uu”)$_b$ | ▸ **LT** | SAK |
| 17 | **CUT** ◂ | ’**95 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL2 |
| 18 | **CUT** ▸ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL2 + BCC |
| End of test: Reset PICC | | | | |

**Scenario 39: Type A Installation with error in the READY’ state (UID size = 2 or 3 / xy=00 to 16)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.22. Type A Installation with error in the READY'' state (UID size = 3) [CA208.xy]

**Test Codification:**

CA208.xy

**ICS Parameters:**

PICC Type = A/ UID size = 3

**Test Objective:**

To ensure that the Type A PICC behaves correctly when it receives an erroneous frame in the READY'' state (i.e. in response to the 2nd SAK for UID size = 3).

**Reference Requirements:**

5.1.1.2, 7.1.1.2, 7.1.1.3, 7.2.5.3

**Procedure:**

Run the following scenario.

On reception of UID CL3, the LT sends an erroneous frame.

After having sent the erroneous frame, the LT stays in reception mode during Mute_Time before sending a correct ANTICOLLISION CL3 Command frame.

After having sent the ANTICOLLISION CL3 Command frame, the LT stays in reception mode during Mute_Time before sending a correct WUPA Command frame.

The following errors are generated by the LT:

- xy=00: SELECT CL3 Command frame with a wrong least significant bit in the first byte (CRC_A1) of CRC_A
- xy=01: SELECT CL3 Command frame with a wrong least significant bit in the second byte (CRC_A2) of CRC_A
- xy=02: SELECT CL3 Command frame with a wrong most significant bit in the second byte (CRC_A2) of CRC_A
- xy=03: SELECT CL3 Command frame with Parity error on the first byte ('97')
- xy=04: SELECT CL3 Command frame with Parity error on the first byte of the UID CL3
- xy=05: SELECT CL3 Command frame with Parity error on the third byte of the UID CL3
- xy=06: SELECT CL3 Command frame with Parity error on the first byte of the CRC bytes
- xy=07: SELECT CL3 Command frame with Parity error on the second byte of the CRC bytes
- xy=08: HLTA Command frame
- xy=09: WUPA Command frame
- xy=10: RATS Command frame
- xy=11: ANTICOLLISION CL1 Command frame
- xy=12: SELECT CL1 Command frame
- xy=13: I-Block frame
- xy=14: SELECT CL3 Command frame with wrong value of BCC
- xy=15: ANTICOLLISION CL3 Command frame with Parity error on the first byte ('97')
- xy=16: ANTICOLLISION CL3 Command frame with Parity error on the second byte ('20')

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the correct ANTICOLLISION CL3 Command frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command, the CUT replies with a correct ATQA.

All UID CL1s sent by CUT have the same value.

All UID CL2s sent by CUT have the same value.

All UID CL3s sent by CUT have the same value.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ '**93 70**' **+ *UID CL1 + BCC*** ◄ **LT** | SELECT CL1 |
| 6 | **CUT** ► ("uu1u u1uu")$_b$ ► **LT** | SAK |
| 7 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ► ***UID CL2 + BCC*** *(no CRC_A)* ► **LT** | UID CL2 + BCC |
| 9 | **CUT** ◄ '**95 70**' **+ *UID CL2 + BCC*** ◄ **LT** | SELECT CL2 |
| 10 | **CUT** ► ("uu1u u1uu")$_b$ ► **LT** | SAK |
| 11 | **CUT** ◄ '**97 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ► ***UID CL3 + BCC*** *(no CRC_A)* ► **LT** | UID CL3 + BCC |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 13 | CUT ◀ | **Erroneous frame in function of 'xy'**:<br>**For xy=00 to 07**: '**97 70**' + ***UID CL3 + BCC*** **with a transmission error**<br>**For xy=08**: '**50 00**'<br>**For xy=09**: '**52**' *(short frame)*<br>**For xy=10**: '**E0 80**'<br>**For xy=11**: '**93 20**' *(no CRC_A)*<br>**For xy=12**: '**93 70**' + ***UID CL1 + BCC***<br>**For xy=13**: **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'] *(SELECT PPSE)***<br>**For xy=14**: **'97 70'** + ***UID CL3*** + **wrong BCC**<br>**For xy=15 to 16**: '**97 20**' *(no CRC_A)* **with a parity error** | ◀ LT | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▶ LT | |
| 14 | CUT ◀ | '**97 20**' *(no CRC_A)* | ◀ LT | ANTICOLLISION CL3 |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▶ LT | |
| 15 | CUT ◀ | '**52**' *(short frame)* | ◀ LT | WUPA |
| 16 | CUT ▶ | ***ATQA*** *(no CRC_A)* | ▶ LT | ATQA |
| 17 | CUT ◀ | '**93 20**' *(no CRC_A)* | ◀ LT | ANTICOLLISION CL1 |
| 18 | CUT ▶ | ***UID CL1 + BCC*** *(no CRC_A)* | ▶ LT | UID CL1 + BCC |
| 19 | CUT ◀ | '**93 70**' + ***UID CL1 + BCC*** | ◀ LT | SELECT CL1 |
| 20 | CUT ▶ | ("uu1u u1uu")$_b$ | ▶ LT | SAK |
| 21 | CUT ◀ | '**95 20**' *(no CRC_A)* | ◀ LT | ANTICOLLISION CL2 |
| 22 | CUT ▶ | ***UID CL2 + BCC*** *(no CRC_A)* | ▶ LT | UID CL2 + BCC |
| 23 | CUT ◀ | '**95 70**' + ***UID CL2 + BCC*** | ◀ LT | SELECT CL2 |
| 24 | CUT ▶ | ("uu1u u1uu")$_b$ | ▶ LT | SAK |
| 25 | CUT ◀ | '**97 20**' *(no CRC_A)* | ◀ LT | ANTICOLLISION CL3 |
| 26 | CUT ▶ | ***UID CL3 + BCC*** *(no CRC_A)* | ▶ LT | UID CL3 + BCC |
| End of test: Reset PICC | | | | |

## Scenario 40: Type A Installation with error in the READY'' state (UID size = 3 / xy=00 to 16)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.23. Type A Installation with error in the ACTIVE state (UID size = 1) [CA216.xy]

**Test Codification:**

CA216.xy

**ICS Parameters:**

PICC Type = A/ UID size = 1

**Test Objective:**

To ensure that the Type A PICC with a single size UID behaves correctly when it receives an erroneous frame in the ACTIVE state (i.e. in response to a SAK = (“uu1u u0uu”)$_b$).

**Reference Requirements:**

5.3.2.4, 5.7.1.4.a, 7.1.1.2, 7.1.1.3, 7.2.6.2, 7.2.6.4

**Procedure:**

Run the following scenario.

On reception of a SAK = (“uu1u u0uu”)$_b$, the LT sends an erroneous frame.

After having sent the erroneous frame, the LT stays in reception mode during Mute_Time before sending a correct RATS frame.

After having sent the RATS frame, the LT stays in reception mode during Mute_Time before sending a correct WUPA Command frame.

The following errors are generated by the LT:

- xy=00: RATS Command frame with a wrong least significant bit in the first byte (CRC_A1) of CRC_A
- xy=01: RATS Command frame with a wrong least significant bit in the second byte (CRC_A2) of CRC_A
- xy=02: RATS Command frame with a wrong most significant bit in the second byte (CRC_A2) of CRC_A
- xy=03: RATS Command frame with Parity error on the first byte (‘E0’)
- xy=04: RATS Command frame with Parity error on the second byte (‘80’)
- xy=05: RATS Command frame with Parity error on the first byte of the CRC bytes
- xy=06: RATS Command frame with Parity error on the second byte of the CRC bytes
- xy=07: WUPA Command frame
- xy=08: ANTICOLLISION CL1 Command frame
- xy=09: SELECT CL1 Command frame
- xy=10: RATS command with CID=15 (RATS = ‘E0 8F’)

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the correct RATS Command frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command frame, the CUT replies with a correct ATQA.

All UID CL1s sent by CUT have the same value.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Optional Accepted Behavior:**

For xy=00 to 06, on reception of a frame with transmission error, the PICC may return a Mifare NAK, i.e. four Type A bits with a value different from $(1010)_b$ preceded by a Start of Frame (SoF), instead of staying mute (Mifare OS compatibility).

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' **+** ***UID CL1 + BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | **Erroneous frame in function of 'xy'**:<br>**For xy=00 to 06**: '**E0 80**' **with a transmission error**<br>**For xy=07**: '**52**' *(short frame)*<br>**For xy=08**: '**93 20**' *(no CRC_A)*<br>**For xy=09**: '**93 70**' **+** ***UID CL1 + BCC***<br>**For xy=10: 'E0 8F'** *(RATS with CID=15)* | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time<br>*(for xy = 00 to 06, the CUT may also return a Mifare NAK)* | ▶ **LT** | |
| 8 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 9 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA |
| 10 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 11 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 12 | **CUT** ▶ | ***UID CL1 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 13 | **CUT** ◀ | '**93 70**' **+** ***UID CL1 + BCC*** | ◀ **LT** | SELECT CL1 |
| 14 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| End of test: Reset PICC | | | | |

**Scenario 41: Type A Installation with error in the ACTIVE state (UID size = 1 / xy=00 to 10)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.24. Type A Installation with error in the ACTIVE state (UID size = 2) [CA217.xy]

**Test Codification:**

CA217.xy

**ICS Parameters:**

PICC Type = A/ UID size = 2

**Test Objective:**

To ensure that the Type A PICC with a double size UID behaves correctly when it receives an erroneous frame in the ACTIVE state (i.e. in response to a SAK = (“uu1u u0uu”)$_b$).

**Reference Requirements:**

5.7.1.4.a, 7.1.1.2, 7.1.1.3, 7.2.6.2, 7.2.6.4

**Procedure:**

Run the following scenario.

On reception of a SAK = (“uu1u u0uu”)$_b$, the LT sends an erroneous frame.

After having sent the erroneous frame, the LT stays in reception mode during Mute_Time before sending a correct RATS frame.

After having sent the RATS frame, the LT stays in reception mode during Mute_Time before sending a correct WUPA Command frame.

The following errors are generated by the LT:

- xy=00: RATS Command frame with a wrong least significant bit in the first byte (CRC_A1) of CRC_A
- xy=01: RATS Command frame with a wrong least significant bit in the second byte (CRC_A2) of CRC_A
- xy=02: RATS Command frame with a wrong most significant bit in the second byte (CRC_A2) of CRC_A
- xy=03: RATS Command frame with Parity error on the first byte (‘E0’)
- xy=04: RATS Command frame with Parity error on the second byte (‘80’)
- xy=05: RATS Command frame with Parity error on the first byte of the CRC bytes
- xy=06: RATS Command frame with Parity error on the second byte of the CRC bytes
- xy=07: WUPA Command frame
- xy=08: ANTICOLLISION CL2 Command frame
- xy=09: SELECT CL2 Command frame
- xy=10: RATS command with CID=15 (RATS = ‘E0 8F’)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the correct RATS Command frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command frame, the CUT replies with a correct ATQA.

All UID CL1s sent by CUT have the same value.

All UID CL2s sent by CUT have the same value.

**Optional Accepted Behavior:**

For xy=00 to 06, on reception of a frame with transmission error, the PICC may return a Mifare NAK, i.e. four Type A bits with a value different from $(1010)_b$ preceded by a Start of Frame (SoF), instead of staying mute (Mifare OS compatibility).

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | **Erroneous frame in function of 'xy'**:<br>**For xy=00 to 06**: '**E0 80**' **with a transmission error**<br>**For xy=07**: '**52**' *(short frame)*<br>**For xy=08**: '**95 20**' *(no CRC_A)*<br>**For xy=09**: '**95 70**' + ***UID CL2* + *BCC***<br>**For xy=10: 'E0 8F'** *(RATS with CID=15)* | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time<br>*(for xy = 00 to 06, the CUT may also return a Mifare NAK)* | ▶ **LT** | |
| 12 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 13 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA |
| 14 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 15 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 16 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 17 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 18 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 19 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 20 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 21 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 22 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| End of test: Reset PICC | | | | |

## Scenario 42: Type A Installation with error in the ACTIVE state (UID size = 2 / xy=00 to 10)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.25. Type A Installation with error in the ACTIVE state (UID size = 3) [CA218.xy]

**Test Codification:**

CA218.xy

**ICS Parameters:**

PICC Type = A/ UID size = 3

**Test Objective:**

To ensure that the Type A PICC with a triple size UID behaves correctly when it receives an erroneous frame in the ACTIVE state (i.e. in response to a SAK = (“uu1u u0uu”)$_b$).

**Reference Requirements:**

5.7.1.4.a, 7.1.1.2, 7.1.1.3, 7.2.6.2, 7.2.6.4

**Procedure:**

Run the following scenario.

On reception of a SAK = (“uu1u u0uu”)$_b$, the LT sends an erroneous frame.

After having sent the erroneous frame, the LT stays in reception mode during Mute_Time before sending a correct RATS frame.

After having sent the RATS frame, the LT stays in reception mode during Mute_Time before sending a correct WUPA Command frame.

The following errors are generated by the LT:

- xy=00: RATS Command frame with a wrong least significant bit in the first byte (CRC_A1) of CRC_A
- xy=01: RATS Command frame with a wrong least significant bit in the second byte (CRC_A2) of CRC_A
- xy=02: RATS Command frame with a wrong most significant bit in the second byte (CRC_A2) of CRC_A
- xy=03: RATS Command frame with Parity error on the first byte (‘E0’)
- xy=04: RATS Command frame with Parity error on the second byte (‘80’)
- xy=05: RATS Command frame with Parity error on the first byte of the CRC bytes
- xy=06: RATS Command frame with Parity error on the second byte of the CRC bytes
- xy=07: WUPA Command frame
- xy=08: ANTICOLLISION CL3 Command frame
- xy=09: SELECT CL3 Command frame
- xy=10: RATS command with CID=15 (RATS = ‘E0 8F’)

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the correct RATS Command frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command frame, the CUT replies with a correct ATQA.

All UID CL1s sent by CUT have the same value.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

All UID CL2s sent by CUT have the same value.

All UID CL3s sent by CUT have the same value.

**Optional Accepted Behavior:**

For xy=00 to 06, on reception of a frame with transmission error, the PICC may return a Mifare NAK, i.e. four Type A bits with a value different from $(1010)_b$ preceded by a Start of Frame (SoF), instead of staying mute (Mifare OS compatibility).

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' **+** ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ | '**95 70**' **+** ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | '**97 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ▶ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◀ | '**97 70**' **+** ***UID CL3* + *BCC*** | ◀ **LT** | SELECT CL3 |
| 14 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 15 | **CUT** ◀ | **Erroneous frame in function of 'xy'**:<br>**For xy=00 to 06**: '**E0 80**' **with a transmission error**<br>**For xy=07**: '**52**' *(short frame)*<br>**For xy=08**: '**97 20**' *(no CRC_A)*<br>**For xy=09**: '**97 70**' **+** ***UID CL1* + *BCC***<br>**For xy=10: 'E0 8F'** *(RATS with CID=15)* | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time<br>*(for xy = 00 to 06, the CUT may also return a Mifare NAK)* | ▶ **LT** | |
| 16 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 17 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA |
| 18 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 19 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 20 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 21 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 22 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 23 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 24 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 25 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 26 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 27 | **CUT** ◀ | '**97 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL3 |
| 28 | **CUT** ▶ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL3 + BCC |
| 29 | **CUT** ◀ | '**97 70**' + ***UID CL3* + *BCC*** | ◀ **LT** | SELECT CL3 |
| 30 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| End of test: Reset PICC | | | | |

**Scenario 43: Type A Installation with error in the ACTIVE state (UID size = 3 / xy=00 to 10)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.26. Type A Installation with error in the HALT state following ACTIVE state (UID size = 1) [CA226]

**Test Codification:**

CA226

**ICS Parameters:**

PICC Type = A / UID size = 1

**Test Objective:**

To ensure that the Type A PICC with a single size UID behaves correctly when it receives an erroneous frame in the HALT state entered following the ACTIVE state (i.e. after reception of the HLTA Command in the ACTIVE state).

**Reference Requirements:**

5.3.2.4, 7.1.1.2, 7.1.1.3, 7.2.8.1, 7.2.8.2

**Procedure:**

Run the following scenario.

On reception of a SAK = (“uu1u u0uu”)$_b$, the LT puts CUT into HALT state by sending a correct HLTA Command sequence, waits with carrier on for at least $t_P$ and, then, the LT sends consecutive erroneous sequences.

After each erroneous sequence, the LT ensures that the CUT stays mute (i.e. does not send any frame for at least Mute_Time).

After having sent all the erroneous sequences, the LT sends a correct WUPA Command sequence and completes the initialization as relevant to PICC ‘*’ states.

Following initialization, LT sends Select PPSE command and receives the response.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous sequence, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command sequence, the CUT replies with a correct ATQA.

On reception of the commands during installation (as relevant to PICC ‘*’ states) and the Select PPSE command, the CUT replies with correct response and behaves as described in the following scenario.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◀ ‘**52**’ *(short frame)* ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ ***ATQA*** *(no CRC_A)* ▶ **LT** | ATQA |
| 3 | **CUT** ◀ ’**93 20**’ *(no CRC_A)* ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ ***UID CL1* + *BCC*** *(no CRC_A)* ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ ’**93 70**’ + ***UID CL1* + *BCC*** ◀ **LT** | SELECT CL1 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 6 | **CUT** ▸ | (“uu1u u0uu”)$_b$ | ▸ **LT** | SAK |
| 7 | **CUT** ◂ | ’**50 00**’ | ◂ **LT** | HLTA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 8 | **CUT** ◂ | ’**93 20**’ *(no CRC_A)* sent after at least $t_P$ | ◂ **LT** | ANTICOLLISION CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 9 | **CUT** ◂ | ’**93 70**’ **+ *UID CL1* + *BCC*** | ◂ **LT** | SELECT CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 10 | **CUT** ◂ | ’**E0 80**’ | ◂ **LT** | RATS |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 11 | **CUT** ◂ | ‘**26**’ *(short frame)* | ◂ **LT** | REQA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 12 | **CUT** ◂ | ‘**52**’ *(short frame)* | ◂ **LT** | WUPA |
| 13 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 14 | **CUT** ◂ | ’**93 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL1 |
| 15 | **CUT** ▸ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL1 + BCC |
| 16 | **CUT** ◂ | ’**93 70**’ **+ *UID CL1* + *BCC*** | ◂ **LT** | SELECT CL1 |
| 17 | **CUT** ▸ | (“uu1u u0uu”)$_b$ | ▸ **LT** | SAK |
| 18 | **CUT** ◂ | **‘E0 80’** | ◂ **LT** | RATS (FSD = 256) |
| 19 | **CUT** ▸ | **ATS** | ▸ **LT** | ATS |
| 20 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** | ◂ **LT** | Select PPSE / b.n. = 0 |
| 21 | **CUT** ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

## Scenario 44: Type A Installation with error in the HALT state following ACTIVE state (UID size = 1)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.27. Type A Installation with error in the HALT state following ACTIVE state (UID size = 2) [CA227]

**Test Codification:**

CA227

**ICS Parameters:**

PICC Type = A / UID size = 2

**Test Objective:**

To ensure that the Type A PICC with a double size UID behaves correctly when it receives an erroneous frame in the HALT state entered following the ACTIVE state (i.e. after reception of the HLTA Command in the ACTIVE state).

**Reference Requirements:**

7.1.1.2, 7.1.1.3, 7.2.8.1, 7.2.8.2

**Procedure:**

Run the following scenario.

On reception of a SAK = (“uu1u u0uu”)$_b$, the LT puts CUT into HALT state by sending a correct HLTA Command sequence, waits with carrier on for at least $t_P$ and, then, the LT sends consecutive erroneous sequences.

After each erroneous sequence, the LT ensures that the CUT stays mute (i.e. does not send any frame for at least Mute_Time).

After having sent all the erroneous sequences, the LT sends a correct WUPA Command sequence and completes the initialization as relevant to PICC ‘*’ states.

Following initialization, LT sends Select PPSE command and receives the response.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous sequence, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command sequence, the CUT replies with a correct ATQA.

On reception of the commands during installation (as relevant to PICC ‘*’ states) and the Select PPSE command, the CUT replies with correct response and behaves as described in the following scenario. All UID CL1s sent by CUT have the same value.

All UID CL2s sent by CUT have the same value.

For fixed UID PICCs: The UID sent by the PICC is the one expected as indicated in ICS.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ '**93 70**' **+** ***UID CL1 + BCC*** ◄ **LT** | SELECT CL1 |
| 6 | **CUT** ► ("uu1u u1uu")$_b$ ► **LT** | SAK |
| 7 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ► ***UID CL2 + BCC*** *(no CRC_A)* ► **LT** | UID CL2 + BCC |
| 9 | **CUT** ◄ '**95 70**' **+** ***UID CL2 + BCC*** ◄ **LT** | SELECT CL2 |
| 10 | **CUT** ► ("uu1u u0uu")$_b$ ► **LT** | SAK |
| 11 | **CUT** ◄ '**50 00**' ◄ **LT** | HLTA |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ► **LT** | |
| 12 | **CUT** ◄ '**93 20**' *(no CRC_A)* sent after at least $t_P$ ◄ **LT** | ANTICOLLISION CL1 |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ► **LT** | |
| 13 | **CUT** ◄ '**93 70**' **+** ***UID CL1 + BCC*** ◄ **LT** | SELECT CL1 |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ► **LT** | |
| 14 | **CUT** ◄ '**E0 80**' ◄ **LT** | RATS |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ► **LT** | |
| 15 | **CUT** ◄ '**26**' *(short frame)* ◄ **LT** | REQA |
| | **CUT** ‖ The CUT does not send any frame during Mute_Time ► **LT** | |
| 16 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA |
| 17 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 18 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |
| 19 | **CUT** ► ***UID CL1 + BCC*** *(no CRC_A)* ► **LT** | UID CL1 + BCC |
| 20 | **CUT** ◄ '**93 70**' **+** ***UID CL1 + BCC*** ◄ **LT** | SELECT CL1 |
| 21 | **CUT** ► ("uu1u u1uu")$_b$ ► **LT** | SAK |
| 22 | **CUT** ◄ '**95 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL2 |
| 23 | **CUT** ► ***UID CL2 + BCC*** *(no CRC_A)* ► **LT** | UID CL2 + BCC |
| 24 | **CUT** ◄ '**95 70**' **+** ***UID CL2 + BCC*** ◄ **LT** | SELECT CL2 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 25 | **CUT** ▸ | (“uu1u u0uu”)$_b$ | ▸ **LT** | SAK |
| 26 | **CUT** ◂ | **‘E0 80’** | ◂ **LT** | RATS (FSD = 256) |
| 27 | **CUT** ▸ | **ATS** | ▸ **LT** | ATS |
| 28 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** | ◂ **LT** | Select PPSE / b.n. = 0 |
| 29 | **CUT** ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

**Scenario 45: Type A Installation with error in the HALT state following ACTIVE State (UID size = 2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.28. Type A Installation with error in the HALT state following ACTIVE state (UID size = 3) [CA228]

**Test Codification:**

CA228

**ICS Parameters:**

PICC Type = A / UID size = 3

**Test Objective:**

To ensure that the Type A PICC with a triple size UID behaves correctly when it receives an erroneous frame in the HALT state entered following the ACTIVE state (i.e. after reception of the HLTA Command in the ACTIVE state).

**Reference Requirements:**

7.1.1.2, 7.1.1.3, 7.2.8.1, 7.2.8.2

**Procedure:**

Run the following scenario.

On reception of a SAK = (“uu1u u0uu”)$_b$, the LT puts CUT into HALT state by sending a correct HLTA Command sequence, waits with carrier on for at least $t_P$ and, then, the LT sends consecutive erroneous sequences.

After each erroneous sequence, the LT ensures that the CUT stays mute (i.e. does not send any frame for at least Mute_Time).

After having sent all the erroneous sequences, the LT sends a correct WUPA Command sequence and completes the initialization as relevant to PICC ‘*’ states.

Following initialization, LT sends Select PPSE command and receives the response.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous sequence, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command sequence, the CUT replies with a correct ATQA.

On reception of the commands during installation (as relevant to PICC ‘*’ states) and the Select PPSE command, the CUT replies with correct response and behaves as described in the following scenario. All UID CL1s sent by CUT have the same value.

All UID CL2s sent by CUT have the same value.

All UID CL3s sent by CUT have the same value.

For fixed UID PICCs: The UID sent by the PICC is the one expected as indicated in ICS.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | '**97 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ▶ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◀ | '**97 70**' + ***UID CL3* + *BCC*** | ◀ **LT** | SELECT CL3 |
| 14 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 15 | **CUT** ◀ | '**50 00**' | ◀ **LT** | HLTA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 16 | **CUT** ◀ | '**93 20**' *(no CRC_A)* sent after at least $t_P$ | ◀ **LT** | ANTICOLLISION CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 17 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 18 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 19 | **CUT** ◀ | '**26**' *(short frame)* | ◀ **LT** | REQA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 20 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA |
| 21 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 22 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 23 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 24 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 25 | **CUT** ▸ | (“uu1u u1uu”)$_b$ | ▸ **LT** | SAK |
| 26 | **CUT** ◂ | ’**95 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL2 |
| 27 | **CUT** ▸ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL2 + BCC |
| 28 | **CUT** ◂ | ’**95 70**’ + ***UID CL2* + *BCC*** | ◂ **LT** | SELECT CL2 |
| 29 | **CUT** ▸ | (“uu1u u1uu”)$_b$ | ▸ **LT** | SAK |
| 30 | **CUT** ◂ | ’**97 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL3 |
| 31 | **CUT** ▸ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL3 + BCC |
| 32 | **CUT** ◂ | ’**97 70**’ + ***UID CL3* + *BCC*** | ◂ **LT** | SELECT CL3 |
| 33 | **CUT** ▸ | (“uu1u u0uu”)$_b$ | ▸ **LT** | SAK |
| 34 | **CUT** ◂ | **‘E0 80’** | ◂ **LT** | RATS (FSD = 256) |
| 35 | **CUT** ▸ | **ATS** | ▸ **LT** | ATS |
| 36 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** | ◂ **LT** | Select PPSE / b.n. = 0 |
| 37 | **CUT** ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

## Scenario 46: Type A Installation with error in the HALT state following ACTIVE State (UID size = 3)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.29. Type A Installation with error in the HALT state following PROTOCOL State (UID size = 1) [CA230]

**Test Codification:**

CA230

**ICS Parameters:**

PICC Type = A / UID size = 1

**Test Objective:**

To ensure that the Type A PICC with a single size UID behaves correctly when it receives an erroneous frame in the HALT state (i.e. after reception of the S(DESELECT) Response in the PROTOCOL state) and continues to respond during installation as relevant to PICC '*' states.

**Reference Requirements:**

5.3.2.4, 7.1.1.2, 7.1.1.3, 7.2.8.1, 7.2.8.2

**Procedure:**

Run the following scenario.

Following the ATS response from the Type A PICC, the LT sends a correct S(DESELECT) Request sequence and receives S(DESELECT) Response sequence, waits with carrier on for at least $t_P$ and, then, the LT sends consecutive erroneous sequences.

After each erroneous sequence, the LT ensures that the CUT stays mute (i.e. does not send any frame for at least Mute_Time).

After having sent all the erroneous sequences, the LT sends a correct WUPA Command sequence and completes the initialization as relevant to PICC '*' states.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On Reception of S(DESELECT) Request the CUT responds with correct S(DESELECT) Response.

On reception of an erroneous sequence, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command sequence, the CUT replies with a correct ATQA.

On reception of the commands during installation (as relevant to PICC '*' states) and the Select PPSE command, the CUT replies with correct response.

All UID CL1s sent by CUT have the same value.

For fixed UID PICCs: The UID sent by the PICC is the one expected as indicated in ICS.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ '**52**' *(short frame)* ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► ***ATQA*** *(no CRC_A)* ► **LT** | ATQA |
| 3 | **CUT** ◄ '**93 20**' *(no CRC_A)* ◄ **LT** | ANTICOLLISION CL1 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 4 | **CUT** ▸ | ***UID CL1 + BCC*** *(no CRC_A)* | ▸ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◂ | **'93 70' + *UID CL1 + BCC*** | ◂ **LT** | SELECT CL1 |
| 6 | **CUT** ▸ | ("uu1u u0uu")$_b$ | ▸ **LT** | SAK |
| 7 | **CUT** ◂ | **'E0 80'** | ◂ **LT** | RATS (FSD = 256) |
| 8 | **CUT** ▸ | **ATS** | ▸ **LT** | ATS |
| 9 | **CUT** ◂ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◂ **LT** | Select PPSE / b.n. = 0 |
| 10 | **CUT** ▸ | **I(0)$_0$ ['6F'** + n data bytes + **'90 00']** | ▸ **LT** | b.n. = 0 |
| 11 | **CUT** ◂ | **S(DESELECT) Request** | ◂ **LT** | DESELECT Request |
| 12 | **CUT** ▸ | **S(DESELECT) Response** | ▸ **LT** | DESELECT Response |
| 13 | **CUT** ◂ | **'93 20'** *(no CRC_A)* sent after at least $t_P$ | ◂ **LT** | ANTICOLLISION CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 14 | **CUT** ◂ | **'93 70' + *UID CL1 + BCC*** | ◂ **LT** | SELECT CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 15 | **CUT** ◂ | **'E0 80'** | ◂ **LT** | RATS |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 16 | **CUT** ◂ | **'26'** *(short frame)* | ◂ **LT** | REQA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 17 | **CUT** ◂ | **'52'** *(short frame)* | ◂ **LT** | WUPA |
| 18 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 19 | **CUT** ◂ | **'93 20'** *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL1 |
| 20 | **CUT** ▸ | ***UID CL1 + BCC*** *(no CRC_A)* | ▸ **LT** | UID CL1 + BCC |
| 21 | **CUT** ◂ | **'93 70' + *UID CL1 + BCC*** | ◂ **LT** | SELECT CL1 |
| 22 | **CUT** ▸ | ("uu1u u0uu")$_b$ | ▸ **LT** | SAK |
| 23 | **CUT** ◂ | **'E0 80'** | ◂ **LT** | RATS (FSD = 256) |
| 24 | **CUT** ▸ | **ATS** | ▸ **LT** | ATS |
| 25 | **CUT** ◂ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◂ **LT** | Select PPSE / b.n. = 0 |
| 26 | **CUT** ▸ | **I(0)$_0$ ['6F'** + n data bytes + **'90 00']** | ▸ **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

## Scenario 47: Type A Installation with error in the HALT state following PROTOCOL state (UID size = 1)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.30. Type A Installation with error in the HALT state following PROTOCOL State (UID size = 2) [CA231]

**Test Codification:**

CA231

**ICS Parameters:**

PICC Type = A / UID size = 2

**Test Objective:**

To ensure that the Type A PICC with a double size UID behaves correctly when it receives an erroneous frame in the HALT state (i.e. after reception of the S(DESELECT) Response in the PROTOCOL state) and continues to respond during installation as relevant to PICC '*' states.

**Reference Requirements:**

7.1.1.2, 7.1.1.3, 7.2.8.1, 7.2.8.2

**Procedure:**

Run the following scenario.

Following the ATS response from the Type A PICC, the LT sends a correct S(DESELECT) Request sequence and receives S(DESELECT) Response sequence, waits with carrier on for at least $t_P$ and, then, the LT sends consecutive erroneous sequences.

After each erroneous sequence, the LT ensures that the CUT stays mute (i.e. does not send any frame for at least Mute_Time).

After having sent all the erroneous sequences, the LT sends a correct WUPA Command sequence and completes the initialization as relevant to PICC '*' states.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On Reception of S(DESELECT) Request the CUT responds with correct S(DESELECT) Response.

On reception of an erroneous sequence, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command sequence, the CUT replies with a correct ATQA.

On reception of the commands during installation (as relevant to PICC '*' states) and the Select PPSE command, the CUT replies with correct response.

All UID CL1s sent by CUT have the same value.

All UID CL2s sent by CUT have the same value.

For fixed UID PICCs: The UID sent by the PICC is the one expected as indicated in ICS.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◄ | '**52**' *(short frame)* | ◄ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ► | ***ATQA*** *(no CRC_A)* | ► **LT** | ATQA |
| 3 | **CUT** ◄ | '**93 20**' *(no CRC_A)* | ◄ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ► | ***UID CL1* + *BCC*** *(no CRC_A)* | ► **LT** | UID CL1 + BCC |
| 5 | **CUT** ◄ | '**93 70**' + ***UID CL1* + *BCC*** | ◄ **LT** | SELECT CL1 |
| 6 | **CUT** ► | ("uu1u u1uu")$_b$ | ► **LT** | SAK |
| 7 | **CUT** ◄ | '**95 20**' *(no CRC_A)* | ◄ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ► | ***UID CL2* + *BCC*** *(no CRC_A)* | ► **LT** | UID CL2 + BCC |
| 9 | **CUT** ◄ | '**95 70**' + ***UID CL2* + *BCC*** | ◄ **LT** | SELECT CL2 |
| 10 | **CUT** ► | ("uu1u u0uu")$_b$ | ► **LT** | SAK |
| 11 | **CUT** ◄ | **'E0 80'** | ◄ **LT** | RATS (FSD = 256) |
| 12 | **CUT** ► | **ATS** | ► **LT** | ATS |
| 13 | **CUT** ◄ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◄ **LT** | Select PPSE / b.n. = 0 |
| 14 | **CUT** ► | **I(0)$_0$ ['6F'** + n data bytes + **'90 00']** | ► **LT** | b.n. = 0 |
| 15 | **CUT** ◄ | **S(DESELECT) Request** | ◄ **LT** | DESELECT Request |
| 16 | **CUT** ► | **S(DESELECT) Response** | ► **LT** | DESELECT Response |
| 17 | **CUT** ◄ | '**93 20**' *(no CRC_A)* sent after at least $t_P$ | ◄ **LT** | ANTICOLLISION CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ► **LT** | |
| 18 | **CUT** ◄ | '**93 70**' + ***UID CL1* + *BCC*** | ◄ **LT** | SELECT CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ► **LT** | |
| 19 | **CUT** ◄ | '**E0 80**' | ◄ **LT** | RATS |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ► **LT** | |
| 20 | **CUT** ◄ | '**26**' *(short frame)* | ◄ **LT** | REQA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ► **LT** | |
| 21 | **CUT** ◄ | '**52**' *(short frame)* | ◄ **LT** | WUPA |
| 22 | **CUT** ► | ***ATQA*** *(no CRC_A)* | ► **LT** | ATQA |
| 23 | **CUT** ◄ | '**93 20**' *(no CRC_A)* | ◄ **LT** | ANTICOLLISION CL1 |
| 24 | **CUT** ► | ***UID CL1* + *BCC*** *(no CRC_A)* | ► **LT** | UID CL1 + BCC |
| 25 | **CUT** ◄ | '**93 70**' + ***UID CL1* + *BCC*** | ◄ **LT** | SELECT CL1 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 26 | **CUT** ▸ | ("uu1u u1uu")$_b$ | ▸ **LT** | SAK |
| 27 | **CUT** ◂ | '**95 20**' *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL2 |
| 28 | **CUT** ▸ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL2 + BCC |
| 29 | **CUT** ◂ | '**95 70**' + ***UID CL2* + *BCC*** | ◂ **LT** | SELECT CL2 |
| 30 | **CUT** ▸ | ("uu1u u0uu")$_b$ | ▸ **LT** | SAK |
| 31 | **CUT** ◂ | **'E0 80'** | ◂ **LT** | RATS (FSD = 256) |
| 32 | **CUT** ▸ | **ATS** | ▸ **LT** | ATS |
| 33 | **CUT** ◂ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◂ **LT** | Select PPSE / b.n. = 0 |
| 34 | **CUT** ▸ | **I(0)$_0$ ['6F'** + n data bytes + **'90 00']** | ▸ **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

**Scenario 48: Type A Installation with error in the HALT state following PROTOCOL State (UID size = 2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.31. Type A Installation with error in the HALT state following PROTOCOL State (UID size = 3) [CA232]

**Test Codification:**

CA232

**ICS Parameters:**

PICC Type = A / UID size = 3

**Test Objective:**

To ensure that the Type A PICC with a triple size UID behaves correctly when it receives an erroneous frame in the HALT state (i.e. after reception of the S(DESELECT) Response in the PROTOCOL state) and continues to respond during installation as relevant to PICC '*' states.

**Reference Requirements:**

7.1.1.2, 7.1.1.3, 7.2.8.1, 7.2.8.2

**Procedure:**

Run the following scenario.

Following the ATS response from the Type A PICC, the LT sends a correct S(DESELECT) Request sequence and receives S(DESELECT) Response sequence, waits with carrier on for at least $t_P$ and, then, the LT sends consecutive erroneous sequences.

After each erroneous sequence, the LT ensures that the CUT stays mute (i.e. does not send any frame for at least Mute_Time).

After having sent all the erroneous sequences, the LT sends a correct WUPA Command sequence and completes the initialization as relevant to PICC '*' states.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On Reception of S(DESELECT) Request the CUT responds with correct S(DESELECT) Response.

On reception of an erroneous sequence, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPA Command sequence, the CUT replies with a correct ATQA.

On reception of the commands during installation (as relevant to PICC '*' states) and the Select PPSE command, the CUT replies with correct response.

All UID CL1s sent by CUT have the same value.

All UID CL2s sent by CUT have the same value.

All UID CL3s sent by CUT have the same value.

For fixed UID PICCs: The UID sent by the PICC is the one expected as indicated in ICS.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | '**95 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL2 |
| 8 | **CUT** ▶ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL2 + BCC |
| 9 | **CUT** ◀ | '**95 70**' + ***UID CL2* + *BCC*** | ◀ **LT** | SELECT CL2 |
| 10 | **CUT** ▶ | ("uu1u u1uu")$_b$ | ▶ **LT** | SAK |
| 11 | **CUT** ◀ | '**97 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL3 |
| 12 | **CUT** ▶ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▶ **LT** | UID CL3 + BCC |
| 13 | **CUT** ◀ | '**97 70**' + ***UID CL3* + *BCC*** | ◀ **LT** | SELECT CL3 |
| 14 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 15 | **CUT** ◀ | **'E0 80'** | ◀ **LT** | RATS (FSD = 256) |
| 16 | **CUT** ▶ | **ATS** | ▶ **LT** | ATS |
| 17 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◀ **LT** | Select PPSE / b.n. = 0 |
| 18 | **CUT** ▶ | **I(0)$_0$ ['6F'** + n data bytes + **'90 00']** | ▶ **LT** | b.n. = 0 |
| 19 | **CUT** ◀ | **S(DESELECT) Request** | ◀ **LT** | DESELECT Request |
| 20 | **CUT** ▶ | **S(DESELECT) Response** | ▶ **LT** | DESELECT Response |
| 21 | **CUT** ◀ | '**93 20**' *(no CRC_A)* sent after at least $t_P$ | ◀ **LT** | ANTICOLLISION CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 22 | **CUT** ◀ | '**93 70**' + ***UID CL1* + *BCC*** | ◀ **LT** | SELECT CL1 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 23 | **CUT** ◀ | '**E0 80**' | ◀ **LT** | RATS |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 24 | **CUT** ◀ | '**26**' *(short frame)* | ◀ **LT** | REQA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 25 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 26 | **CUT** ▸ | ***ATQA*** *(no CRC_A)* | ▸ **LT** | ATQA |
| 27 | **CUT** ◂ | ’**93 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL1 |
| 28 | **CUT** ▸ | ***UID CL1* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL1 + BCC |
| 29 | **CUT** ◂ | ’**93 70**’ **+** ***UID CL1* + *BCC*** | ◂ **LT** | SELECT CL1 |
| 30 | **CUT** ▸ | (“uu1u u1uu”)$_b$ | ▸ **LT** | SAK |
| 31 | **CUT** ◂ | ’**95 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL2 |
| 32 | **CUT** ▸ | ***UID CL2* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL2 + BCC |
| 33 | **CUT** ◂ | ’**95 70**’ **+** ***UID CL2* + *BCC*** | ◂ **LT** | SELECT CL2 |
| 34 | **CUT** ▸ | (“uu1u u1uu”)$_b$ | ▸ **LT** | SAK |
| 35 | **CUT** ◂ | ’**97 20**’ *(no CRC_A)* | ◂ **LT** | ANTICOLLISION CL3 |
| 36 | **CUT** ▸ | ***UID CL3* + *BCC*** *(no CRC_A)* | ▸ **LT** | UID CL3 + BCC |
| 37 | **CUT** ◂ | ’**97 70**’ **+** ***UID CL3* + *BCC*** | ◂ **LT** | SELECT CL3 |
| 38 | **CUT** ▸ | (“uu1u u0uu”)$_b$ | ▸ **LT** | SAK |
| 39 | **CUT** ◂ | **‘E0 80’** | ◂ **LT** | RATS (FSD = 256) |
| 40 | **CUT** ▸ | **ATS** | ▸ **LT** | ATS |
| 41 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** | ◂ **LT** | Select PPSE / b.n. = 0 |
| 42 | **CUT** ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

## Scenario 49: Type A Installation with error in the HALT state following PROTOCOL State (UID size = 3)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 3.32. Type A Installation with polling and with PICC Reset [CA233.xy]

**Test Codification:**

CA233.xy

**ICS Parameters:**

PICC Type = A

**Test Objective:**

To ensure that the Type A PICC supports the minimum timing with carrier on before WUPA and REQA Commands (i.e. $t_P$).

To ensure that the Type A PICC goes back to the IDLE state when the PCD performs a PICC Reset during installation.

**Reference Requirements:**

3.2.7.1, 7.2.2.1, 7.1.1.4, 7.1.1.5

**Procedure:**

Run the following scenario with step 7 in function of UID size.

Following the WUPB Command, the LT sends a WUPA Command sequence (xy=00 and 01) or REQA Command sequence (xy=10 and 11) sent using the delay $t_X$ (for y=0: $t_X$ = 5ms and for y=1: $t_X$ = 30ms) between the end of the WUPB Command sequence and the beginning of the WUPA Command sequence (xy=00 and 01) or REQA Command sequence (xy=10 and 11).

On reception of the first SAK during the collision detection phase, the LT performs a PICC Reset i.e. stops sending the carrier (no RF field $H_{OV}$) for a time $t_{RESET}$. Then the LT sends the carrier for a time $t_P$ before sending a WUPA Command Sequence (xy=00 and 01) or REQA Command sequence (xy=10 and 11).

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of the last WUPA Command (xy=00 and 01) or REQA Command (xy=10 and 11), the CUT sends the ATQA.

For fixed UID PICCs: The UID CL1 sent by the PICC is the one expected as indicated in ICS.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | **For xy=00 and 01:** '**52**' *(short frame)* sent with a delay equal to $t_P$<br>**For xy=10 and 11: '26'** *(short frame)* sent with a delay equal to $t_P$ | ◀ **LT** | WUPA/REQA to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 3 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 4 | **CUT** ▶ | ***UID CL1 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 5 | **CUT** ◀ | '**93 70**' **+** ***UID CL1 + BCC*** | ◀ **LT** | SELECT CL1 |
| 6 | **CUT** ▶ | ("uu1u u0uu")$_b$ | ▶ **LT** | SAK |
| 7 | **CUT** ◀ | **For xy=00 and 01:** '**52**' *(short frame)*<br>**For xy=10 and 11: '26'** *(short frame)* | ◀ **LT** | WUPA/REQA to poll for the CUT |
| 8 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent after $t_P$ | ◀ **LT** | WUPB (polling) |
| 9 | **CUT** ◀ | **For xy=00 and 01:** '**52**' *(short frame)* sent with a delay equal to $\mathbf{t_X}$ in function of 'y'<br>**For xy=10 and 11: '26'** *(short frame)* sent with a delay equal to $\mathbf{t_X}$ in function of 'y' | ◀ **LT** | WUPA/REQA to poll for the CUT |
| 10 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 11 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 12 | **CUT** ▶ | ***UID CL1 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| 13 | **CUT** ◀ | '**93 70**' **+** ***UID CL1 + BCC*** | ◀ **LT** | SELECT CL1 |
| 14 | **CUT** ▶ | ("uu1u u0uu")$_b$ (for UID size = 1) or<br>("uu1u u1uu")$_b$ (for UID size = 2 or 3) | ▶ **LT** | SAK (in function of UID size) |
| 15 | **CUT** ◀ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◀ **LT** | PICC Reset |
| 16 | **CUT** ◀ | **For xy=00 and 01:** '**52**' *(short frame)* sent with a delay equal to $t_P$<br>**For xy=10 and 11: '26'** *(short frame)* sent with a delay equal to $t_P$ | ◀ **LT** | Last WUPA/REQA Command |
| 17 | **CUT** ▶ | ***ATQA*** *(no CRC_A)* | ▶ **LT** | ATQA |
| 18 | **CUT** ◀ | '**93 20**' *(no CRC_A)* | ◀ **LT** | ANTICOLLISION CL1 |
| 19 | **CUT** ▶ | ***UID CL1 + BCC*** *(no CRC_A)* | ▶ **LT** | UID CL1 + BCC |
| End of test: Reset PICC | | | | |

## Scenario 50: Type A Installation with polling and with PICC Reset (xy=00, 01, 10 and 11)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4. Type B Executable Tests

## 4.1. Basic Type B Exchange and Timings Measurement [CB001]

**Test Codification:**

CB001

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC respects the frame format, the timings and the order of the commands during a basic exchange (installation and block protocol).

**Reference Requirements:**

4.3.2.4, 4.3.2.6, 4.6.2.4, 4.6.2.64.7.2.1, 4.8.1.7, 4.8.1.9, 4.8.1.11, 4.8.1.15, 6.2.1.1, 6.2.1.2, 6.3.2.1, 6.3.2.5, 6.3.2.9, 6.3.2.11, 6.3.2.12, 6.4.1.11, 6.4.2.1, 8.1.1.5, 10.3.4.8

**Procedure:**

Run the following scenario.

During the whole test, the LT measures the delay between two consecutive characters within the sequences sent by the CUT. As this measurement is not always possible, it can be replaced by the measurement of the delay between the start bits of two consecutive characters.

When a sequence sent by the LT is followed by a sequence sent by the CUT, the LT measures the Frame Delay Time between the end of the sequence sent by the LT and the beginning of the sequence replied by the CUT.

When a sequence sent by the LT is followed by a sequence sent by the CUT, the LT measures the time during which the CUT does not generate a subcarrier just after the end of the sequence sent by the LT and the time during which the CUT generates a subcarrier with no phase transition just before the beginning of the sequence sent by the CUT.

During the removal sequence, the LT applies a delay of at least $t_P$ between the end of the ATQB sequence sent by the CUT and the beginning of the WUPB Command sequence sent by the LT.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

Each frame sent by the CUT is a Type B frame composed of the bytes coding the CUT response (including the CRC). Each byte is transmitted LSB first in a Character which consists of a start bit = Logic “0”, the eight data bits and a stop bit = Logic “1” (these 10 bits long characters are separated by a time EGT – see below).

Each Type B frame sent by the CUT is contained in a sequence starting with a Start of Sequence i.e. SoS = 1272/ $f_c$ to 1416/ $f_c$ of logical state $(0)_b$ followed by 248/ $f_c$ to 392/ $f_c$ of logical state $(1)_b$, and ending with an End of sequence i.e. EoS = 1272/ $f_c$ to 1416/ $f_c$ of logical state $(0)_b$ potentially followed by at most $t_{FSOFF,MAX}$ of logical state $(1)_b$ (then the CUT turns the subcarrier off).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

Remark: The EoS may be coded by only $1272/f_c$ to $1416/f_c$ of logical state ('1') before the subcarrier is directly turned off (when $t_{FSOFF} = 0/f_c$).

All Commands and Block frames sent by the CUT contain 2 CRC_B bytes (last two bytes of the frames) calculated as defined in ISO/IEC 13239 (with a register initialized to 'FFFF').

The measured delay between two consecutive characters within the frames sent by the CUT does not exceed $EGT_{PICC,MAX}$ i.e. the measured delay between the start bits of two consecutive characters does not exceed ($1280/ f_c + EGT_{PICC,MAX}$).

There is no delay between the last character and the EoS within the frames sent by the CUT (i.e. no EGT).

Remark: a warning must be indicated if the measured delay between the start bits of two consecutive characters is less than $1280/ f_c$.

On reception of the WUPB Command before the RESET, the CUT sends always the same ATQB. On reception of the WUPB after the RESET, all the ATQB have the same value. An ATQB shall always be sent having the value:

| **ATQB** | | | |
|---|---|---|---|
| Byte 1 | Byte 2 – 5 | Byte 6 – 9 | Byte 10 – 12 |
| $('50')_h$ | PUPI | Any Value | Protocol Info |

Where: PUPI is a 4 bytes number

- All the PUPIs received before PICC reset have the same value.
- All the PUPIs received after PICC reset have the same value.
- For the PICCs with fixed PUPI: all the PUPIs sent by CUT have the same value during the test as indicated in ICS.
- Byte 1 of Protocol Info (Byte 10 of ATQB) = $('00')_h$ or $('80')_h$
- Byte 2 of Protocol Info (Byte 11 of ATQB) =
  - most significant nibble = Max_Frame_Size where
    - $('2')_h \leq$ Max_Frame_Size $\leq ('C')_h$
  - least significant nibble = $('1')_h$
- Byte 3 of Protocol Info (Byte 12 of ATQB) =
  - most significant nibble = FWI where FWI $\leq ('7')_h$
  - least significant nibble (ADC and FO) where
    - bit 4=0
    - bit 3, bit 2, bit 1 = any value
- No 13th byte (reserved for extended ATQB) in response to WUPB

On reception of an I-Block with a block number equal to 0, the CUT replies with an I-Block having a PCB field (i.e. first byte of the Block) equal to $('02')_h$ *or* $('12')_h$.
On reception of an I-Block with a block number equal to 1, the CUT replies with an I-Block having a PCB field (i.e. first byte of the Block) equal to $('03')_h$ *or* $('13')_h$.

Remark: If a block is received with a PCB field equal to $('12')_h$ or $('13')_h$, it means that the CUT is using chaining.

During the whole transaction (i.e. installation and block protocol), the time during which the CUT does not generate a subcarrier just after the end of the sequences sent by the LT is at least $TR0_{MIN}$.

During the whole transaction (i.e. installation and block protocol), the time during which the CUT generates a subcarrier with no phase transition just before the beginning of the sequences sent by the CUT is at least $TR1_{MIN}$ and is at most $TR1_{MAX}$.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

During the whole transaction after the WUPB Commands, the delay between the end of the sequence sent by the LT and the beginning of the unmodulated subcarrier sent by the CUT is at most $TR0_{MAX,ATQB}$.

During installation and after the HLTB and the ATTRIB Commands, the delay between the end of the sequence sent by the LT and the beginning of the next sequence sent by the CUT is at most FWT = (256 x 16/ $f_c$) x $2^{FWI}$.

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that does not contain an S(WTX) Response and the beginning of the next sequence sent by the CUT is at most FWT = (256 x 16/$f_c$) x $2^{FWI}$.

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that contains an S(WTX) Response and the beginning of the next sequence sent by the CUT is at most $FWT_{TEMP}$ = ((256 x 16/$f_c$) x $2^{FWI}$) x WTXM where WTXM is the FWT Extension Multiplier indicated in the S(WTX) Request and (WTXM x (256 x 16/ $f_c$)x $2^{FWI}$) ≤ $FWT_{MAX}$.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◂ | **'05 00 08'** | ◂ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▸ | ***ATQB*** *(containing PUPI)* sent between $FDT_{B,PICC,MIN}$ and $FWT_{ATQB}$=$TR0_{MAX,ATQB}$+$TR1_{MAX}$ | ▸ **LT** | ATQB |
| 3 | **CUT** ◂ | **'52'** *(Type A short frame)* sent after $t_P$ | ◂ **LT** | WUPA (polling) |
| 4 | **CUT** ◂ | **'05 00 08'** sent after $t_P$ | ◂ **LT** | WUPB to wake up the CUT |
| 5 | **CUT** ▸ | ***ATQB*** *(containing PUPI)* sent between $FDT_{B,PICC,MIN}$ and $FWT_{ATQB}$=$TR0_{MAX,ATQB}$+$TR1_{MAX}$ | ▸ **LT** | ATQB |
| 6 | **CUT** ◂ | **'1D' + *PUPI*** + **'00 08 01 00'** | ◂ **LT** | ATTRIB |
| 7 | **CUT** ▸ | **'00'** sent between $FDT_{B,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | ▸ **LT** | ATTRIB Response |
| 8 | **CUT** ◂ | **$I(0)_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◂ **LT** | Select PPSE / b.n. = 0 |
| 9 | **CUT** ▸ | **$I(0)_0$ ['6F' +** n data bytes + **'90 00']** sent between $FDT_{B,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | ▸ **LT** | b.n. = 0 |
| 10 | **CUT** ◂ | **$I(0)_1$ ['00 A4 04 00'** + Lc + AID + **'00']** | ◂ **LT** | Select AID / b.n. = 1 |
| 11 | **CUT** ▸ | **$I(0)_1$ ['6F' +** n data bytes + **'90 00']** sent between $FDT_{B,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | ▸ **LT** | b.n. = 1 |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of R-APDUs are sent between $FDT_{B,PICC,MIN}$ and FWT = (256 x 16/ $f_c$) x $2^{FWI}$ | | | |
| 12+j | **CUT** ◂ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◂ **LT** | PICC Reset |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 13+j | **CUT** ◀ | '**05 00 08**' sent after $t_P$ | ◀ **LT** | WUPB to poll for the CUT |
| 14+j | **CUT** ▶ | ***ATQB*** *(containing PUPI)* sent between $FDT_{B,PICC,MIN}$ and $FWT_{ATQB}=TR0_{MAX,ATQB}+TR1_{MAX}$ | ▶ **LT** | ATQB |
| 15+j | **CUT** ◀ | '**05 00 08**' sent after $t_P$ | ◀ **LT** | WUPB to poll for the CUT |
| 16+j | **CUT** ▶ | ***ATQB*** *(containing PUPI)* sent between $FDT_{B,PICC,MIN}$ and $FWT_{ATQB}=TR0_{MAX,ATQB}+TR1_{MAX}$ | ▶ **LT** | ATQB |
| End of test: Reset PICC | | | | |

**Scenario 51: Basic Type B Exchange and Timings Measurement**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.2. Basic Type B Exchange with the minimum and longer Frame Delay Times PCD→PICC [CB010.xy]

**Test Codification:**

CB010.xy

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC accepts sequences received with the minimum and longer Frame Delay Time during installation and block protocol.

**Reference Requirements:**

4.8.2.2, 4.8.2.4, 4.8.2.6, 8.1.1.5, 10.1.2.1, 10.3.6.1, 10.3.6.2

**Procedure:**

Run the following scenarios.

In the case of 2 consecutive sequences sent in opposite directions (except the HLTB Response exchanged during the installation procedure), the LT sends all its responses and blocks using the $FDT_{PCD}$ as indicated below:

- For xy=00, 02 and 10: $FDT_{PCD} = FDT_{B,PCD,MIN}$
- For xy=01: $FDT_{PCD}$ = 100 ms

between the beginning of the End of Sequence of the sequence sent by the CUT and the beginning of the sequence replied by the LT.

During the removal procedure, the LT applies a delay of at least $t_P$ between the end of the ATQB sequence sent by the CUT and the beginning of the WUPB Command sequence sent by the LT.

Remark: For x=0, the EMV CL v.2.0 installation procedure is performed. For x=1, a “backward compatibility” installation procedure is performed.

For xy=02:
The LT sends an ATTRIB Command with bit 2 to 3 of Param 3 byte set to $(1)_b$.

For xy=10:
When a sequence that contains a S(DESELECT) sent by the LT is followed by a sequence that contains a S(DESELECT) sent by the CUT, the LT measures the delay between the end of the sequence sent by the LT and the beginning of the sequence replied by the CUT.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT accepts and replies to the sequences sent by the LT with the minimum Frame Delay Time).

All the PUPIs received before PICC reset have the same value.

All the PUPIs received after PICC reset have the same value.

For the PICCs with fixed PUPI: all the PUPIs sent by CUT have the same value during the test as indicated in ICS.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

For xy=10:
On reception of an S(DESELECT) Block, the CUT replies with an S(DESELECT) Block having a PCB field (i.e. first byte of the Block) equal to $(\text{'C2'})_h$ (i.e. $b_2$ of S-Block PCB is set to $(1)_b$).
Between the S(DESELECT) command and the S(DESELECT) Response, the time during which the CUT does not generate a subcarrier just after the end of the sequences sent by the LT is at least $TR0_{MIN}$. The time during which the CUT generates a subcarrier with no phase transition just before the beginning of the sequences sent by the CUT is at least $TR1_{MIN}$ and is at most $TR1_{MAX}$. The delay between the sequence that contains the S(DESELECT) sent by the LT and the sequence that contains the S(DESELECT) replied by the CUT is at most $FWT_{DESELECT}$.

**Pre-processing with chaining:**

If the test script needs to be processed because the CUT sends an I-Block indicating chaining during the transaction, then the LT shall send the R(ACK) Block using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT.

**Pre-processing with S(WTX):**

If the test script needs to be processed because the CUT sends a S(WTX) Request during the transaction, then the LT shall send the S(WTX) Response using the Frame Delay Time $FDT_{PCD}$ between the end of the sequence sent by the CUT and the beginning of the sequence replied by the LT.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 3 | **CUT** ◀ | **'52'** *(Type A short frame)* sent after $t_P$ | ◀ **LT** | WUPA (polling) |
| 4 | **CUT** ◀ | **'05 00 08'** sent after $t_P$ | ◀ **LT** | WUPB to wake up the CUT |
| 5 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 6 | **CUT** ◀ | **For xy=00 to 01:'1D' + *PUPI* + '00 08 01 00'**<br>**For xy=02: '1D' + *PUPI* + '00 08 07 00'**<br>sent using the delay FDT $_{PCD}$ | ◀ **LT** | ATTRIB |
| 7 | **CUT** ▶ | **'00'** | ▶ **LT** | ATTRIB Response |
| 8 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** sent using the delay FDT $_{PCD}$ | ◀ **LT** | Select PPSE / b.n. = 0 |
| 9 | **CUT** ▶ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▶ **LT** | b.n. = 0 |
| 10 | **CUT** ◀ | **I(0)$_1$ ['00 A4 04 00'** + Lc + AID + **'00']** sent using the delay FDT $_{PCD}$ | ◀ **LT** | Select AID / b.n. = 1 |
| 11 | **CUT** ▶ | **I(0)$_1$ ['6F' +** n data bytes + **'90 00']** | ▶ **LT** | b.n. = 1 |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent using the delay FDT $_{PCD}$ | | | |
| 12+j | **CUT** ◀ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◀ **LT** | PICC Reset |
| 13+j | **CUT** ◀ | **'05 00 08'** sent after $t_P$ | ◀ **LT** | WUPB to poll for the CUT |
| 14+j | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 15+j | **CUT** ◀ | **'05 00 08'** sent after $t_P$ | ◀ **LT** | WUPB to poll for the CUT |
| 16+j | **CUT** ▶ | **ATQB** | ▶ **LT** | ATQB |
| End of test: Reset PICC | | | | |

## Scenario 52: Basic Type B Exchange with the minimum and longer Frame Delay Time PCD→PICC (xy=00 to 02)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 3 | **CUT** ◀ | **'50'** + ***PUPI*** sent using the delay FDT $_{PCD}$ | ◀ **LT** | HLTB |
| 4 | **CUT** ▶ | **'00'** | ▶ **LT** | HLTB Response |
| 5 | **CUT** ◀ | **'05 00 00'** sent after $t_P$ | ◀ **LT** | REQB |
| 6 | **CUT** ◀ | **'52'** *(Type A short frame)* sent after $t_P$ | ◀ **LT** | WUPA |
| 7 | **CUT** ◀ | **'05 00 08'** sent after $t_P$ | ◀ **LT** | WUPB to wake up the CUT |
| 8 | **CUT** ▶ | **ATQB** | ▶ **LT** | ATQB |
| 9 | **CUT** ◀ | **'1D' + *PUPI*** + **'00 08 01 00'** sent using the delay FDT $_{PCD}$ | ◀ **LT** | ATTRIB |
| 10 | **CUT** ▶ | **'00'** | ▶ **LT** | ATTRIB Response |
| 11 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** sent using the delay FDT $_{PCD}$ | ◀ **LT** | Select PPSE / b.n. = 0 |
| 12 | **CUT** ▶ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▶ **LT** | b.n. = 0 |
| 13 | **CUT** ◀ | **I(0)$_1$ ['00 A4 04 00'** + Lc + AID + **'00']** sent using the delay FDT $_{PCD}$ | ◀ **LT** | Select AID / b.n. = 1 |
| 14 | **CUT** ▶ | **I(0)$_1$ ['6F' +** n data bytes + **'90 00']** | ▶ **LT** | b.n. = 1 |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent using the delay FDT $_{PCD}$ | | | |
| 15+j | **CUT** ◀ | **S(DESELECT) Request**<br>sent using the Frame Delay Time FDT$_{PCD}$ | ◀ **LT** | DESELECT Request |
| 16+j | **CUT** ▶ | **S(DESELECT) Response** sent between FDT$_{B,PICC,MIN}$ and FWT$_{DESELECT}$ | ◀ **LT** | DESELECT Response |
| 17+j | **CUT** ◀ | **'05 00 08'** sent after $t_P$ | ◀ **LT** | WUPB to poll for the CUT |
| 18+j | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 19+j | **CUT** ◀ | **'05 00 08'** sent after $t_P$ | ◀ **LT** | WUPB to poll for the CUT |
| 20+j | **CUT** ▶ | **ATQB** | ▶ **LT** | ATQB |
| End of test: Reset PICC | | | | |

## Scenario 53: Basic Type B Exchange with the minimum and longer Frame Delay Time PCD→PICC (xy=10)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.3. Basic Type B Exchange with the minimum supported SFGT PCD→PICC [CB015.x]

**Test Codification:**

CB015.x

**ICS Parameters:**

PICC Type = B

PICC supports Extended ATQB (sends optional Extended ATQB byte (Byte 13) in ATQB)

**Test Objective:**

To ensure that the values in the optional Byte 13 of ATQB sent by Type B PICC are as required in the specification when PICC supports Extended ATQB (by sending optional Byte 13 of ATQB).

To ensure that the Type B PICC accepts the first I-Block received with the minimum supported SFGT following ATTRIB Response when PICC supports Extended ATQB (by sending optional Byte 13 of ATQB).

Remark: This test is only applicable to Type B PICCs supporting Extended ATQB by sending optional Byte 13 of ATQB as indicated in ICS.

**Reference Requirements:**

4.8.2.4, 4.8.2.6

**Procedure:**

Run the following scenario.

The LT sends a correct WUPB and REQB Command sequence with Extended ATQB supported (i.e bit 5of PARAM byte set to $(1)_b$):

- For x=0: WUPB Command with Extended ATQB support is sent
- For x=1: REQB Command with Extended ATQB support is sent

On reception of the response to the ATTRIB Command sequence sent by the CUT, the LT sends a Select PPSE command as the first I-Block of the half-duplex block protocol using the delay SFGT between the end of the sequence sent by the CUT (i.e. ATTRIB Command response) and the beginning of the sequence sent by the LT:

- SFGT = $FDTB_{,PCD,MIN}$ if SFGI = 0
- SFGT = $(256 \times 16/ f_c) \times 2^{SFGI}$ if SFGI > 0

**Acceptance Criteria:**

The CUT behaves as described in the following scenario

On reception of the WUPB Command sequence with Extended ATQB supported (x=0) or REQB Command sequence with Extended ATQB supported (x=1), the CUT replies with a correct ATQB containing optional Extended ATQB byte (Byte 13) having the value of $(\text{“XY”})_h$, where:

- X = SFGI where $X \leq (\text{‘8’})_h$
- Y = $(\text{‘0’})_h$

as indicated in ICS.

On reception of the Select PPSE command (i.e. the first I-Block sent by LT), the CUT replies with correct response.

All the PUPIs received have the same value.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

For the PICCs with fixed PUPI: all the PUPIs sent by CUT have the same value during the test as indicated in ICS.

**Scenario:**

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| 1 | **CUT** ◄ | **For x=0: ’05 00 18’**<br>**For x=1: ’05 00 10’** | ◄ **LT** | | WUPB/REQB to poll for the CUT |
| 2 | **CUT** ► | ***ATQB*** *(containing PUPI)* | ► **LT** | | ATQB with Byte 13 |
| 3 | **CUT** ◄ | ’**52**’ *(Type A short frame)* | ◄ **LT** | | WUPA (polling) |
| 4 | **CUT** ◄ | **‘05 00 18’** | ◄ **LT** | | WUPB to wake up the CUT |
| 5 | **CUT** ► | ***ATQB*** *(containing PUPI)* | ► **LT** | | ATQB with Byte 13 |
| 6 | **CUT** ◄ | ’**1D**’ **+** ***PUPI*** + ’**00 08 01 00**’ | ◄ **LT** | | ATTRIB |
| 7 | **CUT** ► | ‘**00**’ | ► **LT** | | ATTRIB Response |
| 8 | **CUT** ◄ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]**<br>sent using the delay SFGT | ◄ **LT** | | Select PPSE / b.n. = 0 |
| 9 | **CUT** ► | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ► **LT** | | b.n. = 0 |
| End of test: Reset PICC | | | | | |

**Scenario 54: Basic Type B Exchange with the minimum supported SFGT PCD→PICC (x=0 and 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.4. Basic Type B Exchange with the minimum and the maximum values of $EGT_{PCD}$ [CB021.x]

**Test Codification:**

CB021.x

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC accepts frames received with the minimum or the maximum character-to-character delay $EGT_{PCD}$ during a basic exchange (installation and block protocol).

**Reference Requirements:**

4.5.1.2, 4.6.2.2a

**Procedure:**

Run the following scenario.

For x = 0:
During the whole test, the LT sends commands and blocks using the minimum character-to-character delay $EGT_{PCD,MIN}$ between two consecutive characters within a frame transmitted to the CUT.

For x = 1:
During the whole test, the LT sends commands and blocks using the maximum character-to-character delay $EGT_{PCD,MAX}$ between two consecutive characters within a frame transmitted to the CUT.
During the whole test, the LT sends commands and blocks using the maximum the delay $EGT_{PCD,MAX}$ between the last character and the EoS within a sequence transmitted to the CUT.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

**Pre-processing with chaining:**

If the test script needs to be processed because the CUT sends an I-Block indicating chaining during the transaction, then the LT shall send the R(ACK) Block using the minimum or the maximum character-to-character delay $EGT_{PCD}$ (depending on the value of ‘x’) between two consecutive characters within the frame transmitted to the CUT.

**Pre-processing with S(WTX):**

If the test script needs to be processed because the CUT sends a S(WTX) Request during the transaction, then the LT shall send the S(WTX) Response using the minimum or the maximum character-to-character delay $EGT_{PCD}$ (depending on the value of ‘x’) between two consecutive characters within the frame transmitted to the CUT.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**05 00 08**' sent with $EGT_{PCD}$ in function of 'x' | ◀ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 3 | **CUT** ◀ | '**1D**' **+** ***PUPI*** + '**00 08 01 00**'<br>sent with EGTPCD in function of 'x' | ◀ **LT** | ATTRIB |
| 4 | **CUT** ▶ | '**00**' | ▶ **LT** | ATTRIB Response |
| 5 | **CUT** ◀ | I(0)0 ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']<br>sent with EGTPCD in function of 'x' | ◀ **LT** | Select PPSE |
| 6 | **CUT** ▶ | **$I(0)_0$ ['6F' +** n data bytes + '**90 00'**] | ▶ **LT** | |
| 7 | **CUT** ◀ | **$I(0)_1$ ['00 A4 04 00'** + Lc + AID + '**00'**]<br>sent with EGTPCD in function of 'x' | ◀ **LT** | Select AID |
| 8 | **CUT** ▶ | **$I(0)_1$ ['6F' +** n data bytes + '**90 00'**] | ▶ **LT** | |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent with $EGT_{PCD}$ in function of 'x' | | | |
| 9+j | **CUT** ◀ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time tRESET) | ◀ **LT** | PICC Reset |
| 10+j | **CUT** ◀ | '05 00 08' sent after tP with EGTPCD in function of 'x' | ◀ **LT** | WUPB to poll for the CUT |
| 11+j | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| End of test: Reset PICC | | | | |

## Scenario 55: Basic Type B Exchange with the minimum and the maximum value of $EGT_{PCD}$ (x=0 to 1)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.5. Basic Type B Exchange with minimum and maximum durations of (SoS) and (EoS) [CB025.x]

**Test Codification:**

CB025.x

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC accepts sequences received with the minimum or the maximum durations of the Start of Sequence (SoS) and End of Sequence (EoS) sequences during a basic exchange (installation and block protocol).

**Reference Requirements:**

4.3.22, 4.6.2.2

**Procedure:**

Run the following scenario.

For x = 0:
During the whole test, the LT sends its sequences with (SoS) and (EoS) having the following formats:
Start of Sequence (SoS) = 1272/ $f_c$ of logical state $(0)_b$ followed by 240/ $f_c$ of logical state $(1)_b$.
End of Sequence (EoS) = 1272/ $f_c$ of logical state $(0)_b$ followed by a logical state transition.

For x = 1:
During the whole test, the LT sends its sequences with (SoS) and (EoS) having the following formats:
Start of Sequence (SoS) = 1424/ $f_c$ of logical state $(0)_b$ followed by 400/ $f_c$ of logical state $(1)_b$.
End of Sequence (EoS) = 1424/ $f_c$ of logical state $(0)_b$ followed by a logical state transition.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

**Pre-processing with chaining:**

If the test script needs to be processed because the CUT sends an I-Block indicating chaining during the transaction, then the LT shall send the R(ACK) Block using the (SoS) and (EoS) formats described above (depending on the value of ‘x’) within the sequence transmitted to the CUT.

**Pre-processing with S(WTX):**

If the test script needs to be processed because the CUT sends a S(WTX) Request during the transaction, then the LT shall send the S(WTX) Response using the (SoS) and (EoS) formats described above (depending on the value of ‘x’) within the sequence transmitted to the CUT.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**05 00 08**' sent with (SoS) and (EoS) in function of 'x' | ◀ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 3 | **CUT** ◀ | '**1D**' **+** ***PUPI*** + '**00 08 01 00**' sent with (SoS) and (EoS) in function of 'x' | ◀ **LT** | ATTRIB |
| 4 | **CUT** ▶ | '**00**' | ▶ **LT** | ATTRIB Response |
| 5 | **CUT** ◀ | **I(0)0 [**'**00 A4 04 00 0E**' + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00**'**]** sent with (SoS) and (EoS) in function of 'x' | ◀ **LT** | Select PPSE |
| 6 | **CUT** ▶ | **I(0)$_0$ [**'**6F**' **+** n data bytes + '**90 00**'**]** | ▶ **LT** | |
| 7 | **CUT** ◀ | **I(0)$_1$ [**'**00 A4 04 00**' + Lc + AID + '**00**'**]** sent with (SoS) and (EoS) in function of 'x' | ◀ **LT** | Select AID |
| 8 | **CUT** ▶ | **I(0)$_1$ [**'**6F**' **+** n data bytes + '**90 00**'**]** | ▶ **LT** | |
| | **Continue C/R – APDU exchanges until transaction is completed**<br>I(0) blocks of C-APDUs are sent with (SoS) and (EoS) in function of 'x' | | | |
| 9+j | **CUT** ◀ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time tRESET) | ◀ **LT** | PICC Reset |
| 10+j | **CUT** ◀ | '05 00 08' sent after tP with (SoS) and (EoS) in function of 'x' | ◀ **LT** | WUPB to poll for the CUT |
| 11+j | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| End of test: Reset PICC | | | | |

## Scenario 56: Basic Type B Exchange with the minimum and the maximum durations of the (SoS) and (EoS) sequences (x=0 to 1)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.6. Type B Correct Installation with respect of the anticollision state machine [CB116]

**Test Codification:**

CB116

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC stays in the READY state and sends its ATQB when it receives a valid WUPB Command in the READY state.

**Reference Requirements:**

8.2.2.1, 8.2.3.1, 8.2.3.3, 8.2.3.4, 8.2.5.1

**Procedure:**

Run the following scenario.

On reception of the first ATQB sent by the CUT, the LT sends the WUPB Command a second time.

On reception of the second ATQB sent by the CUT, the LT ensure that it is the same as the first received ATQB and the LT sends the HLTB Command (using the PUPI of the CUT).

On reception of the HLTB Response sent by the CUT, the LT sends the WUPB Command a third time.

On reception of the third ATQB sent by the CUT, the LT sends the WUPB Command a fourth time.

On reception of the fourth ATQB sent by the CUT, the LT sends the ATTRIB Command (using the PUPI of the CUT).

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT replies to all the WUPB Commands with the same ATQB and sends the ATTRIB Response on reception of the ATTRIB Command).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ **'05 00 08'** ◄ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ► ***ATQB*** *(containing PUPI)* ► **LT** | ATQB |
| 3 | **CUT** ◄ **'05 00 08'** ◄ **LT** | WUPB |
| 4 | **CUT** ► ***ATQB*** *(same as Step 2)* ► **LT** | ATQB |
| 5 | **CUT** ◄ **'50'** + ***PUPI*** ◄ **LT** | HLTB |
| 6 | **CUT** ► **'00'** ► **LT** | HLTB Response |
| 7 | **CUT** ◄ **'05 00 08'** ◄ **LT** | WUPB |
| 8 | **CUT** ► ***ATQB*** *(same as Step 2)* ► **LT** | ATQB |
| 9 | **CUT** ◄ **'05 00 08'** ◄ **LT** | WUPB |
| 10 | **CUT** ► ***ATQB*** *(same as Step 2)* ► **LT** | ATQB |
| 11 | **CUT** ◄ **'1D' + *PUPI*** + **'00 08 01 00'** ◄ **LT** | ATTRIB |
| 12 | **CUT** ► **'00'** ► **LT** | ATTRIB Response |
| End of test: Reset PICC | | |

**Scenario 57: Type B Correct Installation with respect of the anticollision state machine**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.7. Type B Consecutive Installations (fixed PUPI) [CB126.x]

**Test Codification:**

CB126.x

**ICS Parameters:**

PICC Type = B / fixed PUPI

**Test Objective:**

To ensure that the PUPI of the PICC is fixed, when indicated as fixed in the ICS.

**Reference Requirements:**

6.3.2.1

**Procedure:**

Run the following scenario.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

All PUPIs sent by the CUT have the same value.

The PUPI sent by the CUT is the one expected as indicated in the ICS.

Command frame:

- For x=0: WUPB
- For x=1: REQB

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◄ | **'05 00 08'** | ◄ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ► | ***ATQB*** *(containing PUPI)* | ► **LT** | ATQB |
| 3 | **CUT** ◄ | **'1D' + *PUPI*** + **'00 08 01 00'** | ◄ **LT** | ATTRIB |
| 4 | **CUT** ► | **'00'** | ► **LT** | ATTRIB Response |
| 5 | **CUT** ◄ | **$I(0)_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◄ **LT** | Select PPSE / b.n. = 0 |
| 6 | **CUT** ► | **$I(0)_0$ ['6F' +** n data bytes + **'90 00']** | ► **LT** | b.n. = 0 |
| 7 | **CUT** ◄ | **S(DESELECT) Request** | ◄ **LT** | DESELECT Request |
| 8 | **CUT** ► | **S(DESELECT) Response** | ◄ **LT** | DESELECT Response |
| 9 | **CUT** ◄ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◄ **LT** | PICC Reset |
| 10 | **CUT** ◄ | **For x=0**: '05 00 08'<br>**For x=1:** '05 00 00' | ◄ **LT** | WUPB for x=0<br>REQB for x=1 |
| 11 | **CUT** ► | ***ATQB*** *(containing PUPI)* | ► **LT** | ATQB |
| 12 | **CUT** ◄ | **'1D' + *PUPI*** + **'00 08 01 00'** | ◄ **LT** | ATTRIB |
| 13 | **CUT** ► | **'00'** | ► **LT** | ATTRIB Response |
| 14 | **CUT** ◄ | **$I(0)_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◄ **LT** | Select PPSE / b.n. = 0 |
| 15 | **CUT** ► | **$I(0)_0$ ['6F' +** n data bytes + **'90 00']** | ► **LT** | b.n. = 0 |
| End of test: Reset PICC | | | | |

**Scenario 58: Type B Consecutive Installations (fixed PUPI, x=0 to 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.8. Type B Handling of the REQB Command [CB130.x]

**Test Codification:**

CB130.x

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC does not send an ATQB in response to a REQB Command when it is not in the IDLE or the READY.

**Reference Requirements:**

8.2.3.3, 8.2.4.2

**Procedure:**

Run the following scenario.

The LT sends a REQB Command to the CUT:

- x=0: after an ATQB Response (CUT in the READY State)
- x=1: after an ATTRIB Response (CUT in ACTIVE State)

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

For x=0: the CUT sends an ATQB in response to the first and second REQB Commands.

For x=1: On reception of the REQB Command, the CUT stays mute (i.e. does not send any frame during Mute_Time). The CUT sends an I-Block in response to the I-Block sent by the LT, following the REQB Command.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ **'05 00 08'** ◄ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ► ***ATQB*** *(containing PUPI)* ► **LT** | ATQB |
| 3 | **CUT** ◄ **'05 00 00'** sent after $t_P$ ◄ **LT** | REQB |
| 4 | **CUT** ► ***ATQB*** *(containing PUPI)* ► **LT** | ATQB |
| 5 | **CUT** ◄ **'05 00 00'** sent after $t_P$ ◄ **LT** | REQB |
| 6 | **CUT** ► ***ATQB*** *(containing PUPI)* ► **LT** | ATQB |
| End of test: Reset PICC | | |

**Scenario 59: Type B Handling of the REQB Command (x=0)**

| Step | Exchanges | Comments |
|---|---|---|
| 1 | **CUT** ◄ **'05 00 08'** ◄ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ► ***ATQB*** *(containing PUPI)* ► **LT** | ATQB |
| 3 | **CUT** ◄ **'1D' + *PUPI*** + **'00 08 01 00'** ◄ **LT** | ATTRIB |
| 4 | **CUT** ► **'00'** ► **LT** | ATTRIB Response |
| 5 | **CUT** ◄ **'05 00 00'** ◄ **LT** | REQB |
| | **CUT** II The CUT does not send any frame during Mute_Time ► **LT** | |
| 6 | **CUT** ◄ **$I(0)_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** ◄ **LT** | Select PPSE |
| 7 | **CUT** ► **$I(0)_0$ ['6F'** + n data bytes + **'90 00']** ► **LT** | |
| End of test: Reset PICC | | |

**Scenario 60: Type B Handling of the REQB Command (x=1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.9. Type B Correct Installation with C-APDU sent in the ATTRIB Command Higher Layer INF field [CB150]

**Test Codification:**

CB150

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC accepts an ATTRIB Command with a Higher Layer INF field.

**Reference Requirements:**

6.4.1.13

**Procedure:**

Run the following scenario.

The LT sends an ATTRIB Command with a Higher Layer INF field and observes the value of the ATTRIB Response replied by the CUT.

In this test the Higher Layer INF field of the ATTRIB Command contains the unknown C-APDU = (“FF FF FF FF FF”)$_h$.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (on reception of the ATTRIB Command with Higher Layer INF field sent by the LT, the CUT sends the ATTRIB Response having the value (‘00’)$_h$).

**Optional accepted behavior:**

On reception of the ATTRIB Command with Higher Layer INF field sent by the LT, the CUT sends the ATTRIB Response with Higher Layer Response field. The ATTRIB Response starts with the byte (‘00’)$_h$ followed by some data bytes coding the R-APDU replied to the C-APDU contained in the ATTRIB Command Higher Layer INF field.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 3 | **CUT** ◀ | **'1D' + *PUPI*** + **'00 08 01 00'** + **'FF FF FF FF FF'** | ◀ **LT** | ATTRIB with Higher Layer INF field |
| 4 | **CUT** ▶ | **'00'**<br>*(the CUT may also return an ATTRIB Response with Higher Layer Response field)* | ▶ **LT** | ATTRIB Response |
| 5 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◀ **LT** | Select PPSE |
| 6 | **CUT** ▶ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▶ **LT** | |
| End of test: Reset PICC | | | | |

**Scenario 61: Type B Correct Installation with C-APDU sent in the ATTRIB Command Higher Layer INF field**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.10. Type B Correct Installation with RFU values or values to be ignored [CB155.x]

**Test Codification:**

CB155.x

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC accepts installation commands with RFU bits set or values to be ignored.

**Reference Requirements:**

1.7.4.1, 1.7.4.2

**Procedure:**

Run the following scenario.

Send the WUPB and ATTRIB Commands with the RFU values set to $(1)_b$ in a function of x as indicated below:

- For x=0: WUPB Command with bit 6 to 8 of PARAM byte set to $(1)_b$
- For x=1: REQB Command with bit 6 to 8 of PARAM byte set to $(1)_b$
- For x=2: ATTRIB Command with bit 1 and 2 of Param 1 byte set to $(1)_b$
- For x=7: ATTRIB Command with bit 5 to 8 of Param 4 byte set to (1)b
- For x=8: ATTRIB Command with bit 2 to 3 of Param 3 byte set to (1)b

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (on reception of the WUPB and ATTRIB Commands with RFU bits set).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | **For x=0: ’05 00 E8’**<br>**For x=1: ’05 00 E0’**<br>**For x=2 and 7 to 8: ’05 00 08’** | ◀ **LT** | WUPB/REQB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 3 | **CUT** ◀ | **ATTRIB Command in function of ‘x’:**<br>**For x=0 and 1: ’1D’** + ***PUPI*** + **’00 08 01 00’**<br>**For x=2: ’1D’** + ***PUPI*** + **’03 08 01 00’**<br>**For x=7: ’1D’** + ***PUPI*** **+ ’00 08 01 F0’**<br>**For x=8: ’1D’** + ***PUPI*** + **’00 08 07 00’** | ◀ **LT** | ATTRIB |
| 4 | **CUT** ▶ | **‘00’** | ▶ **LT** | ATTRIB Response |
| 5 | **CUT** ◀ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** | ◀ **LT** | Select PPSE |
| 6 | **CUT** ▶ | **I(0)$_0$ [‘6F’** + n data bytes + **’90 00’]** | ▶ **LT** | |
| End of test: Reset PICC | | | | |

**Scenario 62: Type B Correct Installation with RFU values (x=0 to 2 and 7 to 8).**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.11. Type B Installation with error in the IDLE state [CB204.x]

**Test Codification:**

CB204.x

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC behaves correctly when it receives an erroneous frame in the IDLE state (i.e. during the polling) and replies to the WUPB or REQB Commands when in the IDLE state.

**Reference Requirements:**

6.1.1.2, 8.1.1.2, 8.1.1.3, 8.1.1.4, 8.2.2.3

**Procedure:**

Run the following scenario.

Instead of the polling commands, the LT sends consecutive erroneous frames.

After each erroneous frame, the LT stays in reception mode during Mute_Time.

After having sent all the erroneous frames, the LT sends a correct WUPB or REQB Command frame.

- For x=0 the WUPB without Extended ATQB support is sent
- For x=1 the WUPB with Extended ATQB support is sent
- For x=2 the REQB without Extended ATQB support is sent
- For x=3 the REQB with Extended ATQB support is sent

**Acceptance Criteria:**

The CUT behaves as described in the following scenario (the CUT accepts and replies to the sequences sent by the LT).

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPB or REQB Command frame, the CUT replies with a correct ATQB.

For fixed PUPI PICCs: The PUPI found in the ATQB is the one expected as indicated in ICS.

For x=0 and x=2: 13$^{th}$ byte in ATQB (4$^{th}$ byte in protocol info) shall not be sent.

Remark: For x=1 and x=3: 13$^{th}$ byte in ATQB (4th byte in Protocol Info) is allowed.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| 1 | **CUT** ◀ | '**05 00 08**' with a wrong least significant bit in the first byte (CRC_B1) of CRC_B | ◀ **LT** | | WUPB with transmission error |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 2 | **CUT** ◀ | '**05 00 08**' with a wrong least significant bit in the second byte (CRC_B2) of CRC_B | ◀ **LT** | | WUPB with transmission error |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 3 | **CUT** ◀ | '**05 00 08**' with a wrong most significant bit in the second byte (CRC_B2) of CRC_B | ◀ **LT** | | WUPB with transmission error |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 4 | **CUT** ◀ | '**50**' + ***PUPI***<br>*(if the CUT supports dynamic PUPI, PUPI is sent as '00 00 00 00' in this step)* | ◀ **LT** | | HLTB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 5 | **CUT** ◀ | '**1D**' **+** ***PUPI*** + '**00 08 01 00**'<br>*(if the CUT supports dynamic PUPI, PUPI is sent as '00 00 00 00' in this step)* | ◀ **LT** | | ATTRIB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 6 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◀ **LT** | | Select PPSE / b.n. = 0 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 7 | **CUT** ◀ | **R(NAK)$_0$** | ◀ **LT** | | Error Notification |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 8 | **CUT** ◀ | **R(ACK)$_0$** | ◀ LT | | Error Notification |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 9 | **CUT** ◀ | **S(WTX)** Response with WTXM **= 1** | ◀ LT | | WTX Response |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 10 | **CUT** ◀ | **S(DESELECT)** | ◀ LT | | S(DESELECT) |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 11 | **CUT** ◀ | '**52**' *(Type A short frame)*<br>REMOVE THIS STEP if PICC supports Type A and Type B | ◀ **LT** | | WUPA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| 12 | **CUT** ◀ | **For x=0: ’05 00 08’**<br>**For x=1: ’05 00 18’**<br>**For x=2: ’05 00 00’**<br>**For x=3: ’05 00 10’** | ◀ **LT** | | WUPB/REQB |
| 13 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | | ATQB |
| 14 | **CUT** ◀ | **’1D’ + *PUPI*** + **’00 08 01 00’** | ◀ **LT** | | ATTRIB |
| 15 | **CUT** ▶ | **‘00’** | ▶ **LT** | | ATTRIB Response |
| 16 | **CUT** ◀ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** | ◀ **LT** | | Select PPSE / b.n. = 0 |
| 17 | **CUT** ▶ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▶ **LT** | | b.n. = 0 |
| End of test: Reset PICC | | | | | |

**Scenario 63: Type B Installation with error in the IDLE state (x=0 to 3)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.12. Type B Installation with error in the READY state [CB206.xy]

**Test Codification:**

CB206.xy

**ICS Parameters:**

PICC Type = B

xy=17 is Not Applicable if the PICC implements a legacy behaviors by accepting an ATTRIB Command with $b_8$-$b_5$ of Param 3 different from $(0000)_b$ (see ICS).

**Test Objective:**

To ensure that the Type B PICC behaves correctly when it receives an erroneous frame in the READY state (i.e. after the ATQB).

**Reference Requirements:**

6.1.1.2, 6.4.1.9, 8.1.1.2, 8.1.1.3, 8.1.1.4, 8.2.3.3, 8.2.3.6

**Procedure:**

Run the following scenario.

On reception of the first ATQB, the LT sends an erroneous frame.

After having sent the erroneous frame, the LT stays in reception mode during Mute_Time before sending a correct ATTRIB Command frame (xy=00 to 05, 07 to 11 and 14 to 17) or a correct WUPB Command frame (xy=18).

The following errors are generated by the LT:

- xy=00: HLTB Command frame with a wrong least significant bit in the first byte (CRC_B1) of CRC_B
- xy=01: HLTB Command frame with a wrong least significant bit in the second byte (CRC_B2) of CRC_B
- xy=02: HLTB Command frame with a wrong most significant bit in the second byte (CRC_B2) of CRC_B
- xy=03: ATTRIB Command frame with a wrong least significant bit in the first byte (CRC_B1) of CRC_B
- xy=04: ATTRIB Command frame with a wrong least significant bit in the second byte (CRC_B2) of CRC_B
- xy=05: ATTRIB Command frame with a wrong most significant bit in the second byte (CRC_B2) of CRC_B
- xy=07: I-Block frame
- xy=08: HLTB Command frame with wrong value of PUPI
- xy=09: ATTRIB Command frame with wrong value of PUPI
- xy=10: ATTRIB Command frame with the m.s. nibble of the byte Param 2 different from ‘0’
- xy=11: ATTRIB Command frame with byte Param 3 equal to ‘00’
- xy=14: R(ACK)1
- xy=15: R(NAK)0
- xy=16: S(WTX) Response
- xy=17: ATTRIB Command frame with byte Param 3 equal to ‘F1’ (see “ICS parameters”)
- xy=18: WUPB with AFI=’90’ (RFU value – unmatched AFI)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

For xy=00 to 05, 07 to 11 and 14 to 17, on reception of the correct ATTRIB Command frame, the CUT replies with a correct ATTRIB Response.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◂ | **'05 00 08'** | ◂ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▸ | ***ATQB*** *(containing PUPI)* | ▸ **LT** | ATQB |
| 3 | **CUT** ◂ | **Erroneous frame in function of 'xy'**:<br>**For xy=00 to 02**: '**50**' + ***PUPI*** **with a transmission error**<br>**For xy=03 to 05**: '**1D**' + ***PUPI*** + '**00 08 01 00**' **with a transmission error**<br>**For xy=07**: **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'] *(SELECT PPSE)***<br>**For xy=08**: '**50**' + (4 bytes ≠ *PUPI*)<br>**For xy=09**: '**1D**' + (4 bytes ≠ *PUPI*) + '**00 08 01 00**'<br>**For xy=10**: '**1D**' **+ *PUPI*** + '**00 F8 01 00**'<br>**For xy=11: '1D' + PUPI + '00 08 00 00'**<br>**For xy=14: R(ACK)$_1$**<br>**For xy=15: R(NAK)$_0$**<br>**For xy=16: S(WTX) Response with WTXM=1**<br>**For xy=17: '1D' + *PUPI* + '00 08 F1 00' (see "ICS parameters")**<br>**For xy=18: '05 90 08'** | ◂ **LT** | erroneous frame |
| | **CUT** ‖ | **The CUT does not send any frame during Mute_Time** | ▸ **LT** | |
| 4 | **CUT** ◂ | **For xy=00 to 11 and 14 to 17**: '**1D**' **+ *PUPI* +** '**00 08 01 00**'<br>For xy= 18**: '**05 00 08**'** | ◂ **LT** | ATTRIB or WUPB |
| 5 | **CUT** ▸ | **For xy=00 to 11 and 14 to 17**: '**00**'<br>**For xy= 18**: ***ATQB*** *(containing PUPI)* | ▸ **LT** | ATTRIB Response or ATQB |
| End of test: Reset PICC | | | | |

**Scenario 64: Type B Installation with error in the READY state (x=00 to 05, 07 to 11 and 14 to 18)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.13. Type B Installation with error in the HALT state following the READY state [CB226.x]

**Test Codification:**

CB226.x

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC behaves correctly when it receives an erroneous frame in the HALT state (i.e. after the HLTB Response) following the READY State.

**Reference Requirements:**

6.1.1.2, 8.1.1.2, 8.1.1.3, 8.1.1.4, 8.2.5.2

**Procedure:**

Run the following scenario.

On reception of the HLTB Response sent by the CUT (i.e. when the CUT is in the HALT state), the LT sends consecutive erroneous frames.

After each erroneous frame, the LT stays in reception mode during Mute_Time.

After having sent all the erroneous frames, the LT sends a correct WUPB Command frame.

For x=0: the WUPB without Extended ATQB support is sent.

For x=1: the WUPB with Extended ATQB support is sent.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPB Command frame, the CUT replies with a correct ATQB.

The PUPIs received at the steps 2 and 17 have the same value.

For fixed PUPI PICCs: The PUPI found in the ATQBs is the one expected as indicated in ICS.

For x=0:

- 13$^{th}$ byte in ATQB (4$^{th}$ byte in protocol info) is not sent.
- All ATQBs have the same value.

Remark: For x=1:13$^{th}$ byte in ATQB (4$^{th}$ byte in protocol info) is allowed.

ATQB2 = ATQB1 + 13$^{th}$ byte (4$^{th}$ byte in protocol info).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## Scenario:

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB1*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 3 | **CUT** ◀ | **'50'** + ***PUPI*** | ◀ **LT** | HLTB |
| 4 | **CUT** ▶ | **'00'** | ▶ **LT** | HLTB Response |
| 5 | **CUT** ◀ | **'05 00 08'** with a wrong least significant bit in the first byte (CRC_B1) of CRC_B | ◀ **LT** | WUPB with transmission error |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 6 | **CUT** ◀ | **'05 00 08'** with a wrong least significant bit in the second byte (CRC_B2) of CRC_B | ◀ **LT** | WUPB with transmission error |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 7 | **CUT** ◀ | **'05 00 08'** with a wrong most significant bit in the second byte (CRC_B2) of CRC_B | ◀ **LT** | WUPB with transmission error |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 8 | **CUT** ◀ | **'50'** + ***PUPI*** | ◀ **LT** | HLTB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 9 | **CUT** ◀ | **'1D' + *PUPI* + '00 08 01 00'** | ◀ **LT** | ATTRIB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 10 | **CUT** ◀ | **'05 00 00'** | ◀ **LT** | REQB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 11 | **CUT** ◀ | **'05 00 10'** | ◀ **LT** | REQB with extended ATQB support |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 12 | **CUT** ◀ | **$I(0)_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◀ **LT** | Select PPSE / b.n. = 0 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 13 | **CUT** ◀ | **$R(NAK)_0$** | ◀ **LT** | Error Notification |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 14 | **CUT** ◀ | **S(WTX) Response with WTXM = 1** | ◀ **LT** | WTX Response |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 15 | **CUT** ◀ | **'52'** *(Type A short frame))* | ◀ **LT** | WUPA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 16 | **CUT** ◂ | **For x=0: '05 00 08'**<br>**For x=1: '05 00 18'** | ◂ **LT** | WUPB |
| 17 | **CUT** ▸ | **For x=0*: ATQB1*** *(containing PUPI)*<br>**For x=1: *ATQB1 or ATQB2*** *(containing PUPI)* | ▸ **LT** | ATQB |
| End of test: Reset PICC | | | | |

**Scenario 65: Type B Installation with error in the HALT state (x=0 and 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.14. Type B Installation with error in the HALT state following the ACTIVE State [CB227.x]

**Test Codification:**

CB227.x

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC behaves correctly when it receives an erroneous frame in the HALT state (i.e. after the HLTB Response) following the ACTIVE State.

**Reference Requirements:**

6.1.1.2, 8.1.1.2, 8.1.1.3, 8.1.1.4, 8.2.5.2

**Procedure:**

Run the following scenario.

Following the ATTRIB response from the Type B PICC, the LT sends a correct S(DESELECT) Request frame and receives S(DESELECT) Response,

On reception of the S(DESELECT) Response sent by the CUT (i.e. when the CUT is in the HALT state), the LT sends consecutive erroneous frames.

After each erroneous frame, the LT stays in reception mode during Mute_Time.

After having sent all the erroneous frames, the LT sends a correct WUPB Command frame.

For x=0: the WUPB without Extended ATQB support is sent.

For x=1: the WUPB with Extended ATQB support is sent.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the last WUPB Command frame, the CUT replies with a correct ATQB.

The PUPIs received at the step 2 and 19 have the same value.

For fixed PUPI PICCs: The PUPI found in the ATQBs is the one expected as indicated in ICS.

For x=0:

- 13$^{th}$ byte in ATQB (4$^{th}$ byte in protocol info) is not sent
- All ATQBs have the same value

Remark:
For x=1:13$^{th}$ byte in ATQB (4$^{th}$ byte in protocol info) is allowed.
ATQB2 = ATQB1 + 13$^{th}$ byte (4$^{th}$ byte in protocol info).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| 1 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | | WUPB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB1*** *(containing PUPI)* | ▶ **LT** | | ATQB |
| 3 | **CUT** ◀ | **'1D' + *PUPI*** + **'00 08 01 00'** | ◀ **LT** | | ATTRIB |
| 4 | **CUT** ▶ | **'00'** | ▶ **LT** | | ATTRIB Response |
| 5 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◀ **LT** | | Select PPSE / b.n. = 0 |
| 6 | **CUT** ▶ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▶ **LT** | | b.n. = 0 |
| 7 | **CUT** ◀ | **S(DESELECT) Request** | ◀ **LT** | | DESELECT Request |
| 8 | **CUT** ▶ | **S(DESELECT) Response** | ◀ **LT** | | DESELECT Response |
| 9 | **CUT** ◀ | **'05 00 08'** with a wrong least significant bit in the first byte (CRC_B1) of CRC_B | ◀ **LT** | | WUPB with transmission error |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 10 | **CUT** ◀ | **'05 00 08'** with a wrong least significant bit in the second byte (CRC_B2) of CRC_B | ◀ **LT** | | WUPB with transmission error |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 11 | **CUT** ◀ | **'05 00 08'** with a wrong most significant bit in the second byte (CRC_B2) of CRC_B | ◀ **LT** | | WUPB with transmission error |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 12 | **CUT** ◀ | **'50'** + ***PUPI*** | ◀ **LT** | | HLTB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 13 | **CUT** ◀ | **'1D' + *PUPI*** + **'00 08 01 00'** | ◀ **LT** | | ATTRIB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 14 | **CUT** ◀ | **'05 00 00'** | ◀ **LT** | | REQB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 15 | **CUT** ◀ | **'05 00 10'** | ◀ **LT** | | REQB with extended ATQB support |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 16 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◀ **LT** | | Select PPSE / b.n. = 0 |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |
| 17 | **CUT** ◀ | **R(NAK)$_0$** | ◀ **LT** | | Error Notification |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | | |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 18 | **CUT** ◀ | **S(WTX) Response with WTXM = 1** | ◀ **LT** | WTX Response |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 19 | **CUT** ◀ | '**52**' *(Type A short frame))* | ◀ **LT** | WUPA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 20 | **CUT** ◀ | **For x=0: '05 00 08'**<br>**For x=1: '05 00 18'** | ◀ **LT** | WUPB |
| 21 | **CUT** ▶ | **For x=0*: ATQB1*** *(containing PUPI)*<br>**For x=1: *ATQB1 or ATQB2*** *(containing PUPI)* | ▶ **LT** | ATQB |
| End of test: Reset PICC | | | | |

**Scenario 66: Type B Installation with error in the HALT state (x=0 and 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.15. Type B Installation with WUPA command in the HALT state [CB228.x]

**Test Codification:**

CB228.x

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC with fixed PUPI remains in the HALT or returns to the IDLE state after it has received a WUPA Command.

To ensure that the Type B PICC with random PUPI stays in the HALT state after it has received a WUPA Command.

**Reference Requirements:**

8.1.1.3, 8.1.1.4, 8.2.5.4

**Procedure:**

Run the following scenarios (in function of x).

x = 0 for PUPI is a fixed number:
On reception of the first HLTB Response sent by the CUT (i.e. when the CUT is in the HALT state), the LT sends a WUPA Command followed by a WUPB Command.
On reception of the second HLTB Response sent by the CUT (i.e. when the CUT is in the HALT state), the LT sends a WUPA Command followed by an ATTRIB Command and a WUPB Command.

x = 1 for PUPI is not a fixed number (random PUPI):
On reception of the first HLTB Response sent by the CUT (i.e. when the CUT is in the HALT state), the LT sends a WUPA Command followed by a WUPB Command.
On reception of the second HLTB Response sent by the CUT (i.e. when the CUT is in the HALT state), the LT sends a WUPA Command followed by an REQB Command and a WUPB Command.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

x = 0 for PUPI is a fixed number:
On reception of the WUPA and ATTRIB Commands, the CUT stays mute (i.e. does not send any frame during Mute_Time).
On reception of each WUPB Command, the CUT replies with a correct ATQB.
The PUPI is the same in all the ATQB Responses.

x = 1 for PUPI is not a fixed number (random PUPI):
On reception of the WUPA and REQB Commands, the CUT stays mute (i.e. does not send any frame during Mute_Time).
On reception of each WUPB Command, the CUT replies with a correct ATQB.
The PUPI is the same in all the ATQB Responses.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**05 00 08**' | ◀ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 3 | **CUT** ◀ | '**50**' + ***PUPI*** | ◀ **LT** | HLTB |
| 4 | **CUT** ▶ | '**00**' | ▶ **LT** | HLTB Response |
| 5 | **CUT** ◀ | '**52**' *(Type A short frame)* | ◀ **LT** | WUPA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 6 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | WUPB |
| 7 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 8 | **CUT** ◀ | '**50**' + ***PUPI*** | ◀ **LT** | HLTB |
| 9 | **CUT** ▶ | '**00**' | ▶ **LT** | HLTB Response |
| 10 | **CUT** ◀ | '**52**' *(Type A short frame)* | ◀ **LT** | WUPA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 11 | **CUT** ◀ | '**1D**' **+** ***PUPI*** + '**00 08 01 00**' | ◀ **LT** | ATTRIB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 12 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | WUPB |
| 13 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| End of test: Reset PICC | | | | |

## Scenario 67: Type B Installation with WUPA command in the HALT state (PUPI is a fixed number / x=0)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 3 | **CUT** ◀ | **'50'** + ***PUPI*** | ◀ **LT** | HLTB |
| 4 | **CUT** ▶ | **'00'** | ▶ **LT** | HLTB Response |
| 5 | **CUT** ◀ | **'52'** *(Type A short frame)* | ◀ **LT** | WUPA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 6 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | WUPB |
| 7 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| 8 | **CUT** ◀ | **'50'** + ***PUPI*** | ◀ **LT** | HLTB |
| 9 | **CUT** ▶ | **'00'** | ▶ **LT** | HLTB Response |
| 10 | **CUT** ◀ | **'52'** *(Type A short frame)* | ◀ **LT** | WUPA |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 11 | **CUT** ◀ | **'05 00 00'** | ◀ **LT** | REQB |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 12 | **CUT** ◀ | **'05 00 08'** | ◀ **LT** | WUPB |
| 13 | **CUT** ▶ | ***ATQB*** *(containing PUPI)* | ▶ **LT** | ATQB |
| End of test: Reset PICC | | | | |

## Scenario 68: Type B Installation with WUPA command in the HALT state (PUPI is not a fixed number (random PUPI) / x=1)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 4.16. Type B Installation with polling and with PICC Reset [CB233.xy]

**Test Codification:**

CB233.xy

**ICS Parameters:**

PICC Type = B

**Test Objective:**

To ensure that the Type B PICC supports the minimum timing with carrier on before a WUPB Command (i.e. $t_P$).

To ensure that the Type B PICC goes back to the IDLE state when the PCD performs a PICC Reset during installation.

**Reference Requirements:**

3.2.7.1, 8.2.2.1, 8.1.1.4, 8.1.1.5

**Procedure:**

Run the following scenario.

For PICC supporting only Type B:
Following the initial delay of at least $t_P$ (see above), the LT sends a WUPA Command followed by a WUPB or REQB Command sent using the delay $t_X$ (for y=0: $t_X$ = 5ms and for y=1: $t_X$ = 10ms) between the end of the WUPA Command sequence and the beginning of the WUPB or REQB Command sequence.

For PICC supporting both Type A and Type B:
Following an initial delay equal to $t_P$ with carrier on the LT sends a WUPB Command sequence (xy=0y and 1y) or REQB Command sequence (xy=2y and 3y).

On reception of the second ATQB, the LT performs a PICC Reset i.e. stops sending the carrier (no RF field $H_{OV}$) for a time $t_{RESET}$. Then the LT sends the carrier for a time $t_P$ before sending a WUPB or REQB Command sequence.
For xy=0y: the WUPB without Extended ATQB support is sent
For xy=1y: the WUPB with Extended ATQB support is sent
For xy=2y: the REQB without Extended ATQB support is sent
For xy=3y: the REQB with Extended ATQB support is sent

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of the last WUPB or REQB Command, the CUT sends the ATQB.

For fixed PUPI PICCs: The PUPI found in the ATQBs is the one expected as indicated in ICS.

For xy=0y and xy=2y: 13$^{th}$ byte in ATQB (4$^{th}$ byte in protocol info) is not sent.

Remark: For xy=1y and xy=3y: 13$^{th}$ byte in ATQB (4$^{th}$ byte in protocol info) is allowed.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◂ | '**05 00 08**' sent with a delay equal to $t_P$ | ◂ **LT** | WUPB to poll for the CUT |
| 2 | **CUT** ▸ | ***ATQB*** *(containing PUPI)* | ▸ **LT** | ATQB |
| 3 | **CUT** ◂ | '**52**' *(Type A short frame)* sent after $t_P$ | ◂ **LT** | WUPA (polling) |
| 4 | **CUT** ◂ | **For xy=00 and 01:** '**05 00 08**' sent with a delay equal to $t_X$ in function of 'y'<br>**For xy=10 and 11:** '**05 00 18**' sent with a delay equal to $t_X$ in function of 'y'<br>**For xy=20 and 21:** '**05 00 00**' sent with a delay equal to $t_X$ in function of 'y'<br>**For xy=30 and 31: '05 00 10'** sent with a delay equal to $t_X$ in function of 'y' | ◂ **LT** | WUPB/REQB to wake up the CUT |
| 5 | **CUT** ▸ | ***ATQB*** *(containing PUPI)* | ▸ **LT** | ATQB |
| 6 | **CUT** ◂ | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◂ **LT** | PICC Reset |
| 7 | **CUT** ◂ | **For xy=00 and 01:** '**05 00 08**' sent with a delay equal to $t_P$<br>**For xy=10 and 11:** '**05 00 18**' sent with a delay equal to $t_P$<br>**For xy=20 and 21:** '**05 00 00**' sent with a delay equal to $t_P$<br>**For xy=30 and 31:** '**05 00 10**' sent with a delay equal to $t_P$ | ◂ **LT** | Last WUPB/REQB |
| 8 | **CUT** ▸ | ***ATQB*** *(containing PUPI)* | ▸ **LT** | ATQB |
| End of test: Reset PICC | | | | |

## Scenario 69: Type B Installation with polling and with PICC Reset (xy=00, 01, 10, 11, 20, 21, 30 and 31)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 5. Block Protocol Executable Tests

## 5.1. Support of the EMV CL Polling [CC001.x]

**Test Codification:**

CC001.x

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC supports the EMV CL Polling i.e. that the EMV CL PCD will select the Type A or Type B requested by the EMV application in the PICC.

**Reference Requirements:**

7.1.1.4, 8.1.1.4

**Procedure:**

Run the following scenarios.

For all subcases:
The LT starts with the carrier off.

For x=0
The LT puts the carrier on.
The LT sends a WUPA Command sequence.
The LT sends a WUPB command sequence after at least $t_{MUTE_TIME}$ measured from the end of the WUPA.
The LT sends a WUPA Command sequence after at least $t_{MUTE_TIME}$ measured from the end of the WUPB.
The LT sends a WUPB command sequence after at least $t_{MUTE=TIME}$ measured from the end of the WUPA.

For x=1
The LT puts the carrier on.
The LT sends a WUPB command sequence.
The LT sends a WUPA Command sequence after at least $t_p$ with the carrier on with no modulation.
The LT sends a WUPB command sequence after at least $t_{MUTE_TIME}$ measured from the end of the WUPA.

For x=2
The LT puts the carrier on.
The LT sends a WUPA Command sequence.
The LT sends a WUPB command sequence after at least $t_{MUTE_TIME}$ measured from the end of the WUPA.
The LT sends a NFC-F Polling (SENSF_REQ) command in an NFC-F sequence after at least $GT_F$ measured from the end of the WUPB.
The LT performs a SUSPEND during a time $t_{RESET}$ after at least $t_{MUTE_TIME}$
The LT sends a WUPA Command sequence after at least $t_p$ with the carrier on with no modulation.
The LT sends a WUPB command sequence after at least $t_{MUTE_TIME}$ measured from the end of the WUPA.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

For x=3
The LT puts the carrier on.
The LT sends a WUPA Command sequence.
The LT sends a WUPB command sequence after at least $t_{MUTE_TIME}$ measured from the end of the WUPA.
The LT performs an Optional SUSPEND during a time $t_{SUSPEND,}$ after at least $t_{MUTE_TIME}$ measured from the end of the WUPB.
The LT sends a WUPA Command sequence after at least $t_p$ with the carrier on with no modulation.
The LT sends a WUPB command sequence after at least $t_{MUTE_TIME}$ measured from the end of the WUPA and keeps the carrier on for at least $t_{MUTE_TIME}$.

**Acceptance Criteria:**

If PICC Type = A only (i.e. Type =A and not Type =B):

- The CUT responds to all the WUPA Commands
- The CUT does not respond to the WUPB Commands or to the SENSF_REQ Commands

If PICC Type = B only (i.e. Type =B and not Type =A):

- The CUT responds to all the WUPB Commands
- The CUT does not respond to the WUPA Commands or to the SENSF_REQ Commands

If PICC Type = A and B:
For x=0 and x=3

- The CUT responds to all the WUPA Commands
- The CUT does not respond to the WUPB Commands

For x=1

- The CUT responds to all the WUPB Commands
- The CUT does not respond to the WUPA Commands

For x=2

- The CUT responds to all the WUPA Commands
- The CUT does not respond to the WUPB Commands or to the SENSF_REQ Commands

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA |
| 2 | **CUT** ▶ | If PICC Type = A only or (A and B): ***ATQA*** *(no CRC_A)*<br>If PICC Type = B only: The CUT does not send any frame during Mute_Time | ▶ **LT** | ATQA for Type A PICCs<br>Mute for Type B only PICCs |
| 3 | **CUT** ◀ | '**50 00**' | ◀ **LT** | HLTA |
| 4 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent at least $t_{MUTE_TIME}$ after WUPA | ◀ **LT** | WUPB |
| 5 | **CUT** ▶ | If PICC Type = A only or (A and B): The CUT does not send any frame during Mute_Time<br>If PICC Type = B only: ***ATQB*** *(Type B frame)* | ▶ **LT** | Mute for Type A PICCs<br>ATQB for Type B only PICCs |
| 6 | **CUT** ◀ | '**52**' *(short frame)* sent at least $t_{MUTE_TIME}$ after WUPB | ◀ **LT** | WUPA |
| 7 | **CUT** ▶ | If PICC Type = A only or (A and B): ***ATQA*** *(no CRC_A)*<br>If PICC Type = B only: The CUT does not send any frame during Mute_Time | ▶ **LT** | ATQA for Type A PICCs<br>Mute for Type B only PICCs |
| 8 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent at least $t_{MUTE_TIME}$ after WUPA | ◀ **LT** | WUPB |
| 9 | **CUT** ▶ | If PICC Type = A only or (A and B): The CUT does not send any frame during Mute_Time<br>If PICC Type = B only: ***ATQB*** *(Type B frame)* | ▶ **LT** | Mute for Type A PICCs<br>ATQB for Type B only PICCs |
| End of test: Reset PICC | | | | |

**Scenario 70: Support of the EMV CL Polling (x=0)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* | ◀ **LT** | WUPB |
| 2 | **CUT** ▶ | If PICC Type = A only: The CUT does not send any frame during Mute_Time<br>If PICC Type = B only or (A and B): ***ATQB*** *(Type B frame)* | ▶ **LT** | Mute for Type A only PICCs<br>ATQB for Type B PICCs |
| 3 | **CUT** ◀ | '**52**' *(short frame)* sent at least $t_{MUTE_TIME}$ after WUPB | ◀ **LT** | WUPA |
| 4 | **CUT** ▶ | If PICC Type = A only: ***ATQA*** *(no CRC_A)*<br>If PICC Type = B only or (A and B): The CUT does not send any frame during Mute_Time | ▶ **LT** | ATQA for Type A only PICCs<br>Mute for Type B PICCs |
| 5 | **CUT** ◀ | '**50 00**' | ◀ **LT** | HLTA |
| 6 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent at least $t_{MUTE_TIME}$ after WUPA | ◀ **LT** | WUPB |
| 7 | **CUT** ▶ | If PICC Type = A only: *The CUT does not send any frame during Mute_Time*<br>If PICC Type = B only or (A and B): ***ATQB*** *(Type B frame)* | ▶ **LT** | Mute for Type A only PICCs<br>ATQB for Type B PICCs |
| 8 | **CUT** ◀ | '**52**' *(short frame)* sent at least $t_{MUTE_TIME}$ after WUPB | ◀ **LT** | WUPA |
| 9 | **CUT** ▶ | If PICC Type = A only: ***ATQA*** *(no CRC_A)*<br>If PICC Type = B only or (A and B): *The CUT does not send any frame during Mute_Time* | ▶ **LT** | ATQA for Type A only PICCs<br>Mute for Type B PICCs |
| End of test: Reset PICC | | | | |

**Scenario 71: Support of the EMV CL Polling (x=1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA |
| 2 | **CUT** ▶ | If PICC Type = A only or (A and B): ***ATQA*** *(no CRC_A)*<br>If PICC Type = B only: The CUT does not send any frame during Mute_Time | ▶ **LT** | ATQA for Type A PICCs<br>Mute for Type B only PICCs |
| 3 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent at least $t_{MUTE_TIME}$ after WUPA | ◀ **LT** | WUPB |
| 4 | **CUT** ▶ | If PICC Type = A only or (A and B): The CUT does not send any frame during Mute_Time<br>If PICC Type = B only: ***ATQB*** *(Type B frame)* | ▶ **LT** | Mute for Type A PICCs<br>ATQB for Type B only PICCs |
| 5 | **CUT** ◀ | **'00 FF FF 00 03'** *(NFC-F Frame) sent at least GTF after WUPB* | ◀ **LT** | SENSF_REQ |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 6 | **CUT** ◀ | **The LT performs a Suspend**<br>(i.e. stops sending the Carrier during a time $t_{RESET}$) | ◀ **LT** | Suspend of the Operating Field |
| 7 | **CUT** ◀ | '**52**' *(short frame)* sent at least $t_P$ after carrier on | ◀ **LT** | WUPA |
| 8 | **CUT** ▶ | If PICC Type = A only or (A and B): ***ATQA*** *(no CRC_A)*<br>If PICC Type = B only: The CUT does not send any frame during Mute_Time | ▶ **LT** | ATQA for Type A PICCs<br>Mute for Type B only PICCs |
| 9 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent at least $t_{MUTE_TIME}$ after WUPA | ◀ **LT** | WUPB |
| 10 | **CUT** ▶ | If PICC Type = A only or (A and B): The CUT does not send any frame during Mute_Time<br>If PICC Type = B only: ***ATQB*** *(Type B frame)* | ▶ **LT** | Mute for Type A PICCs<br>ATQB for Type B only PICCs |
| End of test: Reset PICC | | | | |

**Scenario 72: Support of the EMV CL Polling (x=2)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 1 | **CUT** ◀ | '**52**' *(short frame)* | ◀ **LT** | WUPA |
| 2 | **CUT** ▶ | If PICC Type = A only or (A and B): ***ATQA*** *(no CRC_A)*<br>If PICC Type = B only: The CUT does not send any frame during Mute_Time | ▶ **LT** | ATQA for Type A PICCs<br>Mute for Type B only PICCs |
| 3 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent at least $t_{MUTE_TIME}$ after WUPA | ◀ **LT** | WUPB |
| 4 | **CUT** ▶ | If PICC Type = A only or (A and B): The CUT does not send any frame during Mute_Time<br>If PICC Type = B only: ***ATQB*** *(Type B frame)* | ▶ **LT** | Mute for Type A PICCs<br>ATQB for Type B only PICCs |
| 5 | **CUT** ◀ | **The LT performs an optional SUSPEND**<br>(i.e. stops sending the Carrier during a time $t_{SUSPEND}$) | ◀ **LT** | Suspend of the Operating Field |
| 6 | **CUT** ◀ | '**52**' *(short frame)* sent at least $t_P$ after carrier on | ◀ **LT** | WUPA |
| 7 | **CUT** ▶ | If PICC Type = A only or (A and B): ***ATQA*** *(no CRC_A)*<br>If PICC Type = B only: The CUT does not send any frame during Mute_Time | ▶ **LT** | ATQA for Type A PICCs<br>Mute for Type B only PICCs |
| 8 | **CUT** ◀ | '**05 00 08**' *(Type B frame)* sent at least $t_{MUTE_TIME}$ after WUPA | ◀ **LT** | WUPB |
| 9 | **CUT** ▶ | If PICC Type = A only or (A and B): The CUT does not send any frame during Mute_Time<br>If PICC Type = B only: ***ATQB*** *(Type B frame)* | ▶ **LT** | Mute for Type A PICCs<br>ATQB for Type B only PICCs |
| End of test: Reset PICC | | | | |

**Scenario 73: Support of the EMV CL Polling (x=3)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 5.2. Reception of an empty I-Block or a R(NAK) block [CC101.x]

**Test Codification:**

CC101.x

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC correctly acknowledges a received empty I-Block (not indicating chaining) by sending either an empty I-Block or an I-Block with any applicative information.

To ensure that the PICC correctly acknowledges a received R(NAK) block with a different block number than the PICC's block number by sending an R(ACK) block.

**Reference Requirements:**

10.3.4.8; 10.3.4.10

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.

For x=0:

The LT sends an empty I-Block after reception of the ATS or ATTRIB and after reception of the response to the first C-APDU sent.

For x=1:

The LT sends a R(NAK) block after reception of the ATS or ATTRIB and after reception of the response to the first C-APDU sent.

**Acceptance Criteria:**

The CUT behaves as described in the following scenarios.

For x=0:

On reception of a empty I-Block, the CUT acknowledges it by sending either an empty I-Block or an I-Block with any applicative information.

For x=1:

On reception of a R(NAK) block with a different block number than the PICC's block number, the CUT acknowledges it by sending an R(ACK) block.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | …<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ **LT** | Basic Installation |
| 1 | **CUT** ◂ | **I(0)$_0$ [ ]** | ◂ LT | Empty I-Block |
| 2 | **CUT** ▸ | **I(0)$_0$ [ ]** | ▸ LT | I-Block with or without INF field |
| 3 | **CUT** ◂ | **I(0)$_1$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | ◂ **LT** | Select PPSE |
| 4 | **CUT** ▸ | **I(0)$_1$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | |
| 5 | **CUT** ◂ | **I(0)$_0$ [ ]** | ◂ LT | Empty I-Block |
| 6 | **CUT** ▸ | **I(0)$_0$ [ ]** | ▸ LT | I-Block with or without INF field |
| 7 | **CUT** ◂ | **I(0)$_1$ [‘00 A4 04 00’ + Lc + AID + ‘00’]** | ◂ LT | Select AID |
| 8 | **CUT** ▸ | **I(0)$_1$ [‘6F’ + n data bytes + ’90 00’]** | ▸ LT | |
| End of test: Reset PICC | | | | |

**Scenario 74: Reception of an empty I-Block or a R(NAK) block (x=0)**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | …<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ **LT** | Basic Installation |
| 1 | **CUT** ◂ | **R(NAK)$_0$** | ◂ LT | R(NAK) block |
| 2 | **CUT** ▸ | **R(ACK)$_1$** | ◂ LT | R(ACK) block |
| 3 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | ◂ **LT** | Select PPSE |
| 4 | **CUT** ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | |
| 5 | **CUT** ◂ | **R(NAK)$_1$** | ◂ LT | R(NAK) block |
| 6 | **CUT** ▸ | **R(ACK)$_0$** | ◂ LT | R(ACK) block |
| 7 | **CUT** ◂ | **I(0)$_1$ [‘00 A4 04 00’ + Lc + AID + ‘00’]** | ◂ LT | Select AID |
| 8 | **CUT** ▸ | **I(0)$_1$ [‘6F’ + n data bytes + ’90 00’]** | ▸ LT | |
| End of test: Reset PICC | | | | |

**Scenario 75: Reception of an empty I-Block or a R(NAK) (x=1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 5.3. Support of presence check procedure [CC102]

**Test Codification:**

CC102

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC correctly acknowledges R(NAK) blocks during presence check procedure.

**Reference Requirements:**

10.3.4.10

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.

Run the following scenario.

After the first I-Block exchange, the LT sends 4 R(NAK) blocks with a block number different from the block number of the last Block sent by the CUT and waits for an answer from the PICC for each R(NAK) block.

The LT initiates another I-block exchange.

After the second I-Block exchange, the LT sends 200 R(NAK) blocks with a block number different from the block number of the last Block sent by the CUT and waits for an answer from the PICC for each R(NAK) block.

When the LT sends an R(NAK) block, the LT measures the Frame Delay Time between the end of the sequence sent by the LT and the beginning of the sequence replied by the CUT.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an R(NAK) block with a block number different from the block number of the last Block sent by the CUT, the CUT acknowledges it by sending an R(ACK) block with a block number equal to the block number of the last I-block sent by the CUT before the R(NAK) block.

After an R(NAK) block, the delay between the end of a sequence with an R(NAK) block sent by the LT and the beginning of the next sequence sent by the CUT is at least $FDT_{PICC,MIN}$ (with $FDT_{PICC,MIN} = FDT_{A,PICC,MIN}$ for Type A and $FDT_{B,PICC,MIN}$ for Type B) and is at most FWT = $(256 \times 16/f_c) \times 2^{FWI}$.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | …<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ **LT** | Basic Installation |
| 1 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | ◂ **LT** | Select PPSE |
| 2 | **CUT** ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + ’**90 00’]** | ▸ **LT** | |
| 3 | **CUT** ◂ | **R(NAK)1 sent after tp** | ◂ LT | b.n. = 1 |
| 4 | **CUT** ▸ | **R(ACK)0** | ▸ LT | R(ACK) block |
| 5 | **CUT** ◂ | **R(NAK)1 sent after tp** | ◂ LT | b.n. = 1 |
| 6 | **CUT** ▸ | **R(ACK)0** | ▸ LT | R(ACK) block |
| 7 | **CUT** ◂ | **R(NAK)1 sent after tp** | ◂ LT | b.n. = 1 |
| 8 | **CUT** ▸ | **R(ACK)0** | ▸ LT | R(ACK) block |
| 9 | **CUT** ◂ | **R(NAK)$_1$** sent after $t_p$ | ◂ **LT** | b.n. = 1 |
| 10 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | R(ACK) block |
| 11 | **CUT** ◂ | **I(0)$_1$ [‘00 A4 04 00’ + Lc + AID + ‘00’]** | ◂ **LT** | Select AID |
| 12 | **CUT** ▸ | **I(0)$_1$ [‘6F’ + n data bytes + ’90 00’]** | ▸ **LT** | |
| 13 | **CUT** ◂ | **R(NAK)$_0$** sent after $t_p$ | ◂ **LT** | b.n. = 0 |
| 14 | **CUT** ▸ | **R(ACK)$_1$** | ▸ **LT** | R(ACK) block |
| | | **Repeat 200 times steps 13 and 14** | | |
| End of test | | | | |

**Scenario 76: Support of presence check procedure**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 5.4. Reception of chained I-Blocks from the PCD [CC110.x]

**Test Codification:**

CC110.x

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC correctly acknowledges a received I-Block indicating chaining by sending an R(ACK) Block.

**Reference Requirements:**

10.3.2.1; 10.3.2.3, 10.3.4.8

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.

Run the following scenarios.
x=0 for FSCI or Max_Frame_Size ≥ 2, x=1 for FSCI or Max_Frame_Size ≥ 3,
x=2 for FSCI or Max_Frame_Size ≥ 4, x=3 for FSCI or Max_Frame_Size ≥ 5,
x=4 for FSCI or Max_Frame_Size ≥ 6, x=5 for FSCI or Max_Frame_Size ≥ 7,
x=6 for FSCI or Max_Frame_Size ≥ 8, x=7 for FSCI or Max_Frame_Size ≥ 9

On reception of an I-Block not indicating chaining, the LT sends a chain of I-Blocks and observes the frames replied by the CUT.

During the Half-Duplex Block Protocol, the LT measures the Frame Delay Time between the end of the sequence sent by the LT and the beginning of the sequence replied by the CUT (the sequence replied by the CUT may be an I-Block, R-Block or an S-Block).

Remark: all the I-Blocks indicating chaining (i.e. all the chained blocks except the last block of the chain) sent by the LT are in frames having a size equal to FSC (i.e. corresponding to the value of the parameter FSCI or Max_Frame_Size)

**Acceptance Criteria:**

The CUT behaves as described in the following scenarios.

On reception of an I-Block indicating chaining, the CUT acknowledges it by sending an R(ACK) Block.

On reception of the last I-Block of a chain (first block not indicating chaining after a block indicating chaining), the CUT acknowledges it by sending an I-Block.

The delay between the end of a sequence sent by the LT that does not contain an S(WTX) Response and the beginning of the next sequence sent by the CUT is at least $FDT_{PICC,MIN}$ (with $FDT_{PICC,MIN} = FDT_{A,PICC,MIN}$ for Type A and $FDT_{B,PICC,MIN}$ for Type B) and is at most FWT = $(256 \times 16/f_c) \times 2^{FWI}$.

The delay between the end of a sequence sent by the LT that contains an S(WTX) Response and the beginning of the next sequence sent by the CUT is at least $FDT_{PICC,MIN}$ (with $FDT_{PICC,MIN} = FDT_{A,PICC,MIN}$ for Type A and $FDT_{B,PICC,MIN}$ for Type B) and is at most $FWT_{TEMP} = ((256 \times 16/f_c) \times 2^{FWI}) \times WTXM$ where WTXM is the FWT Extension Multiplier indicated in the S(WTX) Request and $(WTXM \times (256 \times 16/ f_c) \times 2^{FWI}) \leq FWT_{MAX}$.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | …<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ LT | Basic Installation |
| 1 | CUT ◂ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◂ LT | Select PPSE |
| 2 | CUT ▸ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▸ LT | |
| 3 | CUT ◂ | **I(1)$_1$ ['80 EE 00 00 FA' + '00 01 … 16 17']** | ◂ LT | 32 bytes loopback frame |
| 4 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | |
| 5 | CUT ◂ | **I(1)$_0$ [ ]** | ◂ LT | Empty I-Block |
| 6 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | |
| 7 | CUT ◂ | **I(1)$_1$ ['18 19 1A 1B … 31 32 33 34']** | ◂ LT | 32 bytes loopback frame |
| 8 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | |
| 9 | CUT ◂ | **I(1)$_0$ ['35 36 37 38 … 4E 4F 50 51']** | ◂ LT | 32 bytes loopback frame |
| 10 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | |
| 11 | CUT ◂ | **I(1)$_1$ ['52 53 54 55 … 6B 6C 6D 6E']** | ◂ LT | 32 bytes loopback frame |
| 12 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | |
| 13 | CUT ◂ | **I(1)$_0$ ['6F 70 71 72 … 88 89 8A 8B']** | ◂ LT | 32 bytes loopback frame |
| 14 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | |
| 15 | CUT ◂ | **I(1)$_1$ ['8C 8D 8E 8F … A5 A6 A7 A8']** | ◂ LT | 32 bytes loopback frame |
| 16 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | |
| 17 | CUT ◂ | **I(1)$_0$ ['A9 AA AB AC … C2 C3 C4 C5']** | ◂ LT | 32 bytes loopback frame |
| 18 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | |
| 19 | CUT ◂ | **I(1)$_1$ ['C6 C7 C8 C9 … DF E0 E1 E2']** | ◂ LT | 32 bytes loopback frame |
| 20 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | |
| 21 | CUT ◂ | **I(1)$_0$ ['E3 E4 E5 E6 … F8 F9' + '00']** | ◂ LT | Last block of the chain indicating chaining |
| 22 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | |
| 23 | CUT ◂ | **I(0)$_1$ [ ]** | ◂ LT | Empty I-Block |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| 24 | **CUT** ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ ['00 01 … F8 F9' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [ 'Status Words']** | ▸ **LT** | | Loopback data /Status Words |
| 25 | **CUT** ◂ | **I(1)$_0$ ['80 EE 00 00 17' + '00 01 … 16' + '00']** | ◂ **LT** | | 32 bytes loopback frame |
| 26 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | | |
| 27 | **CUT** ◂ | **I(0)$_1$ [ ]** | ◂ **LT** | | Empty I-Block |
| 28 | **CUT** ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ ['00 01 … 16' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [ 'Status Words']** | ▸ **LT** | | Loopback data /Status Words |
| End of test: Reset PICC | | | | | |

## Scenario 77: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 2 / x=0)

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| … | **CUT** ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ **LT** | | Basic Installation |
| 1 | **CUT** ◂ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◂ **LT** | | Select PPSE |
| 2 | **CUT** ▸ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▸ **LT** | | |
| 3 | **CUT** ◂ | **I(1)$_1$ ['80 EE 00 00 FA' + '00 01 … 1E 1F']** | ◂ **LT** | | 40 bytes loopback frame |
| 4 | **CUT** ▸ | **R(ACK)$_1$** | ▸ **LT** | | |
| 5 | **CUT** ◂ | **I(1)$_0$ [ ]** | ◂ **LT** | | Empty I-Block |
| 6 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | | |
| 7 | **CUT** ◂ | **I(1)$_1$ ['20 21 22 23 … 41 42 43 44']** | ◂ **LT** | | 40 bytes loopback frame |
| 8 | **CUT** ▸ | **R(ACK)$_1$** | ▸ **LT** | | |
| 9 | **CUT** ◂ | **I(1)$_0$ ['45 46 47 48 … 66 67 68 69']** | ◂ **LT** | | 40 bytes loopback frame |
| 10 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | | |
| 11 | **CUT** ◂ | **I(1)$_1$ ['6A 6B 6C 6D … 8B 8C 8D 8E']** | ◂ **LT** | | 40 bytes loopback frame |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 12 | **CUT ▸** | **R(ACK)$_1$** | **▸ LT** | |
| 13 | **CUT ◂** | **I(1)$_0$ ['8F 90 91 92 … B0 B1 B2 B3']** | **◂ LT** | 40 bytes loopback frame |
| 14 | **CUT ▸** | **R(ACK)$_0$** | **▸ LT** | |
| 15 | **CUT ◂** | **I(1)$_1$ ['B4 B5 B6 B7 … D5 D6 D7 D8']** | **◂ LT** | 40 bytes loopback frame |
| 16 | **CUT ▸** | **R(ACK)$_1$** | **▸ LT** | |
| 17 | **CUT ◂** | **I(1)$_0$ ['D9 DA DB DC … F8 F9' + '00']** | **◂ LT** | Last block of the chain indicating chaining |
| 18 | **CUT ▸** | **R(ACK)$_0$** | **▸ LT** | |
| 19 | **CUT ◂** | **I(0)$_1$ [ ]** | **◂ LT** | Empty I-Block |
| 20 | **CUT ▸** | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ ['00 01 … F8 F9' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [ 'Status Words']** | **▸ LT** | Loopback data /Status Words |
| 21 | **CUT ◂** | **I(1)$_0$ ['80 EE 00 00 1F' + '00 01 … 1E' + '00']** | **◂ LT** | 40 bytes loopback frame |
| 22 | **CUT ▸** | **R(ACK)$_0$** | **▸ LT** | |
| 23 | **CUT ◂** | **I(0)$_1$ [ ]** | **◂ LT** | Empty I-Block |
| 24 | **CUT ▸** | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ ['00 01 … 1E' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [ 'Status Words']** | **▸ LT** | Loopback data /Status Words |
| End of test: Reset PICC | | | | |

## Scenario 78: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 3 / x=1)

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | **CUT ▸** | …<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | **▸ LT** | Basic Installation |
| 1 | **CUT ◂** | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | **◂ LT** | Select PPSE |
| 2 | **CUT ▸** | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | **▸ LT** | |
| 3 | **CUT ◂** | **I(1)$_1$ ['80 EE 00 00 FA' + '00 01 … 26 27']** | **◂ LT** | 48 bytes loopback frame |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 4 | **CUT ▸** | **$R(ACK)_1$** | **▸ LT** | |
| 5 | **CUT ◂** | **$I(1)_0$ [ ]** | **◂ LT** | Empty I-Block |
| 6 | **CUT ▸** | **$R(ACK)_0$** | **▸ LT** | |
| 7 | **CUT ◂** | **$I(1)_1$ ['28 29 2A 2B … 51 52 53 54']** | **◂ LT** | 48 bytes loopback frame |
| 8 | **CUT ▸** | **$R(ACK)_1$** | **▸ LT** | |
| 9 | **CUT ◂** | **$I(1)_0$ ['55 56 57 58 … 7E 7F 80 81']** | **◂ LT** | 48 bytes loopback frame |
| 10 | **CUT ▸** | **$R(ACK)_0$** | **▸ LT** | |
| 11 | **CUT ◂** | **$I(1)_1$ ['82 83 84 85 … AB AC AD AE']** | **◂ LT** | 48 bytes loopback frame |
| 12 | **CUT ▸** | **$R(ACK)_1$** | **▸ LT** | |
| 13 | **CUT ◂** | **$I(1)_0$ ['AF B0 B1 B2 … D8 D9 DA DB']** | **◂ LT** | 48 bytes loopback frame |
| 14 | **CUT ▸** | **$R(ACK)_0$** | **▸ LT** | |
| 15 | **CUT ◂** | **$I(1)_1$ ['DC DD DE DF … F8 F9' + '00']** | **◂ LT** | Last block of the chain indication chaining |
| 16 | **CUT ▸** | **$R(ACK)_1$** | **▸ LT** | |
| 17 | **CUT ◂** | **$I(0)_0$ [ ]** | **◂ LT** | Empty I-Block |
| 18 | **CUT ▸** | CUT supporting Loopback Command as indicated in ICS:<br>**$I(0)_0$ ['00 01 … F8 F9' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**$I(0)_0$ [ 'Status Words']** | **▸ LT** | Loopback data /Status Words |
| 19 | **CUT ◂** | **$I(1)_1$ ['80 EE 00 00 27' + '00 01 … 26' + '00']** | **◂ LT** | 48 bytes loopback frame |
| 20 | **CUT ▸** | **$R(ACK)_1$** | **▸ LT** | |
| 21 | **CUT ◂** | **$I(0)_0$ [ ]** | **◂ LT** | Empty I-Block |
| 22 | **CUT ▸** | CUT supporting Loopback Command as indicated in ICS:<br>**$I(0)_0$ ['00 01 … 26' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**$I(0)_0$ [ 'Status Words']** | **▸ LT** | Loopback data /Status Words |
| End of test: Reset PICC | | | | |

## Scenario 79: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 4 / x=2)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| … | CUT ▸ | … <br> ***ATS*** for PICC Type = A <br> ***ATTRIB Response*** for PICC Type = B | ▸ LT | | Basic Installation |
| 1 | CUT ◂ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◂ LT | | Select PPSE |
| 2 | CUT ▸ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▸ LT | | |
| 3 | CUT ◂ | **I(1)$_1$ ['80 EE 00 00 FA' + '00 01 … 36 37']** | ◂ LT | | 64 bytes loopback frame |
| 4 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | |
| 5 | CUT ◂ | **I(1)$_0$ [ ]** | ◂ LT | | Empty I-Block |
| 6 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | |
| 7 | CUT ◂ | **I(1)$_1$ ['38 39 3A 3B … 71 72 73 74']** | ◂ LT | | 64 bytes loopback frame |
| 8 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | |
| 9 | CUT ◂ | **I(1)$_0$ ['75 76 77 78 … AE AF B0 B1']** | ◂ LT | | 64 bytes loopback frame |
| 10 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | |
| 11 | CUT ◂ | **I(1)$_1$ ['B2 B3 B4 B5 … EB EC ED EE']** | ◂ LT | | 64 bytes loopback frame |
| 12 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | |
| 13 | CUT ◂ | **I(1)$_0$ ['EF F0 F1 F2 … F8 F9' + '00']** | ◂ LT | | Last block of the chain indicating chaining |
| 14 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | |
| 15 | CUT ◂ | **I(0)$_1$ [ ]** | ◂ LT | | Empty I-Block |
| 16 | CUT ▸ | CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ ['00 01 … F8 F9' + '90 00']** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ [ 'Status Words']** | ▸ LT | | Loopback data /Status Words |
| 17 | CUT ◂ | **I(1)$_0$ ['80 EE 00 00 37' + '00 01 … 36' + '00']** | ◂ LT | | 64 bytes loopback frame |
| 18 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | |
| 19 | CUT ◂ | **I(0)$_1$ [ ]** | ◂ LT | | Empty I-Block |
| 20 | CUT ▸ | CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ ['00 01 … 36' + '90 00']** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ [ 'Status Words']** | ▸ LT | | Loopback data /Status Words |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| End of test: Reset PICC | | |

## Scenario 80: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 5 / x=3)

| Step | Exchanges | Comments |
|---|---|---|
| … | **CUT** ▸ … <br> ***ATS*** for PICC Type = A <br> ***ATTRIB Response*** for PICC Type = B ▸ **LT** | Basic Installation |
| 1 | **CUT** ◂ **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** ◂ **LT** | Select PPSE |
| 2 | **CUT** ▸ **I(0)$_0$ ['6F'** + n data bytes + **'90 00']** ▸ **LT** | |
| 3 | **CUT** ◂ **I(1)$_1$ ['80 EE 00 00 FA'** + **'00 01 … 56 57']** ◂ **LT** | 96 bytes loopback frame |
| 4 | **CUT** ▸ **R(ACK)$_1$** ▸ **LT** | |
| 5 | **CUT** ◂ **I(1)$_0$ [ ]** ◂ **LT** | Empty I-Block |
| 6 | **CUT** ▸ **R(ACK)$_0$** ▸ **LT** | |
| 7 | **CUT** ◂ **I(1)$_1$ ['58 59 5A 5B … B1 B2 B3 B4']** ◂ **LT** | 96 bytes loopback frame |
| 8 | **CUT** ▸ **R(ACK)$_1$** ▸ **LT** | |
| 9 | **CUT** ◂ **I(1)$_0$ ['B5 B6 B7 B8 … F8 F9'** + **'00']** ◂ **LT** | Last block of the chain indicating chaining |
| 10 | **CUT** ▸ **R(ACK)$_0$** ▸ **LT** | |
| 11 | **CUT** ◂ **I(0)$_1$ [ ]** ◂ **LT** | Empty I-Block |
| 12 | **CUT** ▸ CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ ['00 01 … F8 F9'** + **'90 00']** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ [ 'Status Words']** ▸ **LT** | Loopback data /Status Words |
| 13 | **CUT** ◂ **I(1)$_0$ ['80 EE 00 00 57'** + **'00 01 … 56'** + **'00']** ◂ **LT** | 96 bytes loopback frame |
| 14 | **CUT** ▸ **R(ACK)$_0$** ▸ **LT** | |
| 15 | **CUT** ◂ **I(0)$_1$ [ ]** ◂ **LT** | Empty I-Block |
| 16 | **CUT** ▸ CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ ['00 01 … 56'** + **'90 00']** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ [ 'Status Words']** ▸ **LT** | Loopback data /Status Words |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| End of test: Reset PICC | | |

## Scenario 81: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 6 / x=4)

| Step | Exchanges | Comments |
|---|---|---|
| … | **CUT** ▸ … <br> ***ATS*** for PICC Type = A <br> ***ATTRIB Response*** for PICC Type = B ▸ **LT** | Basic Installation |
| 1 | **CUT** ◂ **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00']** ◂ **LT** | Select PPSE |
| 2 | **CUT** ▸ **I(0)$_0$ ['6F' +** n data bytes + '**90 00']** ▸ **LT** | |
| 3 | **CUT** ◂ **I(1)$_1$ ['80 EE 00 00 FA'** + '**00 01 … 76 77']** ◂ **LT** | 128 bytes loopback frame |
| 4 | **CUT** ▸ **R(ACK)$_1$** ▸ **LT** | |
| 5 | **CUT** ◂ **I(1)$_0$ [ ]** ◂ **LT** | Empty I-Block |
| 6 | **CUT** ▸ **R(ACK)$_0$** ▸ **LT** | |
| 7 | **CUT** ◂ **I(1)$_1$ ['78 79 7A 7B … F1 F2 F3 F4']** ◂ **LT** | 128 bytes loopback frame |
| 8 | **CUT** ▸ **R(ACK)$_1$** ▸ **LT** | |
| 9 | **CUT** ◂ **I(1)$_0$ ['F5 F6 F7 F8 F9'** + '**00']** ◂ **LT** | Last block of the chain indicating chaining |
| 10 | **CUT** ▸ **R(ACK)$_0$** ▸ **LT** | |
| 11 | **CUT** ◂ **I(0)$_1$ [ ]** ◂ **LT** | Empty I-Block |
| 12 | **CUT** ▸ CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ ['00 01 … F8 F9'** + '**90 00']** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ [ 'Status Words']** ▸ **LT** | Loopback data /Status Words |
| 13 | **CUT** ◂ **I(1)$_0$ ['80 EE 00 00 77'** + '**00 01 … 76'** + '**00']** ◂ **LT** | 128 bytes loopback frame |
| 14 | **CUT** ▸ **R(ACK)$_0$** ▸ **LT** | |
| 15 | **CUT** ◂ **I(0)$_1$ [ ]** ◂ **LT** | Empty I-Block |
| 16 | **CUT** ▸ CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ ['00 01 … 76'** + '**90 00']** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ [ 'Status Words']** ▸ **LT** | Loopback data /Status Words |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| End of test: Reset PICC | | |

## Scenario 82: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 7 / x=5)

| Step | Exchanges | Comments |
|---|---|---|
| … | **CUT** ▸ … <br> ***ATS*** for PICC Type = A <br> ***ATTRIB Response*** for PICC Type = B ▸ **LT** | Basic Installation |
| 1 | **CUT** ◂ **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00']** ◂ **LT** | Select PPSE |
| 2 | **CUT** ▸ **I(0)$_0$ ['6F'** + n data bytes + '**90 00']** ▸ **LT** | |
| 3 | **CUT** ◂ **I(1)$_1$ ['80 EE 00 00 FA'** + '**00 01 … F6 F7']** ◂ **LT** | 256 bytes loopback frame |
| 4 | **CUT** ▸ **R(ACK)$_1$** ▸ **LT** | |
| 5 | **CUT** ◂ **I(1)$_0$ [ ]** ◂ **LT** | Empty I-Block |
| 6 | **CUT** ▸ **R(ACK)$_0$** ▸ **LT** | |
| 7 | **CUT** ◂ **I(1)$_1$ ['F8 F9'** + '**00']** ◂ **LT** | Last block of the chain indicating chaining |
| 8 | **CUT** ▸ **R(ACK)$_1$** ▸ **LT** | |
| 9 | **CUT** ◂ **I(0)$_0$ [ ]** ◂ **LT** | Empty I-Block |
| 10 | **CUT** ▸ CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_0$ ['00 01 … F8 F9'** + '**90 00']** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_0$ [ 'Status Words']** ▸ **LT** | Loopback data /Status Words |
| 11 | **CUT** ◂ **I(1)$_1$ ['80 EE 00 00 F7'** + '**00 01 … F6'** + '**00']** ◂ **LT** | 256 bytes loopback frame |
| 12 | **CUT** ▸ **R(ACK)$_1$** ▸ **LT** | |
| 13 | **CUT** ◂ **I(0)$_0$ [ ]** ◂ **LT** | Empty I-Block |
| 14 | **CUT** ▸ CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_0$ ['00 01 … F6'** + '**90 00']** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_0$ [ 'Status Words']** ▸ **LT** | Loopback data /Status Words |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | Comments |
|---|---|---|
| End of test: Reset PICC | | |

## Scenario 83: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 8 / x=6)

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | **CUT** ▸ | …<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ **LT** | Basic Installation |
| 1 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | ◂ **LT** | Select PPSE |
| 2 | **CUT** ▸ | **I(0)$_0$ [‘6F’ + n data bytes + ’90 00’]** | ▸ **LT** | |
| 3 | **CUT** ◂ | **I(0)$_1$ [‘80 EE 00 00 FA’ + ‘00 01 … F6 F7 F8 F9’+ ‘00’]** | ◂ **LT** | 259 bytes loopback frame |
| 4 | **CUT** ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [‘00 01 … F8 F9’ + ‘90 00’]**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [ ‘Status Words’]** | ▸ **LT** | Loopback data /Status Words |
| 5 | **CUT** ◂ | **I(1)$_0$ [‘80 EE 00 00 F7’ + ‘00 01 … F6’ + ‘00’]** | ◂ **LT** | 256 bytes loopback frame |
| 6 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | |
| 7 | **CUT** ◂ | **I(0)$_1$ [ ]** | ◂ **LT** | Empty I-Block |
| 8 | **CUT** ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [‘00 01 … F6’ + ‘90 00’]**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [ ‘Status Words’]** | ▸ **LT** | Loopback data /Status Words |
| End of test: Reset PICC | | | | |

## Scenario 84: Reception of chained I-Blocks from the PCD (FSCI or Max_Frame_Size ≥ 9 / x=7)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 5.5. Emission of APDUs with different FSDI values [CC120.x]

**Test Codification:**

CC120.x

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC correctly manages long R-APDUs, taking into account FSDI as received during installation.

**Reference Requirements:**

4.7.3.2, 4.7.3.4, 5.7.1.2, 6.4.1.7

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT, but uses different FSDI values depending on the subcase:

- For x=0: FSDI = ($'8'$)$_h$
- For x=1: FSDI = ($'9'$)$_h$
- For x=2: FSDI = ($'A'$)$_h$
- For x=3: FSDI = ($'B'$)$_h$
- For x=4: FSDI = ($'C'$)$_h$
- For x=5: FSDI = ($'D'$)$_h$
- For x=6: FSDI = ($'E'$)$_h$
- For x=7: FSDI = ($'F'$)$_h$

Remarks:

- The LT shall use chaining if necessary.
- During the installation, the RATS Command should be set to 'E0 X0' with X = FSDI, depending on x.
- During the installation, the ATTRIB Command should be set to '1D' + *PUPI* + '00 0X 01 00' with X = FSDI, depending on x.

**Acceptance Criteria:**

The CUT behaves as described in the following scenarios.

When the CUT sends an I-Block, the size of the I-Block is inferior or equal to FSD (e.g. 256B when FSDI=8h).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ **LT** | Basic Installation with FSDI depending on x |
| 1 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** | ◂ **LT** | Select PPSE |
| 2 | **CUT** ▸ | **I(0)$_0$ [‘6F’** + n data bytes + **’90 00’]** | ▸ **LT** | |
| 3 | **CUT** ◂ | **I(0)$_1$ [‘00 A4 04 00’** + Lc + AID + **‘00’]** | ◂ **LT** | Select AID |
| 4 | **CUT** ▸ | **I(0)$_1$ [‘6F’** + n data bytes + **’90 00’]** | ▸ **LT** | |
| **Continue C/R – APDU exchanges until transaction is completed** | | | | |
| End of test: Reset PICC | | | | |

**Scenario 85: Emission of APDUs with different FSDI values (x=0 to 7)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 5.6. Error notification on an I-Block not indicating chaining [CC203.x]

**Test Codification:**

CC203.x

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC respects the frame retransmission sequence on reception of an error notification after reception of a non chained I-Block.

**Reference Requirements:**

10.3.4.9

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.
Run the following scenario.

On reception of an I-Block not indicating chaining the LT sends an R-Block, with a block number equal to the block number of the last I-Block sent by the CUT.

For x=0: the LT sends an R(NAK) Block

For x=1: the LT sends an R(NAK) Block

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an R(NAK) Blocks sent by the LT to notify an error on an I-Block sent by the CUT, the CUT retransmits this I-Block *(and does not send a S(WTX) Request block instead of the retransmitted I-Block).*

**Pre- and post-processing with chaining:**

If the test script needs to be processed because the CUT uses chaining during the transaction, then:

- after the first I-Block of the chain, the LT shall send an R-Block.
  - For x=0: the LT sends an R(NAK) Block with a block number equal to the block number of the last I-Block sent by the CUT (i.e. first I-Block of the chain) and then accept the retransmitted I-Block.
  - For x=1: the LT sends an R(ACK) Block with a block number different from the block number of the last I-Block sent by the CUT (i.e. first I-Block of the chain) and then, after receiving the second I-Block of the chain from the CUT, the LT sends an R(ACK) Block with a block number equal to the block number of the last I-Block sent by the CUT (i.e. second I-Block of the chain) and then accept the retransmitted I-Block.

- after the last block of the chain (not indicating chaining), the LT shall send an R-Block .
  - For x=0: the LT sends an R(NAK) Block with a block number different from the block number of the last I-Block sent by the CUT (i.e. the last block of the chain), and then accepts the R(ACK) block sent in response

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

by the CUT with a block number equal to the block number of the last I-Block sent by the CUT (i.e. the last block of the chain).

- For x=1: the LT sends an R(ACK) Block with a block number equal to the block number of the last I-Block sent by the CUT (i.e. the last block of the chain) and then accept the retransmitted I-Block.

**Pre- and post-processing with S(WTX):**

If the test script needs to be processed because the CUT sends a S(WTX) Request during the transaction, the LT shall ensure that this S(WTX) Request is not sent instead of a retransmitted I-Block. Then the LT shall send an R-Block with a block number equal to the block number of the last correct I-Block or R-Block it has sent as an answer to the S(WTX) Request, the R-Block shall be sent after 2 x [$FWT_{MAX}$ + ΔFWT + $t_{RETRANSMISSION}$] =105 ms.

- For x=0: the LT sends an R(NAK) Block
- For x=1: if the last block sent by the CUT before the S(WTX) Request indicated chaining, the LT sends an R(ACK) Block. Else, the LT sends a correct S(WTX) Response and ends this pre- and post-processing (i.e. it does not send an R-Block and it does not wait for the re-transmitted S(WTX) Request – i.e. it sends the correct S(WTX) Response with no error notification).

On reception of the retransmitted S(WTX) Request, the LT shall ensure that it is identical to the previous S(WTX) Request and send a correct S(WTX) Response with the INF field from the S(WTX) Request sent by the CUT (i.e. the LT recovers from the error).

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that contains a R(NAK) Block command and the beginning of the retransmitted S(WTX) Request sent by the CUT is at least $FDT_{PICC,MIN}$ and is at most FWT = (256 x 16/$f_c$) x $2^{FWI}$.

Remark: when the CUT sends several S(WTX) Request during a single transaction, then the additional error notification test shall be performed after at least a single S(WTX) Request (the other S(WTX) Requests may be processed with no error notification from the LT).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | **CUT** ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ **LT** | Basic Installation |
| 1 | **CUT** ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** | ◂ **LT** | Select PPSE |
| 2 | **CUT** ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | |
| 3 | **CUT** ◂ | **I(0)$_1$ [’00 A4 04 00’** + Lc + AID + **‘00’]** | ◂ **LT** | Select AID |
| 4 | **CUT** ▸ | **I(0)$_1$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ **LT** | |
| 5 | **CUT** ◂ | **R(NAK)$_1$** | ◂ **LT** | Error Notification |
| 6 | **CUT** ▸ | **I(0)$_1$ [‘6F’ +** n data bytes + **’90 00’]** (same as step 4) | ▸ **LT** | |
| 7 | **CUT** ◂ | **R(NAK)$_1$** | ◂ **LT** | Error Notification (2nd) |
| 8 | **CUT** ▸ | **I(0)$_1$ [‘6F’ +** n data bytes + **’90 00’]** (same as step 4) | ▸ **LT** | |
| 9 | **CUT** ◂ | **R(NAK)$_1$** | ◂ **LT** | Error Notification (3rd) |
| 10 | **CUT** ▸ | **I(0)$_1$ [‘6F’ +** n data bytes + **’90 00’]** (same as step 4) | ▸ **LT** | |
| 11 | **CUT** ◂ | **I(0)$_0$ [C-APDU -1]** | ◂ **LT** | C-APDU -1 / b.n. = 0 |
| 12 | **CUT** ▸ | **I(0)$_0$ [R-APDU -1]** | ▸ **LT** | R-APDU -1 / b.n. = 0 |
| 13 | **CUT** ◂ | **R(NAK)$_0$** | ◂ **LT** | Error Notification |
| 14 | **CUT** ▸ | **I(0)$_0$ [R-APDU -1]** | ▸ **LT** | R-APDU -1 / b.n. = 0 |
| 15 | **CUT** ◂ | **I(0)$_1$ [C-APDU -2]** | ◂ **LT** | C-APDU -2 / b.n. = 1 |
| 16 | **CUT** ▸ | **I(0)$_1$ [R-APDU -2]** | ▸ **LT** | R-APDU -2 / b.n. = 1 |
| **Continue C/R – APDU exchanges until transaction is completed** | | | | |
| End of test: Reset PICC | | | | |

**Scenario 86: Error notification on an I-Block not indicating chaining (x=0 and 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 5.7. Error after reception of a non-chained I-Block [CC206.xy]

**Test Codification:**

CC206.xy

**ICS Parameters:**

xy=13 is Not Applicable if the PICC implements the following legacy behavior:

- PICC accepts S-Blocks (e.g. S-DESELECT) with bit b1 of PCB set to '1' (see ICS)

xy=14 is Not Applicable if the PICC implements at least one of the following legacy behaviors:

- PICC accepts I-Block with bit b6 of the PCB set to '1' (see ICS)
- PICC accepts R-Blocks with bit b2 of the PCB set to '0' (see ICS)
- PICC accepts S(WTX) Block with Power indicator different from $(00)_b$ (see ICS)

xy=15 is Not Applicable if the PICC implements at least one of the following legacy behaviors:

- PICC accepts I-Block with bit b6 of the PCB set to '1' (see ICS)
- PICC accepts R-Blocks with bit b2 of the PCB set to '0' (see ICS)
- PICC accepts S-Blocks (e.g. S(WTX) Block frame) with bit b1 of the PCB equal to '1' (see ICS)

**Test Objective:**

To ensure that the PICC behaves correctly when it receives an erroneous frame after reception of a non-chained I-Block.

**Reference Requirements:**

10.1.5.1, 10.2.1.1, 10.3.5.1, 10.3.5.2

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.

Run the following scenario.

On reception of an I-Block not indicating chaining, the LT sends an erroneous frame (when indicated in the following scenario).

After having sent the erroneous frame, the LT stays in reception mode during Mute_Time and then sends an R(NAK) Block with a block number different from the one of the last I-Block sent by the CUT.

The following errors are generated by the LT:

- xy=00: I-Block frame (not indicating chaining) with a wrong least significant bit in the first byte of the CRC
- xy=01: I-Block frame (not indicating chaining) with a wrong least significant bit in the second byte of the CRC
- xy=02: I-Block frame (not indicating chaining) with a wrong most significant bit in the second byte of the CRC
- xy=03: I-Block frame (not indicating chaining) with Parity error on any byte but the CRC bytes (only for PICC Type = A)
- xy=04: I-Block frame (not indicating chaining) with Parity error on one of the CRC bytes (only for PICC Type = A)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

- xy=05: I-Block frame (not indicating chaining) with Parity error on any byte except the CRC bytes (i.e. Parity on the data bytes) and with 2 wrong bits in one of the bytes of the CRC (only for PICC Type = A)
- xy=06: I-Block frame (not indicating chaining) with bit $b_2$ of the PCB equal to '0'
- xy=08: I-Block frame (not indicating chaining) with bit $b_7$ of the PCB equal to '1'
- xy=09: S(WTX) Response Block frame
- xy=10: I-Block frame (not indicating chaining) with bit $b_8$-$b_7$ of the PCB equal to '01'
- xy=11: S-DESELECT Block frame (not indicating chaining) with bit $b_6$-$b_5$ of the PCB equal to '01'
- xy=12: S-DESELECT Block frame (not indicating chaining) with bit $b_6$-$b_5$ of the PCB equal to '10'
- xy=13: S-DESELECT Block frame (not indicating chaining) with bit $b_1$ of the PCB equal to '1' (see "ICS parameters")
- xy=14: I-Block frame (not indicating chaining) with bit $b_6$ of the PCB equal to '1' to be performed only if the PICC does not implement some legacy behaviors (see "ICS parameters")
- xy=15: I-Block frame (not indicating chaining) with bit $b_6$ of the PCB equal to '1' to be performed only if the PICC does not implement some legacy behaviors (see "ICS parameters")

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of an R(NAK) Block with a block number equal to 0, the CUT replies with an R(ACK) Block having a PCB field equal to $('A3')_h$.

On reception of an R(NAK) Block with a block number equal to 1, the CUT replies with an R(ACK) Block having a PCB field equal to $('A2')_h$.

On reception of a valid I-Block, the CUT replies with an I-Block.

**Pre- and post-processing with chaining:**

If the test script needs to be processed because the CUT uses chaining during the transaction, then:

- the LT shall send an erroneous frame (in function of 'xy') after the first I-Block of the chain, stay in reception mode during Mute_Time, and send an R(ACK) Block with a block number different from the block number of the last I-Block sent by the CUT (i.e. the LT recovers from the error). *Remark: after an I-Block indicating chaining, the following erroneous frames replace the erroneous frames indicated above:*
    - xy=00: R(ACK) Block frame with correct block number and with a wrong least significant bit in the first byte of the CRC
    - xy=01: R(ACK) Block frame with correct block number and with a wrong least significant bit in the second byte of the CRC
    - xy=02: R(ACK) Block frame with correct block number and with a wrong most significant bit in the second byte of the CRC
    - xy=03: R(ACK) Block frame with correct block number and with Parity error on the PCB byte (only for PICC Type = A)
    - xy=04: R(ACK) Block frame with correct block number and with Parity error on one of the CRC bytes (only for PICC Type = A)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

  - xy=05: R(ACK) Block frame with correct block number and with Parity error on any byte except the CRC bytes (i.e. Parity on the data bytes) and with 2 wrong bits in one of the bytes of the CRC (only for PICC Type = A)
  - xy=06: R(ACK) Block frame with correct block number and with bit $b_6$ of the PCB equal to '0'
  - xy=08: Select PPSE frame (not indicating chaining) with a correct block number (bit $b_1$) indicated in the PCB (i.e. with a block number different from the block number of the last I-Block from the CUT)
  - xy=09: S(WTX) Response Block frame
  - xy=10: R(ACK) Block frame with correct block number and with a 1 byte INF field
  - xy=11: R(ACK) Block frame with correct block number and with bit $b_8$-$b_7$ of the PCB equal to '01'
  - xy=12: R(ACK) Block frame with correct block number and with bit $b_8$-$b_7$ of the PCB equal to '01'
  - xy=13: R(ACK) Block frame with correct block number and with a 1 byte INF field (see "ICS parameters")
  - xy=14: R(ACK) Block frame with correct block number and with bit $b_2$ of the PCB equal to '0' (see "ICS parameters")
  - xy=15: R(ACK) Block frame with correct block number and with bit $b_2$ of the PCB equal to '0' (see "ICS parameters")
- the LT shall send an erroneous frame (in function of 'xy') after the last block of the chain (not indicating chaining), stay in reception mode during Mute_Time, send an R(NAK) Block with a block number different from the block number of the last I-Block sent by the CUT and ensure that the CUT replies with a R(ACK) Block with a block number equal to the block number of the last I-Block sent by the CUT. *Remark: after the last I-Block of a chain, the erroneous frames indicated above (in the 'Procedure') are used.*

**Pre- and post-processing with S(WTX):**

If the test script needs to be processed because the CUT sends a S(WTX) Request during the transaction, then the LT shall send an erroneous frame (in function of 'xy') as an answer to the S(WTX) Request, stay in reception mode during Mute_Time, send an R(NAK) Block with a block number equal to the block number of the last correct I-Block or R-Block it has sent to the CUT and ensure that the CUT retransmits the last S(WTX) Request. Then the LT shall send a correct S(WTX) Response frame with the INF field from the S(WTX) Request sent by the CUT (i.e. the LT recovers from the error).

Remark: after an S(WTX) Request, the following erroneous frames replace the erroneous frames indicated above:

- xy=00: S(WTX) Response frame (correct WTXM) with a wrong least significant bit in the first byte of the CRC
- xy=01: S(WTX) Response frame (correct WTXM) with a wrong least significant bit in the second byte of the CRC
- xy=02: S(WTX) Response frame (correct WTXM) with a wrong most significant bit in the second byte of the CRC
- xy=03: S(WTX) Response frame (correct WTXM) with Parity error on the PCB byte (only for PICC Type = A)
- xy=04: S(WTX) Response frame (correct WTXM) with Parity error on one of the CRC bytes (only for PICC Type = A)
- xy=05: S(WTX) Response frame (correct WTXM) with Parity error on any byte except the CRC bytes (i.e. Parity on the data bytes) and with 2 wrong bits in one of the bytes of the CRC (only for PICC Type = A)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

- xy=06: S(WTX) Response frame with an incorrect WTXM (i.e. INF field different from the INF field of the FWT extension Request and different from '00')
- xy=08: S(WTX) Response frame with a 2 byte INF field
- xy=09: I-Block frame (not indicating chaining) with a correct block number (bit $b_1$) indicated in the PCB (i.e. with a block number different from the block number of the last I-Block from the CUT.
- xy=10: S(WTX) Block frame with bit $b_6$-$b_5$ of the PCB equal to '10'
- xy=11: S(WTX) Block frame with bit $b_8$-$b_7$ of the PCB equal to '01'
- xy=12: S(WTX) Block frame with bit $b_6$-$b_5$ of the PCB equal to '01'
- xy=13: S(WTX) Block frame with bit $b_6$-$b_5$ of the PCB equal to '10' (see "ICS parameters")
- xy=14: S(WTX) Block with Power indicator different from $(00)_b$ (see "ICS parameters")
- xy=15: S(WTX) Block frame with bit $b_1$ of the PCB equal to '1' (see "ICS parameters")

During the block protocol and until the PICC reset (not included), the delay between the end of a sequence sent by the LT that contains a R(NAK) Block command and the beginning of the retransmitted S(WTX) Request sent by the CUT is at least $FDT_{PICC,MIN}$ and is at most FWT = $(256 \times 16/f_c) \times 2^{FWI}$.

Remark: when the CUT sends several S(WTX) Request during a single transaction, then the additional error test shall be performed after at least a single S(WTX) Request (the other S(WTX) Requests may be processed with no error generated by the LT).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ LT | Basic Installation |
| 1 | CUT ◂ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◂ LT | Select PPSE |
| 2 | CUT ▸ | **I(0)$_0$ ['6F'** + n data bytes + **'90 00']** | ▸ LT | |
| 3 | CUT ◂ | **I(0)$_1$ ['00 A4 04 00'** + Lc + AID + **'00']** | ◂ LT | Select AID |
| 4 | CUT ▸ | **I(0)$_1$ ['6F'** + n data bytes + **'90 00']** | ▸ LT | |
| 5 | CUT ◂ | **I(0)$_0$ [C-APDU -1]** | ◂ LT | C-APDU -1 / b.n. = 0 |
| 6 | CUT ▸ | **I(0)$_0$ [R-APDU -1]** | ▸ LT | R-APDU -1/ b.n. = 0 |
| 7 | CUT ◂ | **Erroneous frame in function of 'xy'**:<br>**For xy=00 to 05**: **I(0)$_1$ [C-APDU -2] with a transmission error**<br>**For xy=06**: **I(0)$_1$ [C-APDU -2] with $b_2$ = '0' in PCB**<br>**For xy=08**: **I(0)$_1$ [C-APDU -2] with $b_7$ = '1' in PCB**<br>**For xy=09**: **'F2 01'** *(FWT extension request with M=1)*<br>**For xy=10: I(0)$_1$ [C-APDU -2] with $b_8$-$b_7$ = '01' in PCB**<br>**For xy=11: S-DESELECT('D2') with $b_6$-$b_5$ = '01' in PCB**<br>**For xy= 12: S-DESELECT('E2') with $b_6$-$b_5$ = '10' in PCB**<br>**For xy=13: S-DESELECT('C3') with $b_1$ = '1' in PCB**<br>(see "ICS parameters")<br>**For xy=14: I(0)$_1$ [C-APDU -2] with $b_6$ = '1' in PCB**<br>**For xy=15: I(0)$_1$ [C-APDU -2] with $b_6$ = '1' in PCB** | ◂ LT | erroneous frame |
| | CUT ‖ | **The CUT does not send any frame during Mute_Time** | ▸ LT | |
| 8 | CUT ◂ | R(NAK)$_1$ | ◂ LT | Error notification / b.n. = 1 |
| 9 | CUT ▸ | R(ACK)$_0$ | ▸ LT | Retrans. Req. / b.n. = 0 |
| 10 | CUT ◂ | **I(0)$_1$ [C-APDU -2]** | ◂ LT | C-APDU -2 / b.n. = 1 |
| 11 | CUT ▸ | **I(0)$_1$ [R-APDU -2]** | ▸ LT | R-APDU -2 / b.n. = 1 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 12 | CUT ◀ | **Erroneous frame in function of 'xy':**<br>**For xy=00 to 05**: **I(0)$_0$ [C-APDU -3] with a transmission error**<br>**For xy=06**: **I(0)$_0$ [C-APDU -3] with b$_2$ = '0' in PCB**<br>**For xy=08**: **I(0)$_0$ [C-APDU -3] with b$_7$ = '1' in PCBFor xy=09**: **'F2 01'** *(FWT extension request with M=1)*<br>**For xy=10: I(0)$_0$ [C-APDU -3] with b8-b7 = '01' in PCB**<br>**For xy=11:** S-DESELECT('D2') with b$_6$-b$_5$ = '01' in PCB<br>**For xy=12:** S-DESELECT('E2') with b$_6$-b$_5$ = '10' in PCB<br>**For xy=13: S-DESELECT('C3') with b$_1$ = '1' in PCB** (see "ICS parameters")<br>**For xy=14:** I(0)$_0$ [C-APDU -3] with b6 = '1' in PCB (see "ICS parameters")<br>**For xy=15:** I(0)$_0$ [C-APDU -3] with b6 = '1' in PCB (see "ICS parameters") | ◀ LT | erroneous frame |
| | CUT ‖ | **The CUT does not send any frame during Mute_Time** | ▶ LT | |
| 13 | CUT ◀ | R(NAK)$_0$ | ◀ LT | Error notification / b.n. = 0 |
| 14 | CUT ▶ | R(ACK)$_1$ | ▶ LT | Retrans. Req. / b.n. = 1 |
| 15 | CUT ◀ | **I(0)$_0$ [C-APDU -3]** | ◀ LT | C-APDU -3 / b.n. = 0 |
| 16 | CUT ▶ | **I(0)$_0$ [R-APDU -3]** | ▶ LT | R-APDU -3/ b.n. = 0 |
| End of test: Reset PICC | | | | |

**Scenario 87: Error after an I-Block not indicating chaining (xy=00 to 6 and 8 to 15)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 5.8. Management of b2 in the PCB of S-Blocks [CC207]

**Test Codification:**

CC207

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC receiving a S(DESELECT) with b2 of the PCB set to (0)b does not handle the S(Block) as a correct S(DESELECT).

To ensure that the PICC receiving a S(PARAMETERS) either considers it as a protocol error or is able to answer to it.

**Reference Requirements:**

10.1.2.4, 10.1.5.1

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.

Run the following scenario.

The LT sends an S-Block with a PCB set to ‘C0’ and no INF field.

After having sent the S-Block with a PCB='C0' and no INF Field the LT stays in reception mode during Mute_Time and then sends the next C-APDU.

Then the LT sends a S-Block with a PCB='F0' and no INF Field.

After having sent the S-Block with a PCB='F0' and no INF Field the LT stays in reception mode during Mute_Time and then continues the transaction.

**Acceptance Criteria:**

The CUT behaves as described in the scenario.

On reception of the S(DESELECT) with PCB set to 'C0’, the CUT stays mute (i.e. does not send any frame during Mute_Time).

On reception of the S(PARAMETERS) with PCB set to 'F0', the CUT either (1) stays mute or (2) sends any response.

**Pre- and post-processing with S(WTX):**

If the test script needs to be processed because the CUT sends an S(WTX) Request during the transaction, then the LT shall send a S-Block with a PCB set to ‘F0’ and with the same INF field as the one received in the S(WTX) Request. The LT shall stay in reception mode for Mute_Time and ensure that the CUT stays Mute. Then the LT shall send an R(NAK) with a block number different from the block number of the last I-Block or R-Block sent by the CUT and ensure that the CUT retransmits the last (S(WTX) Request. Then the LT shall send a correct S(WTX) Response frame with the INF field from the S(WTX) Request sent by the CUT (i.e. the LT recovers from the error).

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ LT | Basic Installation |
| 1 | CUT ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | ◂ LT | Select PPSE |
| 2 | CUT ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + ’**90 00’] with PCB =** ‘**02**’ | ▸ LT | |
| 3 | CUT ◂ | **I(0)$_1$ [’00 A4 04 00**’ + Lc + AID + ‘**00’]** | ◂ LT | Select AID / b.n. = 1 |
| 4 | CUT ▸ | **I(0)$_1$ [‘6F’ +** n data bytes + ’**90 00’]** | ▸ LT | b.n. = 1 |
| 5 | CUT ◂ | **S(DESELECT) with PCB=’ C0’** | ◂ LT | S-Block with b2 of PCB set to 0 |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 6 | CUT ◂ | **I(0)$_0$ [C-APDU -1]** | ◂ LT | C-APDU -1 / b.n. = 0 |
| 7 | CUT ▸ | **I(0)$_0$ [R-APDU -1]** | ▸ LT | R-APDU -1/ b.n. = 0 |
| 8 | CUT ◂ | **S(PARAMETERS)** | ◂ LT | S-Block with b2 of PCB set to 0 |
| 9 | CUT ▸ | The CUT does not send any frame during Mute_Time<br>Or<br>The CUT sends any response | ▸ LT | |
| **Continue C/R – APDU exchanges until transaction is completed** | | | | |
| End of test: Reset PICC | | | | |

**Scenario 88: Management of b2 in the PCB of S-Blocks**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 5.9. Error notification and error after reception of a chained I-Block [CC210.xy]

**Test Codification:**

CC210.xy

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC respects the frame retransmission sequence when it receives an error notification after reception of a chained I-Block.

To ensure that the PICC behaves correctly when it receives an erroneous frame after reception of a chained I-Block.

**Reference Requirements:**

10.3.4.10, 10.3.5.1, 10.3.5.2

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.

Run the following scenarios (in function of 'x').

- x=0 for FSCI or Max_Frame_Size = 2, x=1 for FSCI or Max_Frame_Size = 3,
- x=2 for FSCI or Max_Frame_Size = 4, x=3 for FSCI or Max_Frame_Size = 5,
- x=4 for FSCI or Max_Frame_Size = 6, x=5 for FSCI or Max_Frame_Size = 7,
- x=6 for FSCI or Max_Frame_Size ≥ 8.

On reception of an R(ACK) Block sent by the CUT to acknowledge an I-Block indicating chaining, the LT sends an R(NAK) Block with a block number equal to the block number of the last Block sent by the CUT (when indicated in the following scenario).

On reception of the retransmitted R(ACK) Block, the LT sends an I-Block indicating chaining in an erroneous frame (when indicated in the following scenario). After having sent the erroneous frame, the LT stays in reception mode during Mute_Time and then sends an R(NAK) Block with a block number different from the one of the last block sent by the CUT.

At the end of the chain, the LT sends an I-Block not indicating chaining in an erroneous frame when it shall send the last block of the chain. After having sent the erroneous frame, the LT stays in reception mode during Mute_Time and then sends an R(NAK) Block with a block number different from the one of the last block sent by the CUT.

The following transmission errors are generated by the LT (in function of 'y'):

- y=0: I-Block frame with a wrong least significant bit in the first byte of the CRC
- y=1: I-Block frame with Parity error (only for PICC Type = A)

Remarks:

- All the I-Blocks indicating chaining (i.e. all the chained blocks except the last block of the chain) sent by the LT are in frames having a size equal to FSC (i.e. corresponding to the value of the parameter FSCI or Max_Frame_Size)
- For x=6, it is not possible to send a chain of more than 2 blocks and the 2 erroneous frames described above are merged

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Acceptance Criteria:**

The CUT behaves as described in the following scenarios.

On reception of an R(NAK) Block sent by the LT to notify an error on an R(ACK) Block sent by the CUT, the CUT retransmits this R(ACK) Block (*and does not send a S(WTX) Request block instead of the retransmitted R-Block*).

On reception of an erroneous frame, the CUT stays mute (i.e. does not send any frame during Mute_Time) and then, on reception of a R(NAK) block, sends an R(ACK) Block with a block number equal to the block number of the last block sent by the CUT.

On reception of a correct I-Block not indicating chaining, the CUT replies with an I-Block.

On reception of a correct I-Block indicating chaining, the CUT replies with an R(ACK) Block.

**Pre- and post-processing with S(WTX):**

If the test script needs to be processed because the CUT sends a S(WTX) Request during the transaction, the LT shall ensure that this S(WTX) Request is not sent instead of a retransmitted R-Block.

**Scenario:**

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| … | **CUT** ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ **LT** | | Basic Installation |
| 1 | **CUT** ◂ | **I(0)$_0$ ['00 A4 04 00 0E'** + **'32 50 41 59 2E 53 59 53 2E 44 44 46 30 31'** + **'00']** | ◂ **LT** | | Select PPSE |
| 2 | **CUT** ▸ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▸ **LT** | | |
| 3 | **CUT** ◂ | **I(1)$_1$ ['80 EE 00 00 56'** + **'00 01 … 16 17']** | ◂ **LT** | | 32 bytes loopback frame |
| 4 | **CUT** ▸ | **R(ACK)$_1$** | ▸ **LT** | | |
| 5 | **CUT** ◂ | **R(NAK)$_1$** | ◂ **LT** | | Error notification |
| 6 | **CUT** ▸ | **R(ACK)$_1$** | ▸ **LT** | | Error recovery |
| 7 | **CUT** ◂ | **I(1)$_0$ ['18 19 1A 1B … 31 32 33 34']**<br>**with a transmission error in function of 'y'** | ◂ **LT** | | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | | |
| 8 | **CUT** ◂ | **R(NAK)$_0$** | ◂ **LT** | | Error notification / b.n. = 0 |
| 9 | **CUT** ▸ | **R(ACK)$_1$** | ▸ **LT** | | Retrans. Req. / b.n. = 1 |
| 10 | **CUT** ◂ | **I(1)$_0$ ['18 19 1A 1B … 31 32 33 34']** | ◂ **LT** | | Error recovery (32 bytes) |
| 11 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | | |
| 12 | **CUT** ◂ | **R(NAK)$_0$** | ◂ **LT** | | Error notification |
| 13 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | | Error recovery |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 14 | **CUT** ◀ | **I(1)$_1$ ['35 36 37 38 … 4E 4F 50 51']**<br>**with a transmission error in function of 'y'** | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 15 | **CUT** ◀ | **R(NAK)$_1$** | ◀ **LT** | Error notification / b.n. = 1 |
| 16 | **CUT** ▶ | **R(ACK)$_0$** | ▶ **LT** | Retrans. Req. / b.n. = 0 |
| 17 | **CUT** ◀ | **I(1)$_1$ ['35 36 37 38 … 4E 4F 50 51']** | ◀ **LT** | Error recovery (32 bytes) |
| 18 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | |
| 19 | **CUT** ◀ | **I(0)$_0$ ['52 53 54 55' + '00']**<br>**with a transmission error in function of 'y'** | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 20 | **CUT** ◀ | **R(NAK)$_0$** | ◀ **LT** | Error notification / b.n. = 0 |
| 21 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | Retrans. Req. / b.n. = 1 |
| 22 | **CUT** ◀ | **I(0)$_0$ ['52 53 54 55' + '00']** | ◀ **LT** | Last block of the chain |
| 23 | **CUT** ▶ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ ['00 01 … 54 55' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ [ 'Status Words']** | ▶ **LT** | Loopback data /Status Words |
| End of test: Reset PICC | | | | |

## Scenario 89: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 2 / xy=00 or 01)

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | **CUT** ▶ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▶ **LT** | Basic Installation |
| 1 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◀ **LT** | Select PPSE |
| 2 | **CUT** ▶ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▶ **LT** | |
| 3 | **CUT** ◀ | **I(1)$_1$ ['80 EE 00 00 6E' + '00 01 … 1E 1F']** | ◀ **LT** | 40 bytes loopback frame |
| 4 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | |
| 5 | **CUT** ◀ | **R(NAK)$_1$** | ◀ **LT** | Error notification |
| 6 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | Error recovery |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 7 | **CUT** ◀ | **I(1)$_0$ ['20 21 22 23 … 41 42 43 44']**<br>**with a transmission error in function of 'y'** | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 8 | **CUT** ◀ | **R(NAK)$_0$** | ◀ **LT** | Error notification / b.n. = 0 |
| 9 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | Retrans. Req. / b.n. = 1 |
| 10 | **CUT** ◀ | **I(1)$_0$ ['20 21 22 23 … 41 42 43 44']** | ◀ **LT** | Error recovery (40 bytes) |
| 11 | **CUT** ▶ | **R(ACK)$_0$** | ▶ **LT** | |
| 12 | **CUT** ◀ | **R(NAK)$_0$** | ◀ **LT** | Error notification |
| 13 | **CUT** ▶ | **R(ACK)$_0$** | ▶ **LT** | Error recovery |
| 14 | **CUT** ◀ | **I(1)$_1$ ['45 46 47 48 … 66 67 68 69']**<br>**with a transmission error in function of 'y'** | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 15 | **CUT** ◀ | **R(NAK)$_1$** | ◀ **LT** | Error notification / b.n. = 1 |
| 16 | **CUT** ▶ | **R(ACK)$_0$** | ▶ **LT** | Retrans. Req. / b.n. = 0 |
| 17 | **CUT** ◀ | **I(1)$_1$ ['45 46 47 48 … 66 67 68 69']** | ◀ **LT** | Error recovery (40 bytes) |
| 18 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | |
| 19 | **CUT** ◀ | **I(0)$_0$ ['6A 6B 6C 6D' + '00']**<br>**with a transmission error in function of 'y'** | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 20 | **CUT** ◀ | **R(NAK)$_0$** | ◀ **LT** | Error notification / b.n. = 0 |
| 21 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | Retrans. Req. / b.n. = 1 |
| 22 | **CUT** ◀ | **I(0)$_0$ ['6A 6B 6C 6D' + '00']** | ◀ **LT** | Last block of the chain |
| 23 | **CUT** ▶ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ ['00 01 … 6C 6D' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ [ 'Status Words']** | ▶ **LT** | Loopback data /Status Words |
| End of test: Reset PICC | | | | |

## Scenario 90: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 3 / xy=10 or 11)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| … | CUT ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ LT | | Basic Installation |
| 1 | CUT ◂ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◂ LT | | Select PPSE |
| 2 | CUT ▸ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▸ LT | | |
| 3 | CUT ◂ | **I(1)$_1$ ['80 EE 00 00 86' + '00 01 … 26 27']** | ◂ LT | | 48 bytes loopback frame |
| 4 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | |
| 5 | CUT ◂ | **R(NAK)$_1$** | ◂ LT | | Error notification |
| 6 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | Error recovery |
| 7 | CUT ◂ | **I(1)$_0$ ['28 29 2A 2B … 51 52 53 54']**<br>**with a transmission error in function of 'y'** | ◂ LT | | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | | |
| 8 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | | Error notification / b.n. = 0 |
| 9 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | Retrans. Req. / b.n. = 1 |
| 10 | CUT ◂ | **I(1)$_0$ ['28 29 2A 2B … 51 52 53 54']** | ◂ LT | | Error recovery (48 bytes) |
| 11 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | |
| 12 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | | Error notification |
| 13 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | Error recovery |
| 14 | CUT ◂ | **I(1)$_1$ ['55 56 57 58 … 7E 7F 80 81']**<br>**with a transmission error in function of 'y'** | ◂ LT | | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | | |
| 15 | CUT ◂ | **R(NAK)$_1$** | ◂ LT | | Error notification / b.n. = 1 |
| 16 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | Retrans. Req. / b.n. = 0 |
| 17 | CUT ◂ | **I(1)$_1$ ['55 56 57 58 … 7E 7F 80 81']** | ◂ LT | | Error recovery (48 bytes) |
| 18 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | |
| 19 | CUT ◂ | **I(0)$_0$ ['82 83 84 85' + '00']**<br>**with a transmission error in function of 'y'** | ◂ LT | | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | | |
| 20 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | | Error notification / b.n. = 0 |
| 21 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | Retrans. Req. / b.n. = 1 |
| 22 | CUT ◂ | **I(0)$_0$ ['82 83 84 85' + '00']** | ◂ LT | | Last block of the chain |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 23 | CUT ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ [‘00 01 … 84 85’ + ‘90 00’]**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ [ ‘Status Words’]** | ▸ LT | Loopback data /Status Words |
| End of test: Reset PICC | | | | |

## Scenario 91: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 4 / xy=20 or 21)

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ LT | Basic Installation |
| 1 | CUT ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | ◂ LT | Select PPSE |
| 2 | CUT ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ LT | |
| 3 | CUT ◂ | **I(1)$_1$ [‘80 EE 00 00 B6’ + ‘00 01 … 36 37’]** | ◂ LT | 64 bytes loopback frame |
| 4 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | |
| 5 | CUT ◂ | **R(NAK)$_1$** | ◂ LT | Error notification |
| 6 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | Error recovery |
| 7 | CUT ◂ | **I(1)$_0$ [’38 39 3A 3B … 71 72 73 74’]**<br>**with a transmission error in function of ‘y’** | ◂ LT | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 8 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | Error notification / b.n. = 0 |
| 9 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | Retrans. Req. / b.n. = 1 |
| 10 | CUT ◂ | **I(1)$_0$ [’38 39 3A 3B … 71 72 73 74’]** | ◂ LT | Error recovery (64 bytes) |
| 11 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | |
| 12 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | Error notification |
| 13 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | Error recovery |
| 14 | CUT ◂ | **I(1)$_1$ [’75 76 77 78 … AE AF B0 B1’]**<br>**with a transmission error in function of ‘y’** | ◂ LT | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 15 | CUT ◂ | **R(NAK)$_1$** | ◂ LT | Error notification / b.n. = 1 |
| 16 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | Retrans. Req. / b.n. = 0 |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 17 | **CUT** ◀ | **I(1)$_1$ ['75 76 77 78 … AE AF B0 B1']** | ◀ **LT** | Error recovery (64 bytes) |
| 18 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | |
| 19 | **CUT** ◀ | **I(0)$_0$ ['B2 B3 B4 B5' + '00']**<br>**with a transmission error in function of 'y'** | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 20 | **CUT** ◀ | **R(NAK)$_0$** | ◀ **LT** | Error notification / b.n. = 0 |
| 21 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | Retrans. Req. / b.n. = 1 |
| 22 | **CUT** ◀ | **I(0)$_0$ ['B2 B3 B4 B5' + '00']** | ◀ **LT** | Last block of the chain |
| 23 | **CUT** ▶ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ ['00 01 … B4 B5' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ [ 'Status Words']** | ▶ **LT** | Loopback data /Status Words |
| End of test: Reset PICC | | | | |

## Scenario 92: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 5 / xy=30 or 31)

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | **CUT** ▶ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▶ **LT** | Basic Installation |
| 1 | **CUT** ◀ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◀ **LT** | Select PPSE |
| 2 | **CUT** ▶ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▶ **LT** | |
| 3 | **CUT** ◀ | **I(1)$_1$ ['80 EE 00 00 FA' + '00 01 … 56 57']** | ◀ **LT** | 96 bytes loopback frame |
| 4 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | |
| 5 | **CUT** ◀ | **R(NAK)$_1$** | ◀ **LT** | Error notification |
| 6 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | Error recovery |
| 7 | **CUT** ◀ | **I(1)$_0$ ['58 59 5A 5B … B1 B2 B3 B4']**<br>**with a transmission error in function of 'y'** | ◀ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▶ **LT** | |
| 8 | **CUT** ◀ | **R(NAK)$_0$** | ◀ **LT** | Error notification / b.n. = 0 |
| 9 | **CUT** ▶ | **R(ACK)$_1$** | ▶ **LT** | Retrans. Req. / b.n. = 1 |
| 10 | **CUT** ◀ | **I(1)$_0$ ['58 59 5A 5B … B1 B2 B3 B4']** | ◀ **LT** | Error recovery (96 bytes) |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 11 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | |
| 12 | **CUT** ◂ | **I(0)$_1$ ['B5 B6 B7 B8 … F8 F9' + '00']**<br>**with a transmission error in function of 'y'** | ◂ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 13 | **CUT** ◂ | **R(NAK)$_1$** | ◂ **LT** | Error notification / b.n. = 1 |
| 14 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | Retrans. Req. / b.n. = 0 |
| 15 | **CUT** ◂ | **I(0)$_1$ ['B5 B6 B7 B8 … F8 F9' + '00']** | ◂ **LT** | Last block of the chain |
| 16 | **CUT** ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ ['00 01 … F8 F9' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [ 'Status Words']** | ▸ **LT** | Loopback data /Status Words |
| 17 | **CUT** ◂ | **I(1)$_0$ ['80 EE 00 00 FA' + '00 01 … 56 57']** | ◂ **LT** | 96 bytes loopback frame |
| 18 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | |
| 19 | **CUT** ◂ | **R(NAK)$_0$** | ◂ **LT** | Error notification |
| 20 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | Error recovery |
| 21 | **CUT** ◂ | **I(1)$_1$ ['58 59 5A 5B … B1 B2 B3 B4']**<br>**with a transmission error in function of 'y'** | ◂ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 22 | **CUT** ◂ | **R(NAK)$_1$** | ◂ **LT** | Error notification / b.n. = 1 |
| 23 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | Retrans. Req. / b.n. = 0 |
| 24 | **CUT** ◂ | **I(1)$_1$ ['58 59 5A 5B … B1 B2 B3 B4']** | ◂ **LT** | Error recovery (96 bytes) |
| 25 | **CUT** ▸ | **R(ACK)$_1$** | ▸ **LT** | |
| 26 | **CUT** ◂ | **I(0)$_0$ ['B5 B6 B7 B8 … F8 F9' + '00']**<br>**with a transmission error in function of 'y'** | ◂ **LT** | erroneous frame |
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 27 | **CUT** ◂ | **R(NAK)$_0$** | ◂ **LT** | Error notification / b.n. = 0 |
| 28 | **CUT** ▸ | **R(ACK)$_1$** | ▸ **LT** | Retrans. Req. / b.n. = 1 |
| 29 | **CUT** ◂ | **I(0)$_0$ ['B5 B6 B7 B8 … F8 F9' + '00']** | ◂ **LT** | Last block of the chain |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| 30 | CUT ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ [‘00 01 … F8 F9’ + ‘90 00’]**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ [ ‘Status Words’]** | ▸ LT | Loopback data /Status Words |
| End of test: Reset PICC | | | | |

## Scenario 93: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 6 / xy=40 or 41)

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ LT | Basic Installation |
| 1 | CUT ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | ◂ LT | Select PPSE |
| 2 | CUT ▸ | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | ▸ LT | |
| 3 | CUT ◂ | **I(1)$_1$ [‘80 EE 00 00 FA’ + ‘00 01 … 76 77’]** | ◂ LT | 128 bytes loopback frame |
| 4 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | |
| 5 | CUT ◂ | **R(NAK)$_1$** | ◂ LT | Error notification |
| 6 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | Error recovery |
| 7 | CUT ◂ | **I(1)$_0$ [’78 79 7A 7B … F1 F2 F3 F4’]**<br>**with a transmission error in function of ‘y’** | ◂ LT | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 8 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | Error notification / b.n. = 0 |
| 9 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | Retrans. Req. / b.n. = 1 |
| 10 | CUT ◂ | **I(1)$_0$ [’78 79 7A 7B … F1 F2 F3 F4’]** | ◂ LT | Error recovery (128 bytes) |
| 11 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | |
| 12 | CUT ◂ | **I(0)$_1$ [‘F5 F6 F7 F8 F9’ + ‘00’]**<br>**with a transmission error in function of ‘y’** | ◂ LT | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 13 | CUT ◂ | **R(NAK)$_1$** | ◂ LT | Error notification / b.n. = 1 |
| 14 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | Retrans. Req. / b.n. = 0 |
| 15 | CUT ◂ | **I(0)$_1$ [‘F5 F6 F7 F8 F9’ + ‘00’]** | ◂ LT | Last block of the chain |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| 16 | CUT ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ ['00 01 … F8 F9' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [ 'Status Words']** | ▸ LT | | Loopback data /Status Words |
| 17 | CUT ◂ | **I(1)$_0$ ['80 EE 00 00 FA' + '00 01 … 76 77']** | ◂ LT | | 128 bytes loopback frame |
| 18 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | |
| 19 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | | Error notification |
| 20 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | Error recovery |
| 21 | CUT ◂ | **I(1)$_1$ ['78 79 7A 7B … F1 F2 F3 F4']**<br>**with a transmission error in function of 'y'** | ◂ LT | | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | | |
| 22 | CUT ◂ | **R(NAK)$_1$** | ◂ LT | | Error notification / b.n. = 1 |
| 23 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | Retrans. Req. / b.n. = 0 |
| 24 | CUT ◂ | **I(1)$_1$ ['78 79 7A 7B … F1 F2 F3 F4']** | ◂ LT | | Error recovery (128 bytes) |
| 25 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | |
| 26 | CUT ◂ | **I(0)$_0$ ['F5 F6 F7 F8 F9' + '00']**<br>**with a transmission error in function of 'y'** | ◂ LT | | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | | |
| 27 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | | Error notification / b.n. = 0 |
| 28 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | Retrans. Req. / b.n. = 1 |
| 29 | CUT ◂ | **I(0)$_0$ ['F5 F6 F7 F8 F9' + '00']** | ◂ LT | | Last block of the chain |
| 30 | CUT ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ ['00 01 … F8 F9' + '90 00']**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_0$ [ 'Status Words']** | ▸ LT | | Loopback data /Status Words |
| End of test: Reset PICC | | | | | |

## Scenario 94: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 7 / xy=50 or 51)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | | Comments |
|---|---|---|---|---|---|
| … | CUT ▸ | … <br> ***ATS*** for PICC Type = A <br> ***ATTRIB Response*** for PICC Type = B | ▸ LT | | Basic Installation |
| 1 | CUT ◂ | **I(0)$_0$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | ◂ LT | | Select PPSE |
| 2 | CUT ▸ | **I(0)$_0$ [‘6F’ + n data bytes + ’90 00’]** | ▸ LT | | |
| 3 | CUT ◂ | **I(1)$_1$ [‘80 EE 00 00 FA’ + ‘00 01 … F6 F7’]** | ◂ LT | | 256 bytes loopback frame |
| 4 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | |
| 5 | CUT ◂ | **R(NAK)$_1$** | ◂ LT | | Error notification |
| 6 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | Error recovery |
| 7 | CUT ◂ | **I(0)$_0$ [‘F8 F9’ + ‘00’] <br> with a transmission error in function of ‘y’** | ◂ LT | | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | | |
| 8 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | | Error notification / b.n. = 0 |
| 9 | CUT ▸ | **R(ACK)$_1$** | ▸ LT | | Retrans. Req. / b.n. = 1 |
| 10 | CUT ◂ | **I(0)$_0$ [‘F8 F9’ + ‘00’]** | ◂ LT | | Error recovery (last block of the chain) |
| 11 | CUT ▸ | CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_0$ [‘00 01 … F8 F9’ + ‘90 00’]** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_0$ [ ‘Status Words’]** | ▸ LT | | Loopback data /Status Words |
| 12 | CUT ◂ | **I(0)$_1$ [‘80 EE 00 00 04’ + ‘00 01 02 03’ + ‘00’]** | ◂ LT | | loopback frame (to adjust the block number) |
| 13 | CUT ▸ | CUT supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ [‘00 01 02 03’ + ‘90 00’]** <br> CUT not supporting Loopback Command as indicated in ICS: <br> **I(0)$_1$ [ ‘Status Words’]** | ▸ LT | | Loopback data /Status Words |
| 14 | CUT ◂ | **I(1)$_0$ [‘80 EE 00 00 FA’ + ‘00 01 … F6 F7’]** | ◂ LT | | 256 bytes loopback frame |
| 15 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | |
| 16 | CUT ◂ | **R(NAK)$_0$** | ◂ LT | | Error notification |
| 17 | CUT ▸ | **R(ACK)$_0$** | ▸ LT | | Error recovery |
| 18 | CUT ◂ | **I(0)$_1$ [‘F8 F9’ + ‘00’] <br> with a transmission error in function of ‘y’** | ◂ LT | | erroneous frame |

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| | **CUT** ‖ | The CUT does not send any frame during Mute_Time | ▸ **LT** | |
| 19 | **CUT** ◂ | **R(NAK)$_1$** | ◂ **LT** | Error notification / b.n. = 1 |
| 20 | **CUT** ▸ | **R(ACK)$_0$** | ▸ **LT** | Retrans. Req. / b.n. = 0 |
| 21 | **CUT** ◂ | **I(0)$_1$ [‘F8 F9’ + ‘00’]** | ◂ **LT** | Error recovery (last block of the chain) |
| 22 | **CUT** ▸ | CUT supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [‘00 01 … F8 F9’ + ‘90 00’]**<br>CUT not supporting Loopback Command as indicated in ICS:<br>**I(0)$_1$ [ ‘Status Words’]** | ▸ **LT** | Loopback data /Status Words |
| End of test: Reset PICC | | | | |

## Scenario 95: Error notification and error after reception of a chained I-Block (FSCI or Max_Frame_Size = 8 / xy=60 or 61)

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

# 5.10. Block Protocol with PICC Reset [CC233.xy]

**Test Codification:**

CC233.xy

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC goes back to the IDLE state when the PCD performs a PICC Reset during the Half-Duplex Block Protocol.

**Reference Requirements:**

3.2.7.1, 3.2.8.1, 7.2.2.1, 8.2.2.1

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.

Run the following scenario (x=0 for PICC Type = A and x=1 for PICC Type = B).

On reception of an I-Block, the LT performs a PICC Reset i.e. stops sending the carrier (no RF field $H_{OV}$) for a time $t_{RESET,TEMP}$ (in function of 'y'). Then the LT sends the carrier for a time $t_P$ before sending the next sequence (WUPA Command for PICC Type = A and WUPB for PICC Type = B).

- **For y = 0**: $t_{RESET,TEMP} = t_{RESET} = 5000\mu s$
- **For y = 1**: $t_{RESET,TEMP} = t_{RESET,MAX,PCD} = 10ms$

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

For PICC Type = A, the CUT replies to the last WUPA Command with an ATQA.

For PICC Type = B, the CUT replies to the last WUPB Command with an ATQB.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | | ***…*** | | Basic Installation |
| | **CUT ▸** | ***ATS*** | **▸ LT** | |
| 1 | **CUT ◂** | **I(0)$_0$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | **◂ LT** | Select PPSE |
| 2 | **CUT ▸** | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | **▸ LT** | |
| 3 | **CUT ◂** | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET,TEMP}$ in function of y) | **◂ LT** | PICC Reset |
| 4 | **CUT ◂** | ‘**52**’ *(short frame)* sent after a delay equal to $t_P$ | **◂ LT** | Last WUPA |
| 5 | **CUT ▸** | ***ATQA*** *(no CRC_A)* | **▸ LT** | ATQA |
| End of test: Reset PICC | | | | |

**Scenario 96: Block Protocol with PICC Reset (PICC Type = A / x=0 / y=0 to 1)**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | | ***…*** | | Basic Installation |
| | **CUT ▸** | ***ATTRIB Response*** for PICC Type = B | **▸ LT** | |
| 1 | **CUT ◂** | **I(0)$_0$ [‘00 A4 04 00 0E’ + ‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’ + ‘00’]** | **◂ LT** | Select PPSE |
| 2 | **CUT ▸** | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | **▸ LT** | |
| 3 | **CUT ◂** | **The LT performs a PICC Reset**<br>(i.e. stops sending the Carrier during a time $t_{RESET,TEMP}$ in function of y) | **◂ LT** | PICC Reset |
| 4 | **CUT ◂** | ’**05 00 08**’ sent after a delay equal to $t_P$ | **◂ LT** | Last WUPB |
| 5 | **CUT ▸** | ***ATQB*** *(containing PUPI)* | **▸ LT** | ATQB |
| End of test: Reset PICC | | | | |

**Scenario 97: Block Protocol with PICC Reset (PICC Type = B / x=1 / y=0 to 1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 5.11. Installation commands during Block Protocol [CC241.x]

**Test Codification:**

CC241.x

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC ignores all installation commands received during the Half-Duplex Block Protocol.

**Reference Requirements:**

10.3.5.1, 10.3.5.2

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.

Run the following scenario (x=0 for PICC Type = A and x=1 for PICC Type = B).

The REQA command is not accepted by the PICC in PROTOCOL State.

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

During the Half-Duplex Block Protocol, the CUT stays mute on reception of the commands and sends an I-Block to acknowledge the I-Block sent by the LT directly after the installation commands.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | …<br>ATS | ▸ LT | Basic Installation |
| 1 | CUT ◂ | **I(0)$_0$ ['00 A4 04 00 0E'** + '**32 50 41 59 2E 53 59 53 2E 44 44 46 30 31**' + '**00'**] | ◂ LT | Select PPSE |
| 2 | CUT ▸ | **I(0)$_0$ ['6F' +** n data bytes + '**90 00'**] | ▸ LT | |
| 3 | CUT ◂ | '**52**' *(short frame)* | ◂ LT | WUPA |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 4 | CUT ◂ | '**93 20**' *(no CRC_A)* | ◂ LT | ANTICOLLISION CL1 |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 5 | CUT ◂ | '**93 70**' + ***UID CL1 + BCC*** | ◂ LT | SELECT CL1 |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 6 | CUT ◂ | '**50 00**' | ◂ LT | HLTA |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 7 | CUT ◂ | '**E0 80**' | ◂ LT | RATS |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 8 | CUT ◂ | **I(0)$_1$ ['00 A4 04 00'** + Lc + AID + '**00'**] | ◂ LT | Select AID |
| 9 | CUT ▸ | **I(0)$_1$ ['6F' +** n data bytes + '**90 00'**] | ▸ LT | |
| End of test: Reset PICC | | | | |

**Scenario 98: Installation commands during Block Protocol (PICC Type = A / x=0)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | | **…** | | Basic Installation |
| | **CUT ▸** | ***ATTRIB Response*** for PICC Type = B | **▸ LT** | |
| 1 | **CUT ◂** | **I(0)$_0$ [‘00 A4 04 00 0E’** + **‘32 50 41 59 2E 53 59 53 2E 44 44 46 30 31’** + **‘00’]** | **◂ LT** | Select PPSE |
| 2 | **CUT ▸** | **I(0)$_0$ [‘6F’ +** n data bytes + **’90 00’]** | **▸ LT** | |
| 3 | **CUT ◂** | **’05 00 08’** | **◂ LT** | WUPB |
| | **CUT ‖** | The CUT does not send any frame during Mute_Time | **▸ LT** | |
| 4 | **CUT ◂** | **’50’** + ***PUPI*** | **◂ LT** | HLTB |
| | **CUT ‖** | The CUT does not send any frame during Mute_Time | **▸ LT** | |
| 5 | **CUT ◂** | **’1D’** + ***PUPI*** + **’00 08 01 00’** | **◂ LT** | ATTRIB |
| | **CUT ‖** | The CUT does not send any frame during Mute_Time | **▸ LT** | |
| 6 | **CUT ◂** | **I(0)$_1$ [’00 A4 04 00’** + Lc + AID + **‘00’]** | **◂ LT** | Select AID |
| 7 | **CUT ▸** | **I(0)$_1$ [‘6F’ +** n data bytes + **’90 00’]** | **▸ LT** | |
| End of test: Reset PICC | | | | |

**Scenario 99: Installation commands during Block Protocol (PICC Type = B / x=1)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

## 5.12. Error at the beginning of the Block Protocol [CC245.x]

**Test Codification:**

CC245.x

**ICS Parameters:**

None

**Test Objective:**

To ensure that the PICC behaves correctly when it receives an erroneous frame as the first frame of the Block Protocol.

**Reference Requirements:**

10.3.5.1, 10.3.5.2

**Procedure:**

The LT initiates the test by performing a basic installation of the CUT.

Run the following scenario.

On reception of the last frame of the installation sequence (i.e. ATS frame sent by the CUT for PICC Type = A and ATTRIB Response frame sent by the CUT for PICC Type = B), the LT sends an erroneous frame.

After having sent the erroneous frame, the LT stays in reception mode during Mute_Time and then sends a correct I-Block frame.

The following errors are generated by the LT:

- x=0: I-Block frame (not indicating chaining) with a wrong most significant bit in the second byte of the CRC
- x=1: I-Block frame (not indicating chaining) with Parity error on the second byte of the CRC (only for PICC Type = A)
- x=3: RATS Command frame (Type A Frame) (Only for PICC Type = A)
- x=4: ATTRIB Command frame (Type B Frame) (Only for PICC Type = B)

**Acceptance Criteria:**

The CUT behaves as described in the following scenario.

The CUT stays mute on reception of the erroneous frame sent by the LT at the beginning of the Half-Duplex Block Protocol and sends an I-Block to acknowledge the correct I-Block frame sent by the LT directly after the erroneous frame.

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

**Scenario:**

| Step | Exchanges | | | Comments |
|---|---|---|---|---|
| … | CUT ▸ | **…**<br>***ATS*** for PICC Type = A<br>***ATTRIB Response*** for PICC Type = B | ▸ LT | Basic Installation |
| 1 | CUT ◂ | **Erroneous frame in function of 'x'**:<br>**For x=0 and 1: I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00'] with a transmission error**<br>**For x=3: 'E0 80' (RATS) (Type A Frame)**<br>**For x=4: '1D' + PUPI + '00 08 01 00' (ATTRIB) (Type B Frame)** | ◂ LT | erroneous frame |
| | CUT ‖ | The CUT does not send any frame during Mute_Time | ▸ LT | |
| 2 | CUT ◂ | **I(0)$_0$ ['00 A4 04 00 0E' + '32 50 41 59 2E 53 59 53 2E 44 44 46 30 31' + '00']** | ◂ LT | Select PPSE |
| 3 | CUT ▸ | **I(0)$_0$ ['6F' +** n data bytes + **'90 00']** | ▸ LT | |
| End of test: Reset PICC | | | | |

**Scenario 100: Error at the beginning of the Block Protocol (x=0, 1, 3 and 4)**

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.

***** END OF DOCUMENT *****

© 2009-2025 EMVCo, LLC. All rights reserved. Reproduction, distribution and other use of this document is permitted only pursuant to the applicable agreement between the user and EMVCo found at www.emvco.com. EMV® is a registered trademark or trademark of EMVCo, LLC in the United States and other countries.